



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

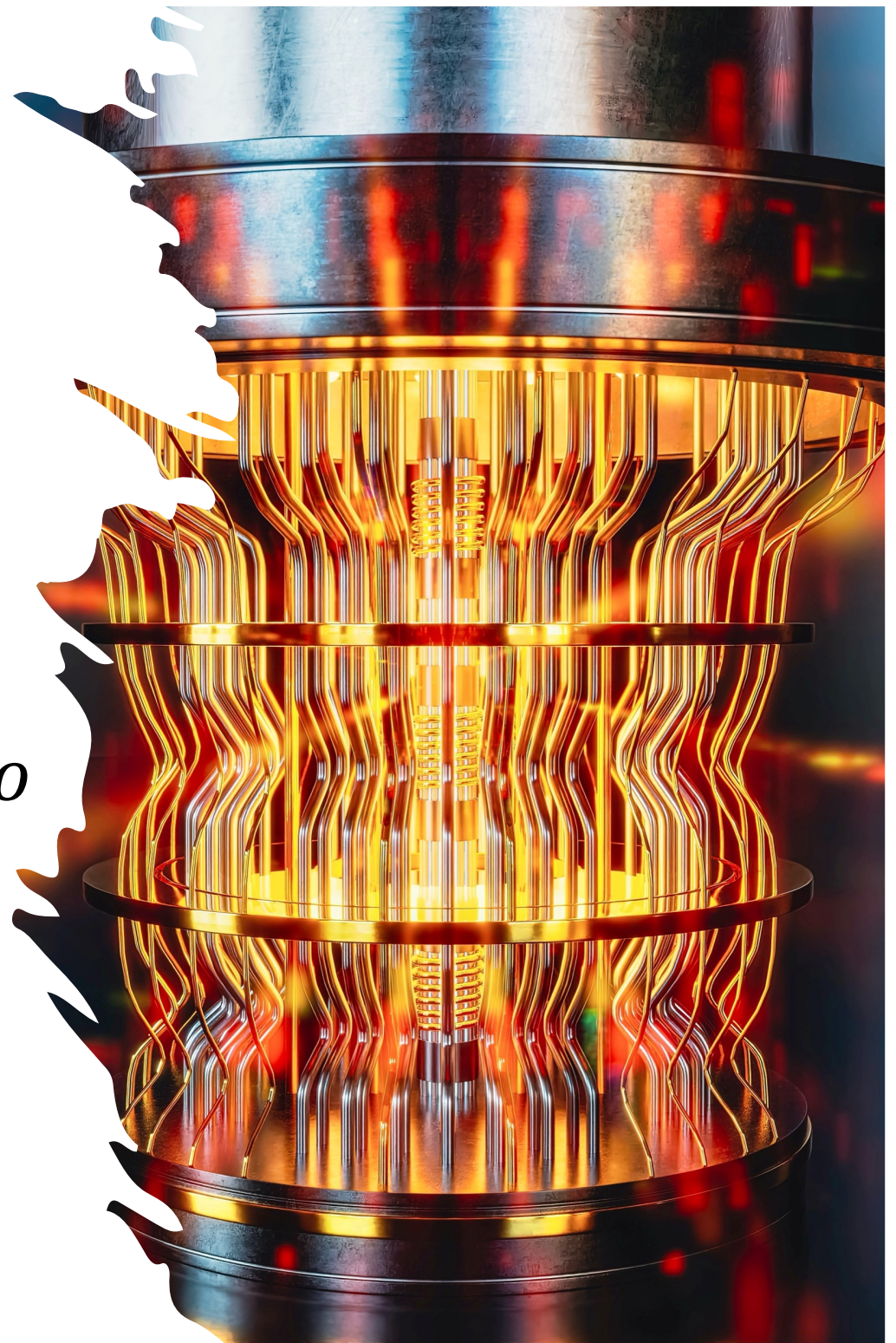
Course on Quantum Tech- nology for Security



Lecture 2 – Introduction to the main Quantum Algorithms

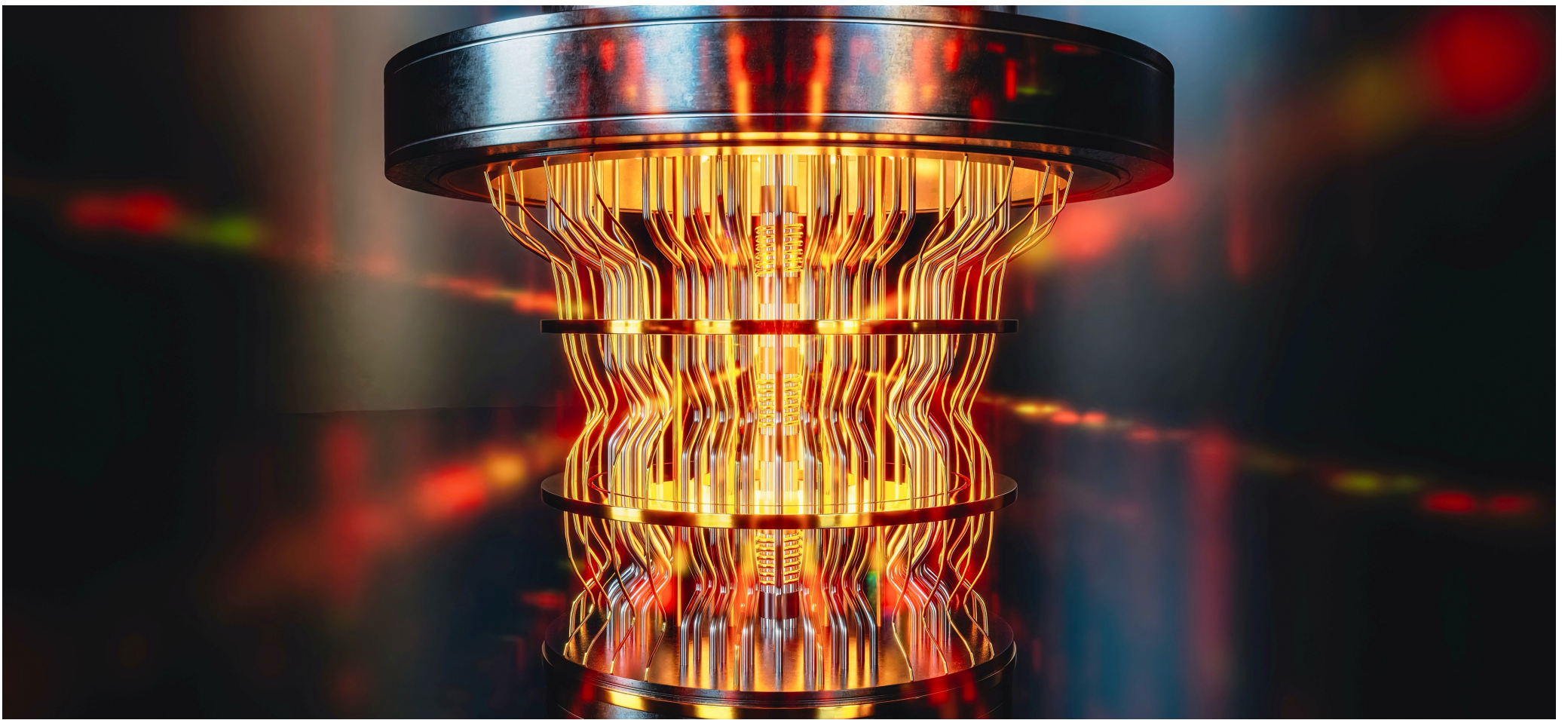


Prof. Esposito Christian



::.. Summary

- Main Quantum Algorithms
 - Deutsch's and Deutsch-Jozsa's Algorithms
 - Simon's Algorithm
 - Search - Grover's Algorithm
 - Factorization - Quantum Fourier Transform and Shor's Algorithm



Main Algorithms in Quantum Computing

::... Deutsch's Algorithm

This is one of the simplest algorithms and one of the first ones to have been formalised. It addresses this simple problem:

$$f: \{0,1\} \rightarrow \{0,1\}$$

a Boolean function f maps a binary input to a possible binary output, and the question is whether f is a constant ($f(0) = f(1)$) or a balanced function (output yields as many 0s as 1s over its input set).

We would evaluate the function for the two possible inputs and compare the outputs. If they are equal, it is a constant function, and if they vary, the function is balanced. This is equivalent to calculate the sum of the outputs modulo 2:

$$f(0) \oplus f(1)$$

In the case of a constant function, this operation returns 0 regardless of the function outputs, if the function is balanced, the output is 1.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:

$$\begin{array}{ccc} |x\rangle & \text{---} & \boxed{U_f} & \text{---} & |x\rangle \\ |y\rangle & \text{---} & & \text{---} & |y \oplus f(x)\rangle \end{array}, \quad x, y \in \{0, 1\}$$

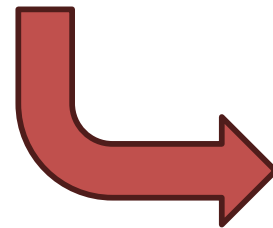
There are two qubits in input to the gate, and when they are in basis states (they are either 0 or 1), the upper qubit exists in the same state, just like the control qubit in the C-NOT gate, while the lower qubit might change its state. Specifically, if $f(x) = 0$, the lower qubit is unchanged, while if $f(x) = 1$, the state of the lower qubit is flipped.

When the basis states are coming in, the oracle leaves the basis state of the upper qubit unchanged and flips the basis state of the lower qubit only if $f(x) = 1$. In order to implement this gate, it must correspond to a unitary transformation, and there must exist a unitary transformation that does what described.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:

$$\begin{array}{ccc} |x\rangle & \text{---} & \boxed{U_f} & \text{---} & |x\rangle \\ |y\rangle & \text{---} & & \text{---} & |y \oplus f(x)\rangle \end{array}, \quad x, y \in \{0, 1\}$$



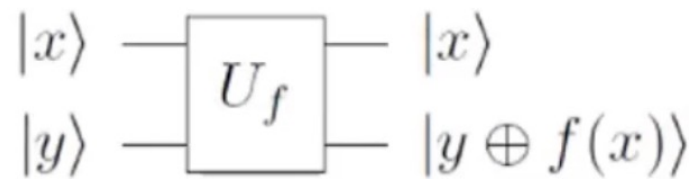
$$U_f = \begin{pmatrix} 1 - f(0) & f(0) & 0 & 0 \\ f(0) & 1 - f(0) & 0 & 0 \\ 0 & 0 & 1 - f(1) & f(1) \\ 0 & 0 & f(1) & 1 - f(1) \end{pmatrix}$$

Here's the matrix representation of this unitary transformation.

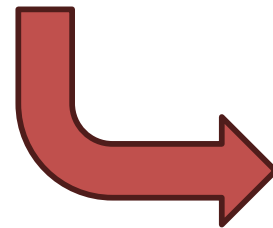
When the basis states are coming in, the oracle leaves the basis state of the upper qubit unchanged and flips the basis state of the lower qubit only if $f(x) = 1$. In order to implement this gate, it must correspond to a unitary transformation, and there must exist a unitary transformation that does what described.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:



If $f(0) = 0$, here we have the identity matrix so no changes are applied to the lower qubit.



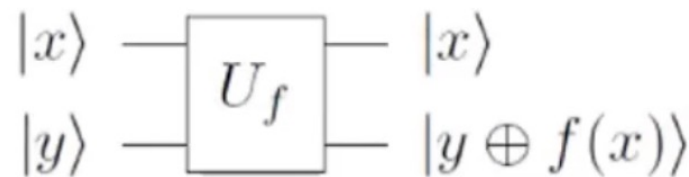
$$U_f = \begin{pmatrix} 1 - f(0) & f(0) & 0 & 0 \\ f(0) & 1 - f(0) & 0 & 0 \\ 0 & 0 & 1 - f(1) & f(1) \\ 0 & 0 & f(1) & 1 - f(1) \end{pmatrix}$$

Here's the matrix representation of this unitary transformation.

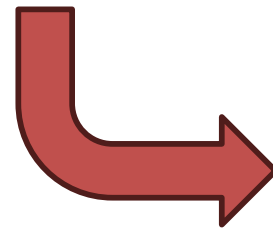
When the basis states are coming in, the oracle leaves the basis state of the upper qubit unchanged and flips the basis state of the lower qubit only if $f(x) = 1$. In order to implement this gate, it must correspond to a unitary transformation, and there must exist a unitary transformation that does what described.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:



If $f(0) = 1$, here we have the representation of the Pauli-X operation, and the lower qubit is flipped.



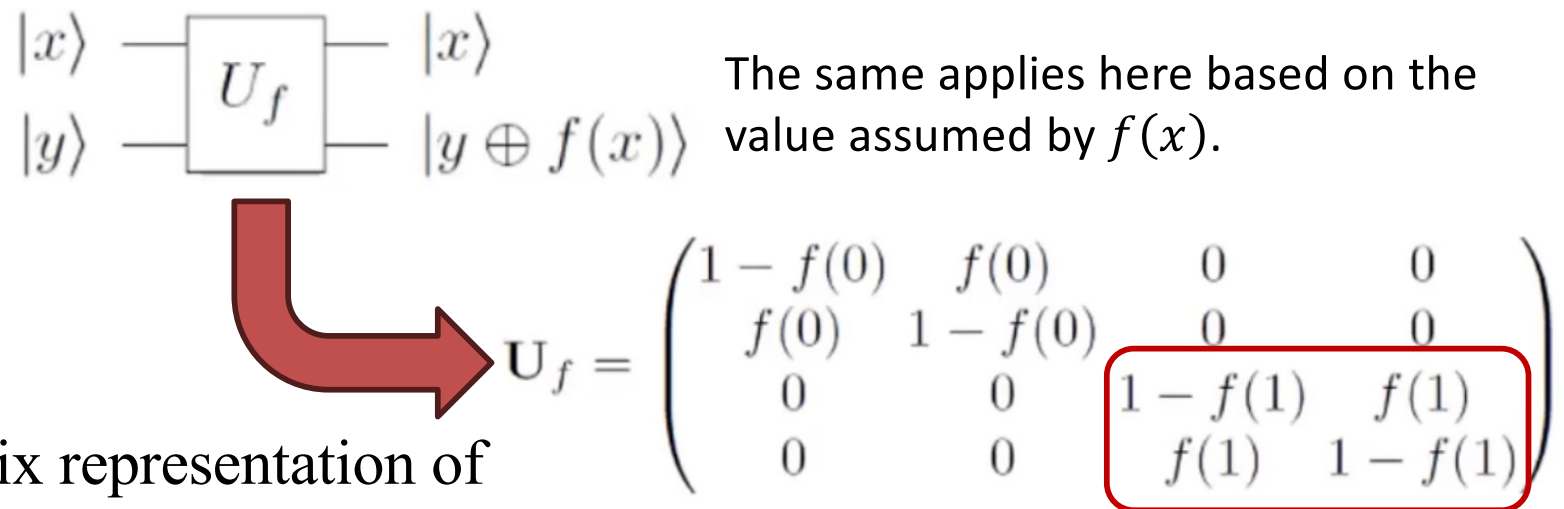
$$U_f = \begin{pmatrix} 1-f(0) & f(0) & 0 & 0 \\ f(0) & 1-f(0) & 0 & 0 \\ 0 & 0 & 1-f(1) & f(1) \\ 0 & 0 & f(1) & 1-f(1) \end{pmatrix}$$

Here's the matrix representation of this unitary transformation.

When the basis states are coming in, the oracle leaves the basis state of the upper qubit unchanged and flips the basis state of the lower qubit only if $f(x) = 1$. In order to implement this gate, it must correspond to a unitary transformation, and there must exist a unitary transformation that does what described.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:



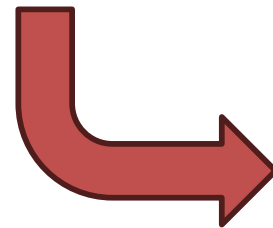
Here's the matrix representation of this unitary transformation.

When the basis states are coming in, the oracle leaves the basis state of the upper qubit unchanged and flips the basis state of the lower qubit only if $f(x) = 1$. In order to implement this gate, it must correspond to a unitary transformation, and there must exist a unitary transformation that does what described.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:

$$\begin{array}{ccc} |x\rangle & \text{---} & \boxed{U_f} & \text{---} & |x\rangle \\ |y\rangle & \text{---} & & & |y \oplus f(x)\rangle \end{array}, \quad x, y \in \{0, 1\}$$



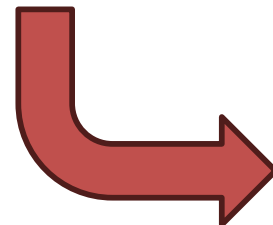
$$U_f = \begin{pmatrix} 1 - f(0) & f(0) & 0 & 0 \\ f(0) & 1 - f(0) & 0 & 0 \\ 0 & 0 & 1 - f(1) & f(1) \\ 0 & 0 & f(1) & 1 - f(1) \end{pmatrix}$$

Here's the matrix representation of this unitary transformation, and it does what expressed in the graphical representation of the circuit, but we have to show that it is unitary.

::... Deutsch's Algorithm

David Deutsch designed a little quantum circuit that performs this calculation and makes use of a particular gate called Oracle:

$$\begin{array}{ccc} |x\rangle & \text{---} & |x\rangle \\ |y\rangle & \text{---} & |y \oplus f(x)\rangle \end{array} \quad U_f, \quad x, y \in \{0, 1\}$$


$$U_f = \begin{pmatrix} 1-f(0) & f(0) & 0 & 0 \\ f(0) & 1-f(0) & 0 & 0 \\ 0 & 0 & 1-f(1) & f(1) \\ 0 & 0 & f(1) & 1-f(1) \end{pmatrix}$$

Here's the matrix representation of this unitary transformation, and it does what expressed in the graphical representation of the circuit, but we have to show that it is unitary.

The matrix is real and symmetric, and its square returns the unit matrix (depending on $f(x)$ in the two 2x2 submatrices we have the unit matrix or the Pauli X and in both cases the square is the unit matrix). By definition, this is a matricial representation of a unitary transformation.

::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state.

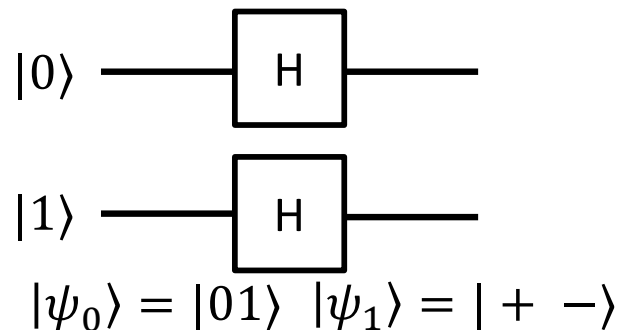
$|0\rangle$ —

$|1\rangle$ —

$|\psi_0\rangle = |01\rangle$

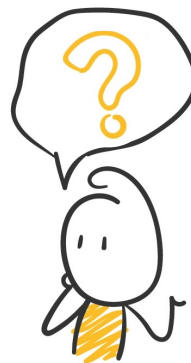
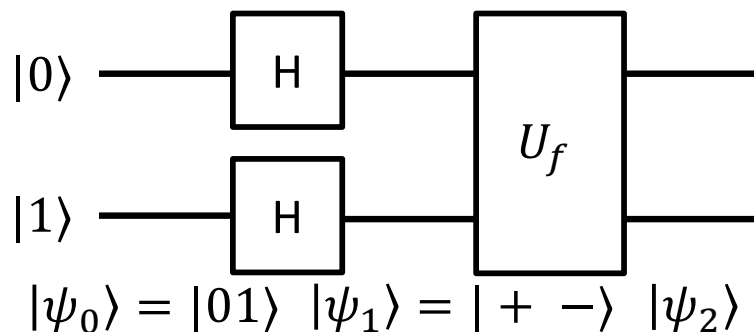
::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$.



::... Deutsch's Algorithm

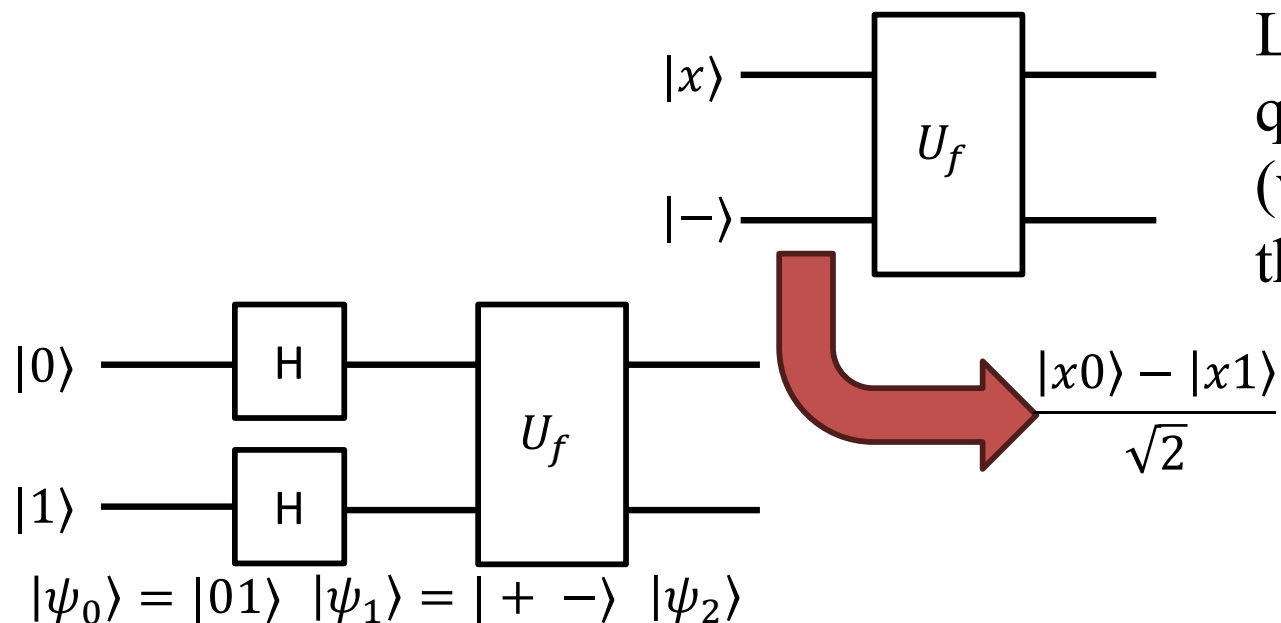
Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



What does the oracle do?
What is the expression for $|\psi_2\rangle$?

::... Deutsch's Algorithm

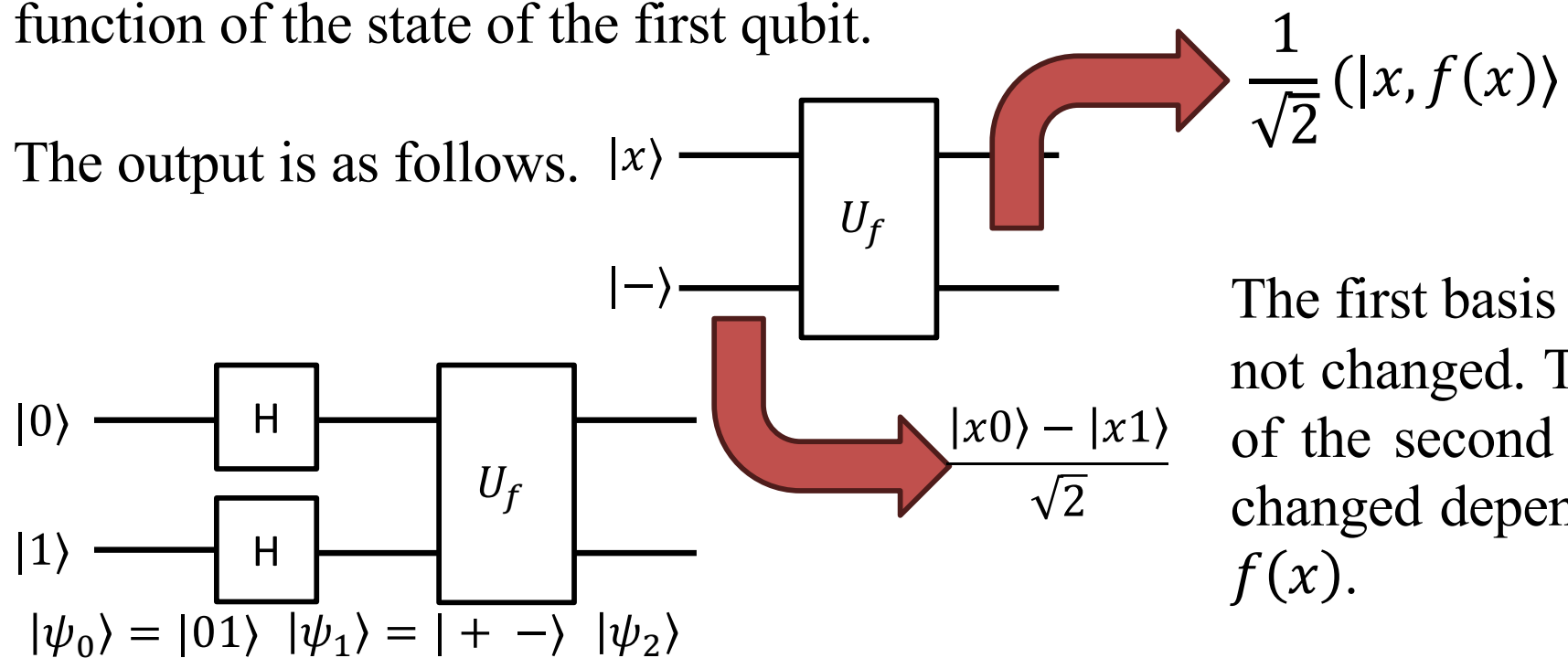
Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



Let us consider the first qubit into a basis state x (which is 0 or 1), while the second is

... Deutsch's Algorithm

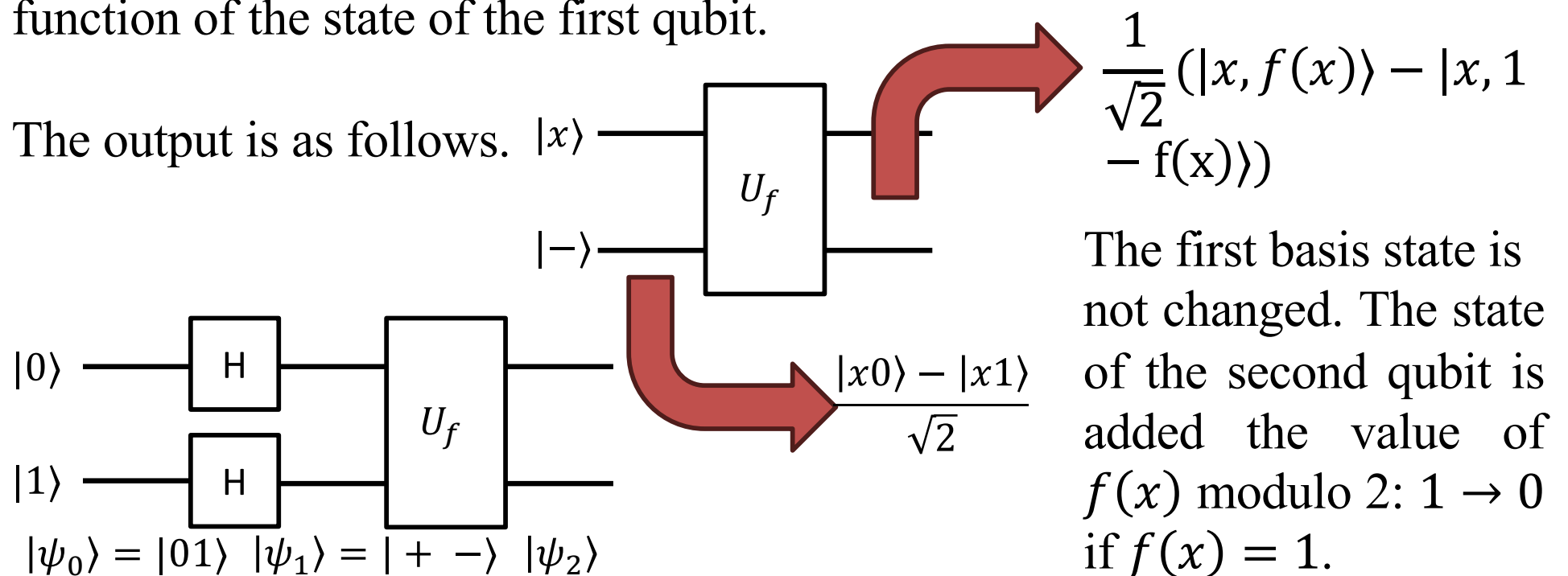
Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



The first basis state is not changed. The state of the second qubit is changed depending on $f(x)$.

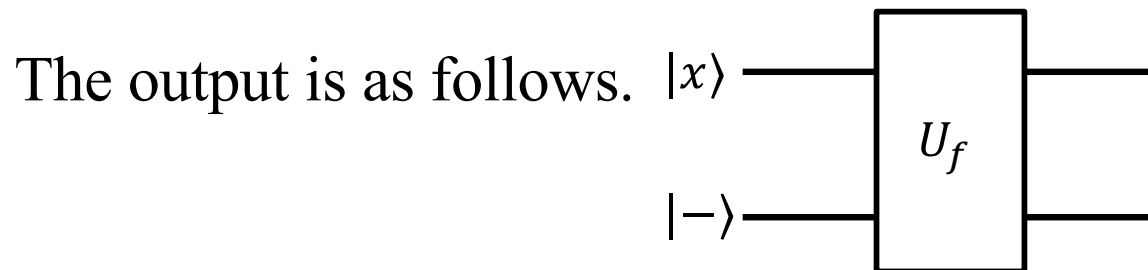
::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



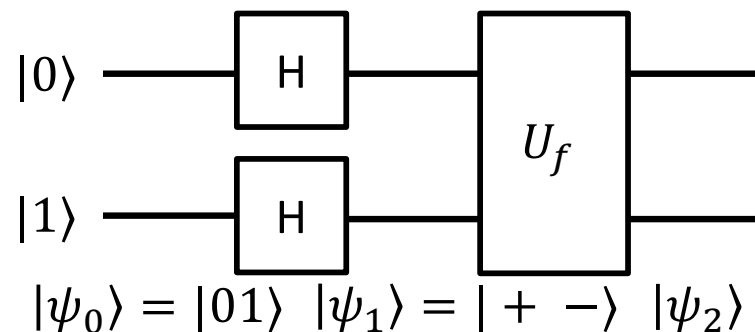
::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



$$\frac{1}{\sqrt{2}} \begin{cases} |x0\rangle - |x1\rangle & : f(x) = 0 \\ |x1\rangle - |x0\rangle & : f(x) = 1 \end{cases}$$

There are two possibilities based on the value of the function.

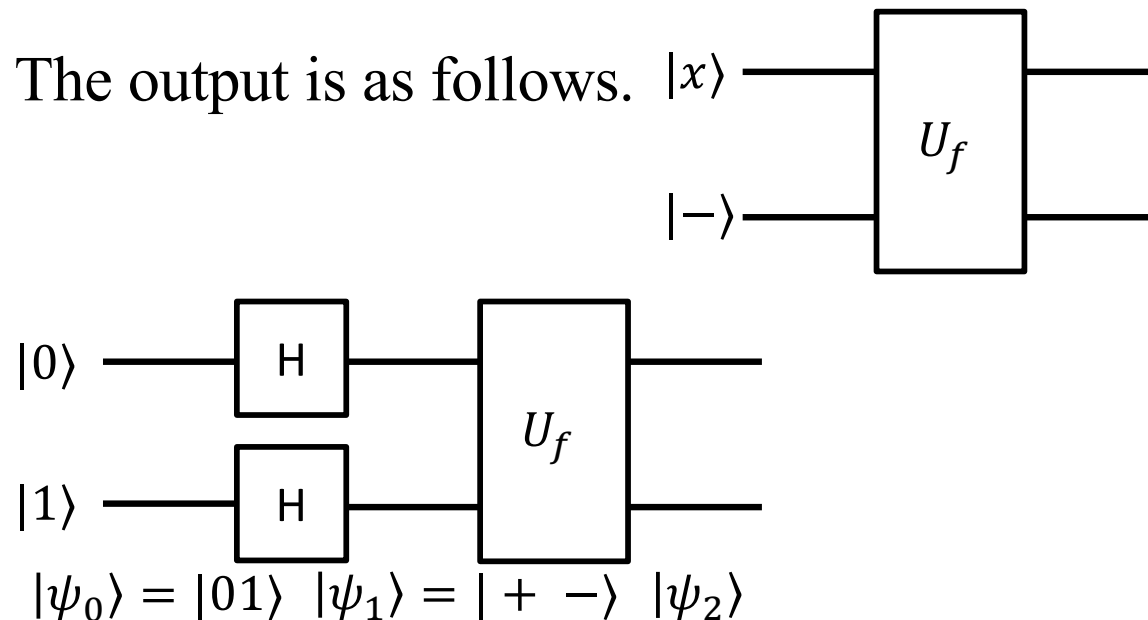


... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.

$$\begin{aligned} |x-\rangle & : f(x) = 0 \\ -|x-\rangle & : f(x) = 1 \end{aligned}$$

The output is as follows.



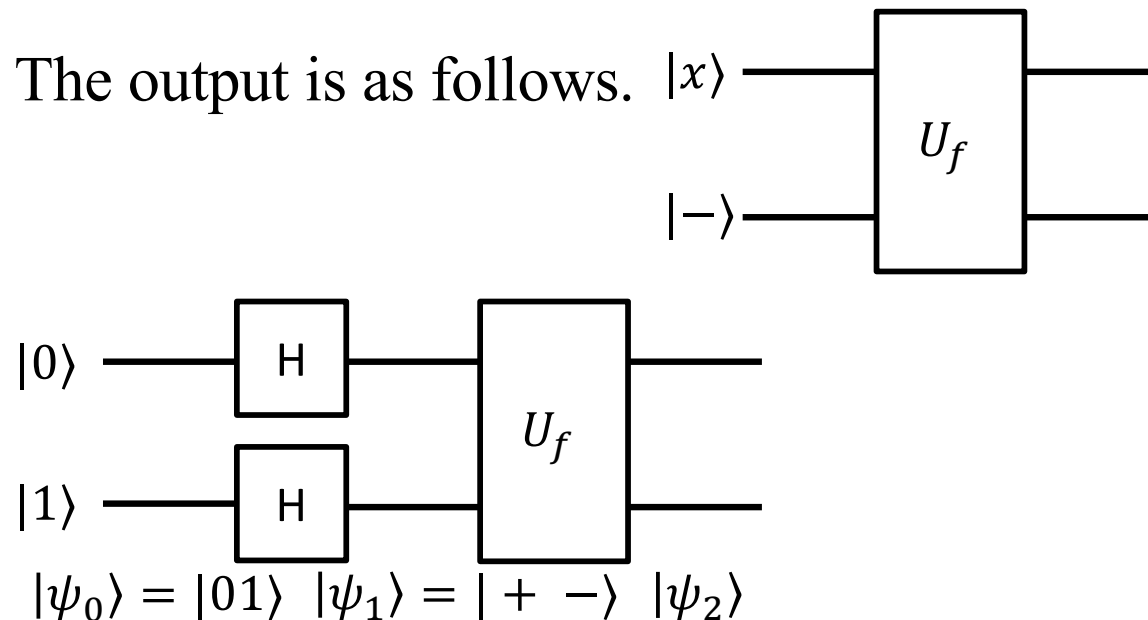
There are two possibilities based on the value of the function.

... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.

$$(-1)^{f(x)}|x-\rangle$$

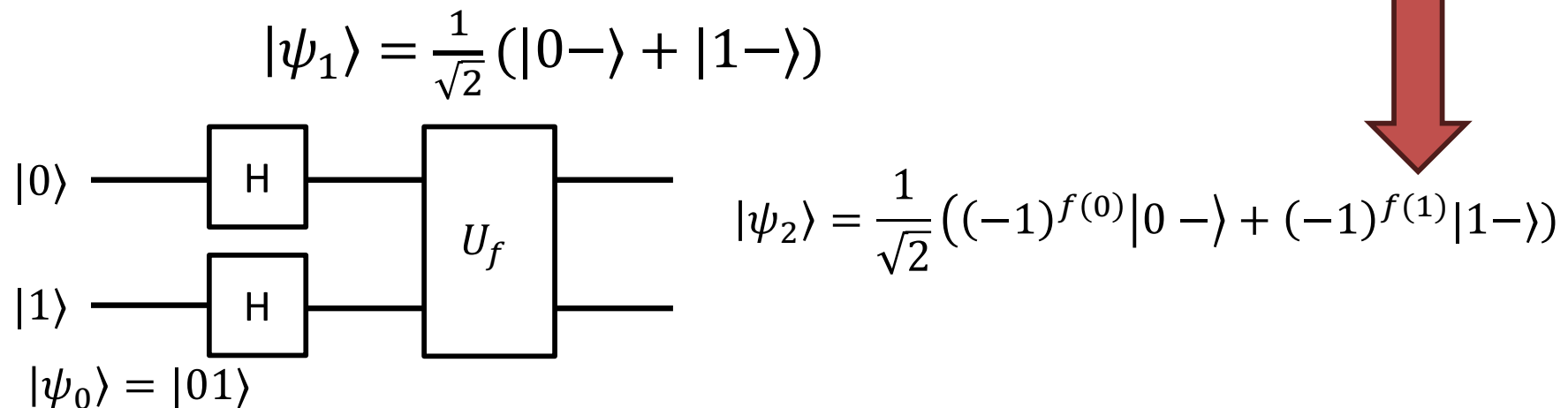
The output is as follows.



This is the output of the oracle.

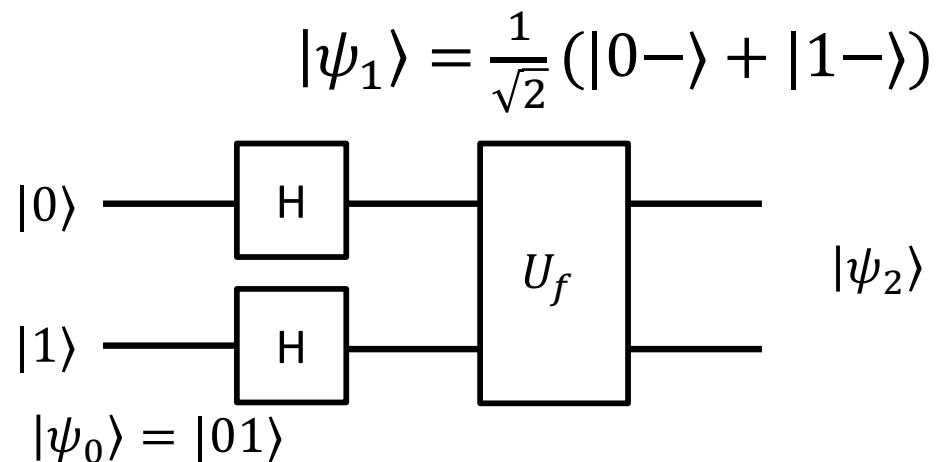
::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enters the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



$$|\psi_2\rangle = \frac{1}{\sqrt{2}} \left\{ \begin{array}{l} : f(0) = f(1) \\ : f(0) \neq f(1) \end{array} \right.$$

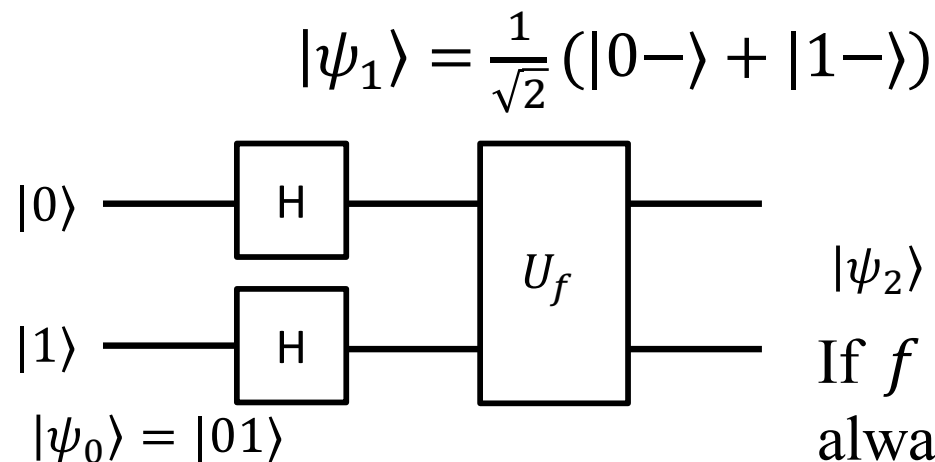
$$\frac{1}{\sqrt{2}} ((-1)^{f(0)}|0-\rangle + (-1)^{f(1)}|1-\rangle)$$

Let's consider two cases depending if the function is constant or balanced:



... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



$$\frac{1}{\sqrt{2}} ((-1)^{f(0)} |0-\rangle + (-1)^{f(1)} |1-\rangle)$$

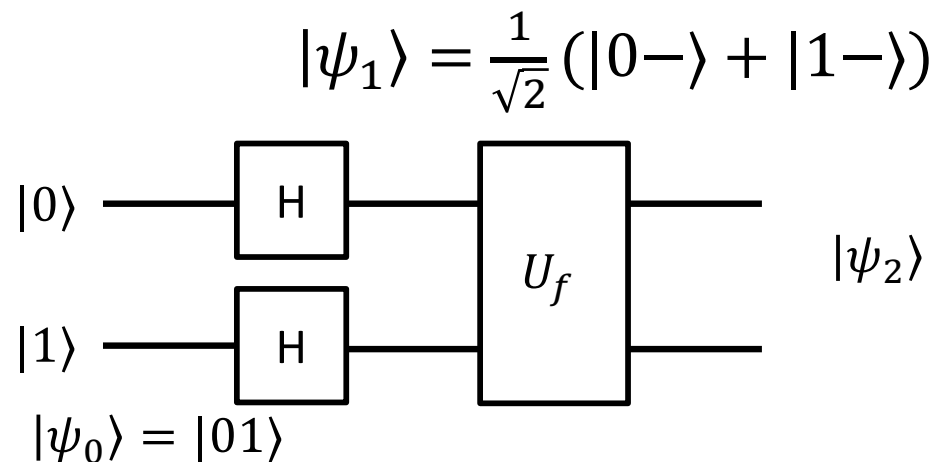
Let's consider two cases depending if the function is constant or balanced:

$$|\psi_2\rangle = \pm \frac{1}{\sqrt{2}} \begin{cases} : f(0) = f(1) \\ : f(0) \neq f(1) \end{cases}$$

If f is constant these have the same sign, always + or always -.

::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enters the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



$$|\psi_2\rangle = \pm \frac{1}{\sqrt{2}} \begin{cases} |0-\rangle + |1-\rangle & : f(0) = f(1) \\ |0-\rangle - |1-\rangle & : f(0) \neq f(1) \end{cases}$$

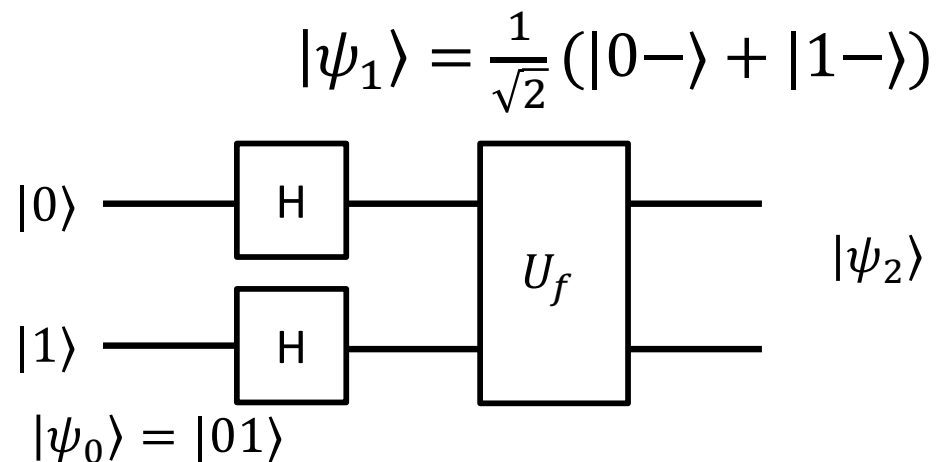
$$\frac{1}{\sqrt{2}} ((-1)^{f(0)}|0-\rangle + (-1)^{f(1)}|1-\rangle)$$

Let's consider two cases depending if the function is constant or balanced:



::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enters the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



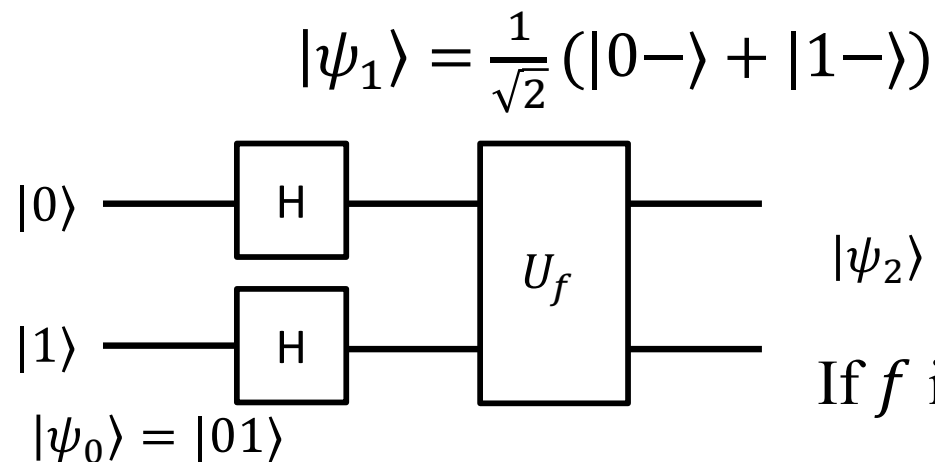
$$\frac{1}{\sqrt{2}} ((-1)^{f(0)}|0-\rangle + (-1)^{f(1)}|1-\rangle)$$

Let's consider two cases depending if the function is constant or balanced:

$$|\psi_2\rangle = \pm \frac{1}{\sqrt{2}} \begin{cases} (|0\rangle + |1\rangle) \otimes |-\rangle & : f(0) = f(1) \\ (|0\rangle - |1\rangle) \otimes |-\rangle & : f(0) \neq f(1) \end{cases}$$

::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enters the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



$$\frac{1}{\sqrt{2}} ((-1)^{f(0)} |0-\rangle + (-1)^{f(1)} |1-\rangle)$$

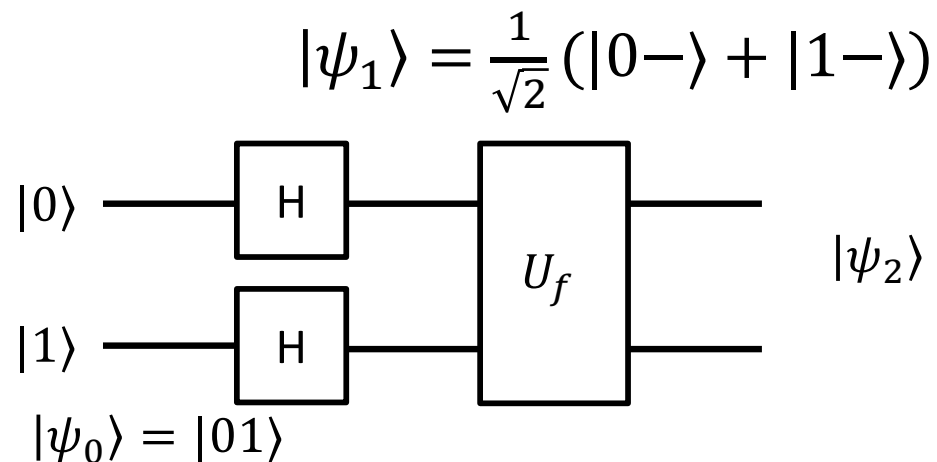
Let's consider two cases depending if the function is constant or balanced:

$$|\psi_2\rangle = \pm \frac{1}{\sqrt{2}} \begin{cases} (|0\rangle + |1\rangle) \otimes |-\rangle & : f(0) = f(1) \\ (|0\rangle - |1\rangle) \otimes |-\rangle & : f(0) \neq f(1) \end{cases}$$

If f is balanced, these have opposing signs.

::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.



$$\frac{1}{\sqrt{2}} ((-1)^{f(0)}|0-\rangle + (-1)^{f(1)}|1-\rangle)$$

Let's consider two cases depending if the function is constant or balanced:

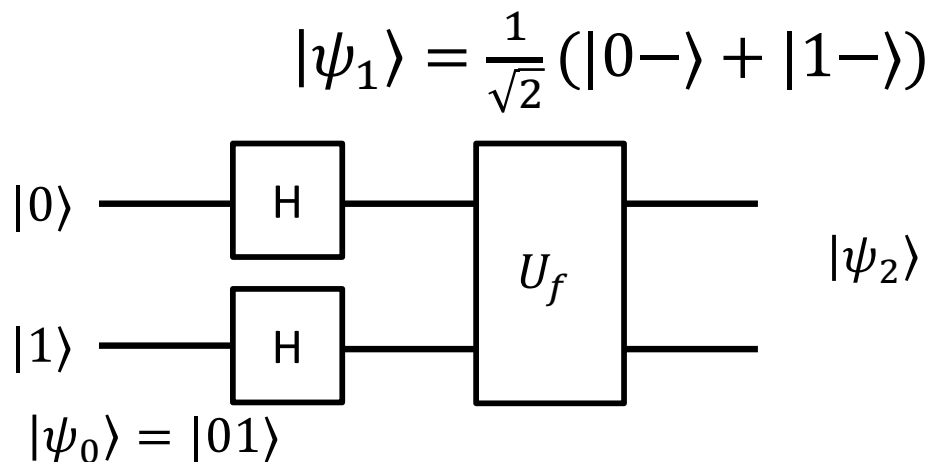
$$|\psi_2\rangle = \pm \frac{1}{\sqrt{2}} \begin{cases} (|0\rangle + |1\rangle) \otimes |-\rangle & : f(0) = f(1) \\ (|0\rangle - |1\rangle) \otimes |-\rangle & : f(0) \neq f(1) \end{cases}$$

::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enters the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit.

$$\frac{1}{\sqrt{2}} ((-1)^{f(0)}|0-\rangle + (-1)^{f(1)}|1-\rangle)$$

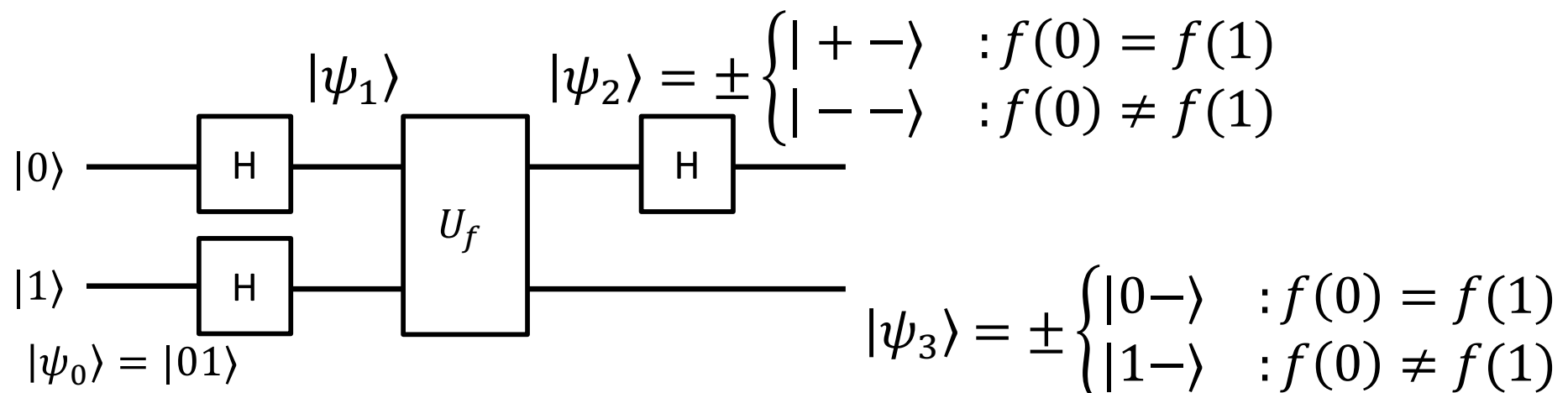
Let's consider two cases depending if the function is constant or balanced:



$$|\psi_2\rangle = \pm \begin{cases} |+-\rangle & : f(0) = f(1) \\ |--\rangle & : f(0) \neq f(1) \end{cases}$$

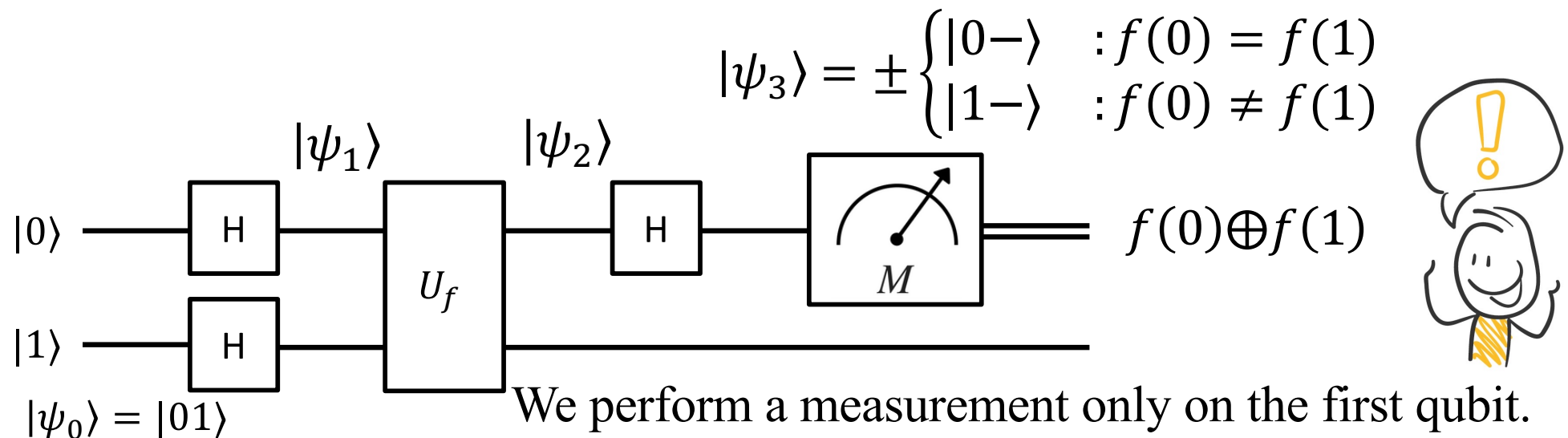
... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enter the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit. We add on the upper qubit an Hadamard gate, and nothing below.



::... Deutsch's Algorithm

Let's go into the details of the circuit. We start with the upper qubit in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. Then, we apply two Hadamard gates, which flip the first qubit to the $|+\rangle$ state and the second to $|-\rangle$, so that the overall state is $|\psi_1\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |-\rangle$. These two qubits enters the oracle gate, and in the output the first qubit remains unchanged while the state of the second qubit depends on the evaluation of the function of the state of the first qubit. We add on the upper qubit an Hadamard gate, and nothing below.



::... Deutsch's Algorithm

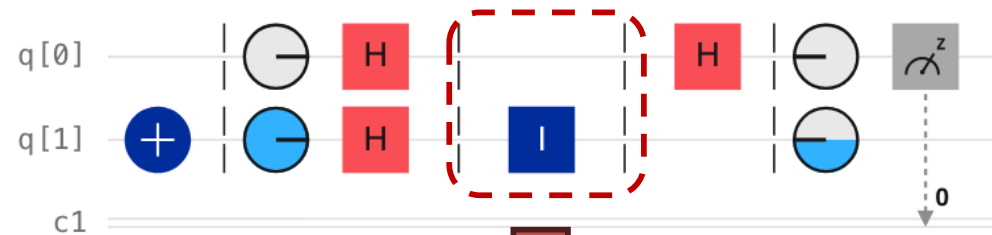
A classic function would call f on the possible values of the input set and compare its outputs to determine if f is constant or not.

This circuit from the Deutsch's algorithm has the interesting aspect that the function has been called only once, by the oracle gate.

Classic manner = two function calls, Quantum manner = one function call. It is not an enormous speed up, but it gives the glimpse of why quantum computers might be faster than classic ones.

::... Deutsch's Algorithm - Example

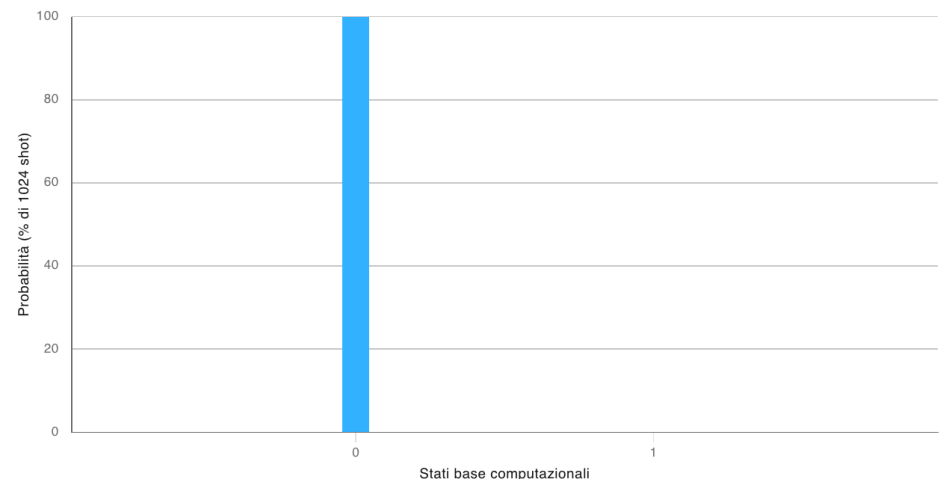
Let us experiment with the realization of the circuit from the Deutsch's



algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.

a	$f_1(a)$	a	$f_2(a)$	a	$f_3(a)$	a	$f_4(a)$
0	0	0	0	0	1	0	1
1	0	1	1	1	0	1	1

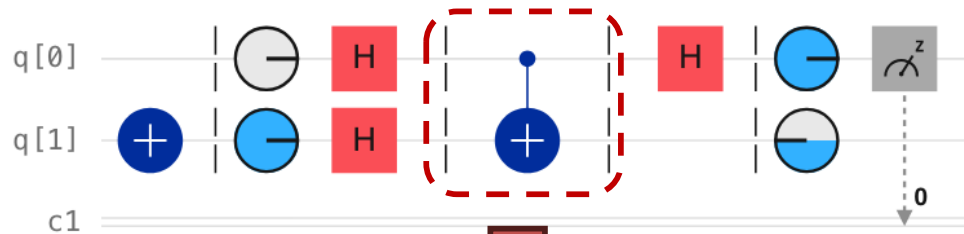
The function is constant.



::... Deutsch's Algorithm - Example

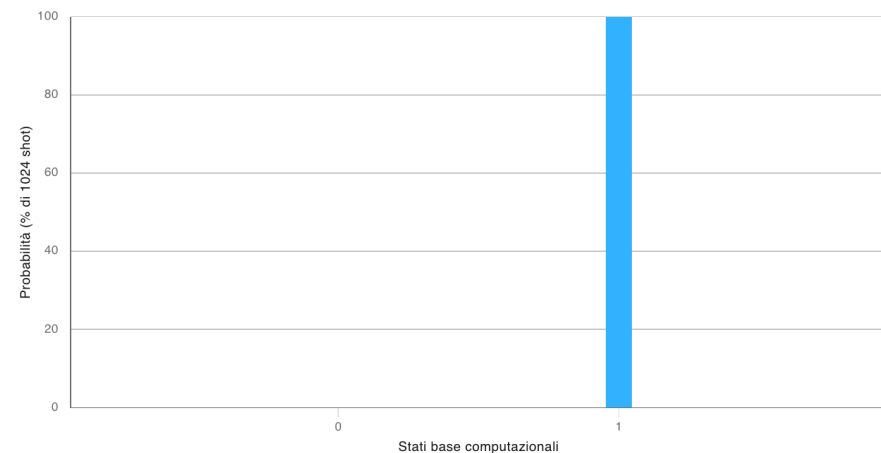
Let us experiment with the realization of the circuit from the Deutsch's

algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



a	$f_1(a)$	a	$f_2(a)$	a	$f_3(a)$	a	$f_4(a)$
0	0	0	0	0	1	0	1
1	0	1	1	1	0	1	1

The function is balanced.



::... Deutsch-Jozsa's Algorithm

The previous problem can be generalized for functions with more than one bit as input. This time, we have a Boolean function f that takes an n -bit number (with n being arbitrary) as input and outputs 0 or 1:

$$f: \{0,1\}^n \rightarrow \{0,1\}$$

Thus, an input to f is one of 16 possible 4 bit binary numbers:

0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111.

f is one of the following two types:

- It is a constant function, i.e., $f(x)$ is the same for all 16 possible values of the input x , or
- It is a balanced function, i.e., $f(x)$ is 0 for exactly 8 of the possible 16 inputs and $f(x)$ is 1 for the remaining 8 of the possible 16 inputs.

A classical deterministic algorithm can determine if f is balanced or constant up to $2^n/2 + 1$.

::... Deutsch-Jozsa's Algorithm

The previous problem can be generalized for functions with more than one bit as input. This time, we have a Boolean function f that takes an n -bit number (with n being arbitrary) as input and outputs 0 or 1:

$$f: \{0,1\}^n \rightarrow \{0,1\}$$

Thus, an input to f is one of 16 possible 4 bit binary numbers:

0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111.

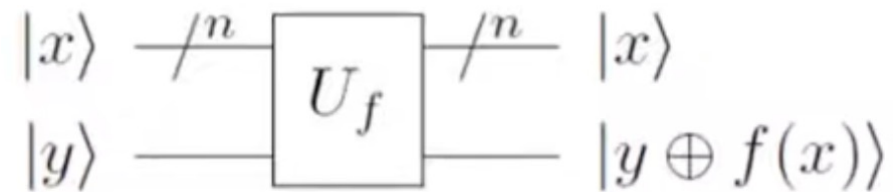
f is one of the following two types:

- It is a constant function, i.e., $f(x)$ is the same for all 16 possible values of the input x , or
- It is a balanced function, i.e., $f(x)$ is 0 for exactly 8 of the possible 16 inputs and $f(x)$ is 1 for the remaining 8 of the possible 16 inputs.

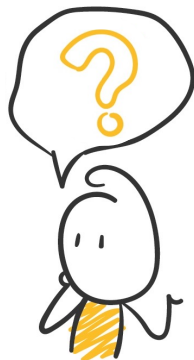
A classical probabilistic algorithm can determine if f is balanced or constant with probability at least $2/3$ by using 2 queries.

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .



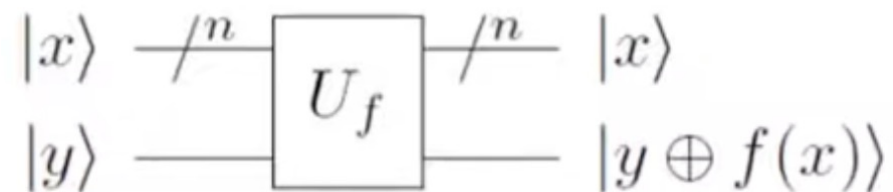
$|x\rangle$ represents the state of n qubits, and if it is in the basis states, it does not change after the application of the oracle gate, while $|y\rangle$ might change based on the value assumed by the f function.



Is this gate a unitary transformation? This is crucial to be implementable by means of unitary gates.

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .



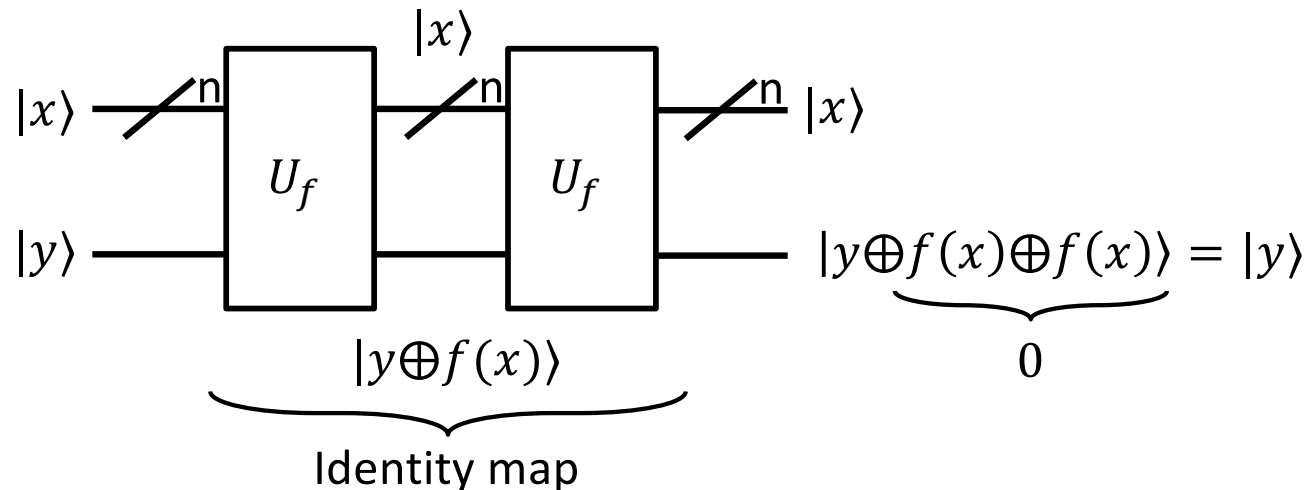
$|x\rangle$ represents the state of n qubits, and if it is in the basis states, it does not change after the application of the oracle gate, while $|y\rangle$ might change based on the value assumed by the f function.



This oracle maps basis states to basis states, and this such a mapping is one to one. This can be demonstrated by applying this gate twice: the upper output is still unchanged state of the first input, and the second is addition of $f(x)$ twice modulo 2.

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

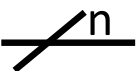


The oracle is its own inverse, which means its transformation is invertible and so the mapping is one-to-one, and it must be unitary.

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state.

$|0 \dots 00\rangle$ 

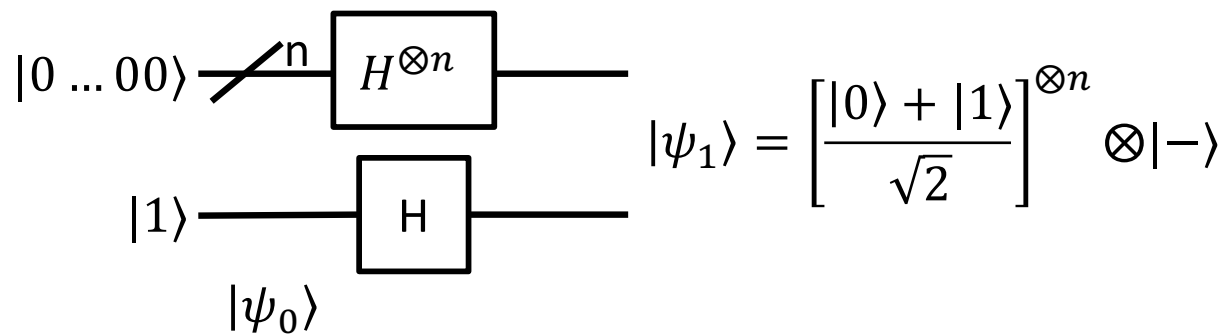
$|1\rangle$ 

$|\psi_0\rangle$

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

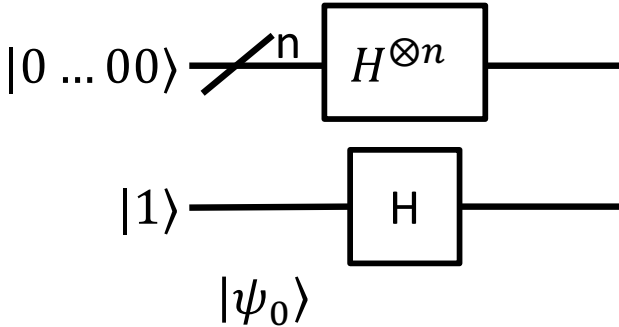
This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits.



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits.



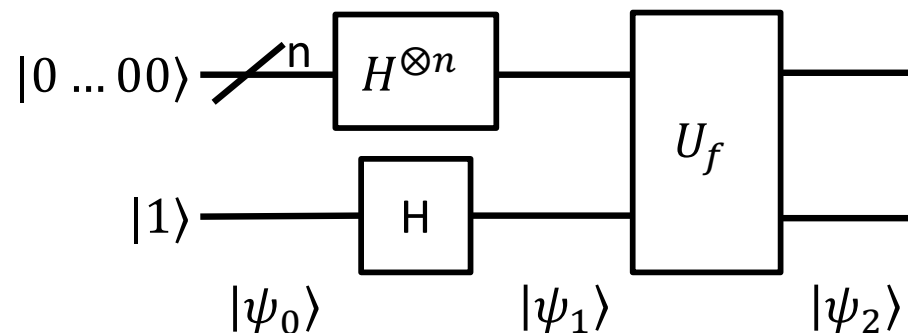
$$|\psi_1\rangle = \left[\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right]^{\otimes n} \otimes |-\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |-\rangle$$

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called.

$$\hat{U}_f|x, -\rangle = (-1)^{f(x)}|x, -\rangle$$



The state $|x-\rangle$ is an eigenstate for the oracle, i.e., a quantum state (wavefunction) that remains unchanged, except for a scalar factor, called eigen value $((-1)^{f(x)})$.

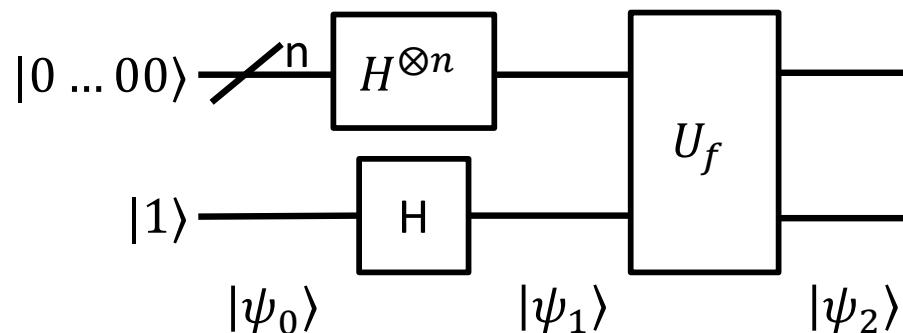
This still holds with n data qubits.

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called.

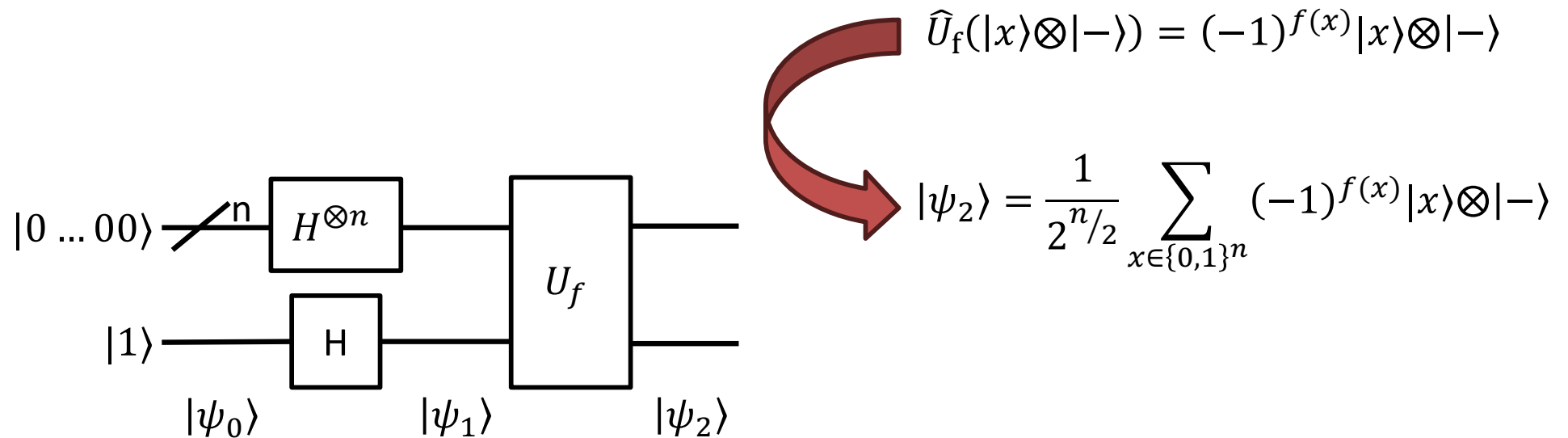
$$\hat{U}_f(|x\rangle \otimes |-\rangle) = (-1)^{f(x)} |x\rangle \otimes |-\rangle$$



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

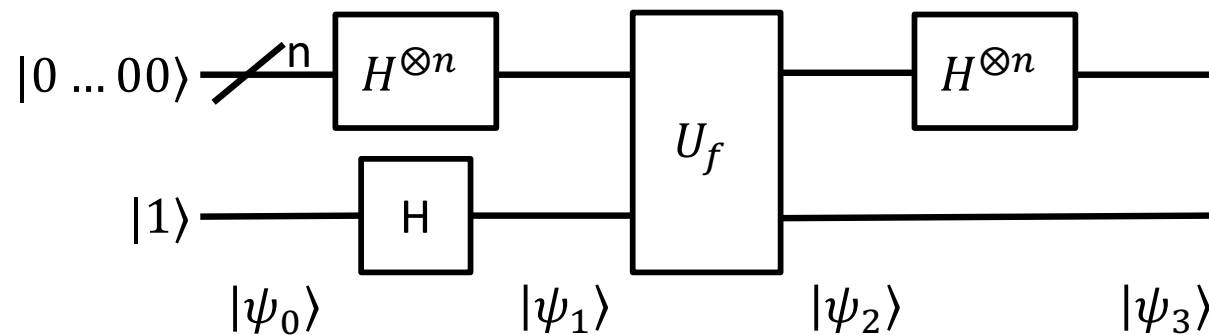
This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called.



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

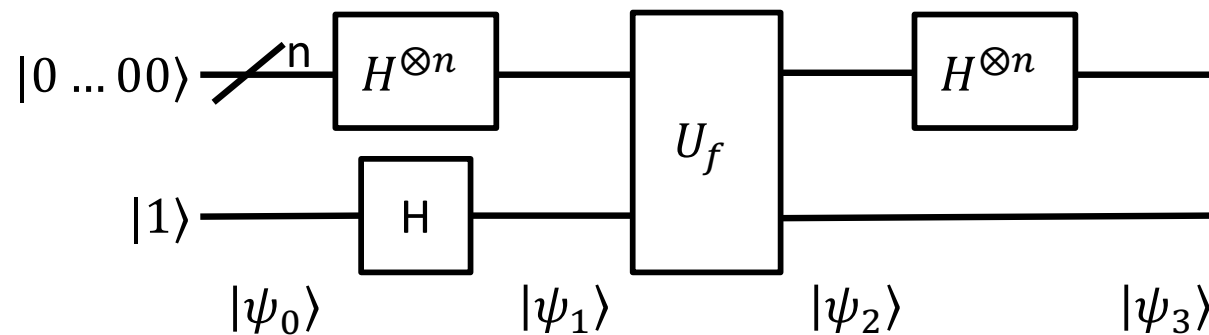
This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.



$$\hat{H}(|0\rangle) = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

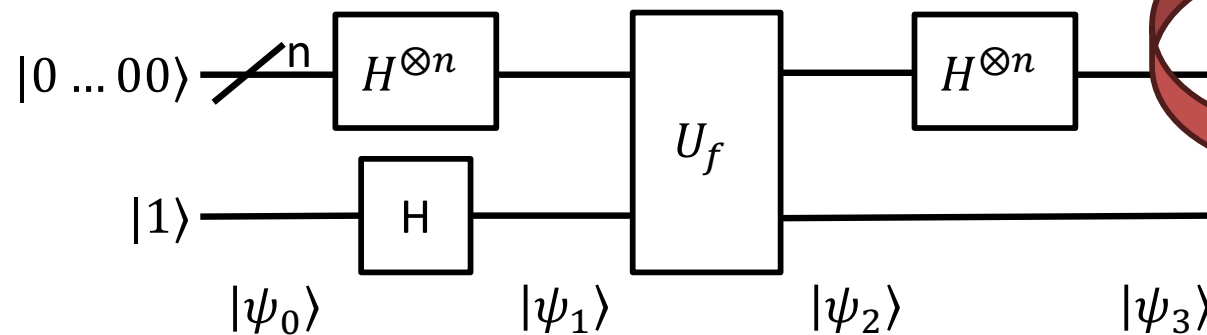
$$\hat{H}(|1\rangle) = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

In the compact form we can write:



$$\hat{H}(|0\rangle) = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

$$\hat{H}(|1\rangle) = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

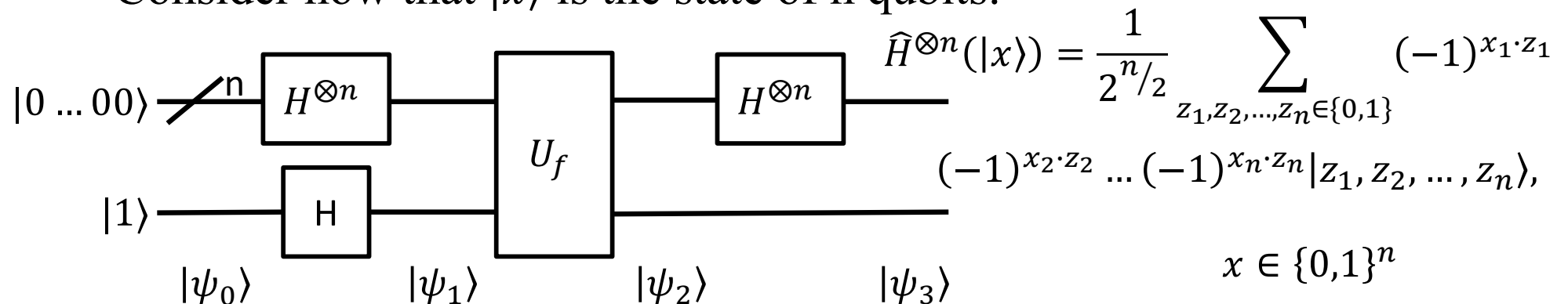
$$\hat{H}(|x\rangle) = \frac{1}{\sqrt{2}} \sum_{z \in \{0,1\}} (-1)^{x \cdot z} |z\rangle, \quad x \in \{0,1\}$$

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

Consider now that $|x\rangle$ is the state of n qubits:



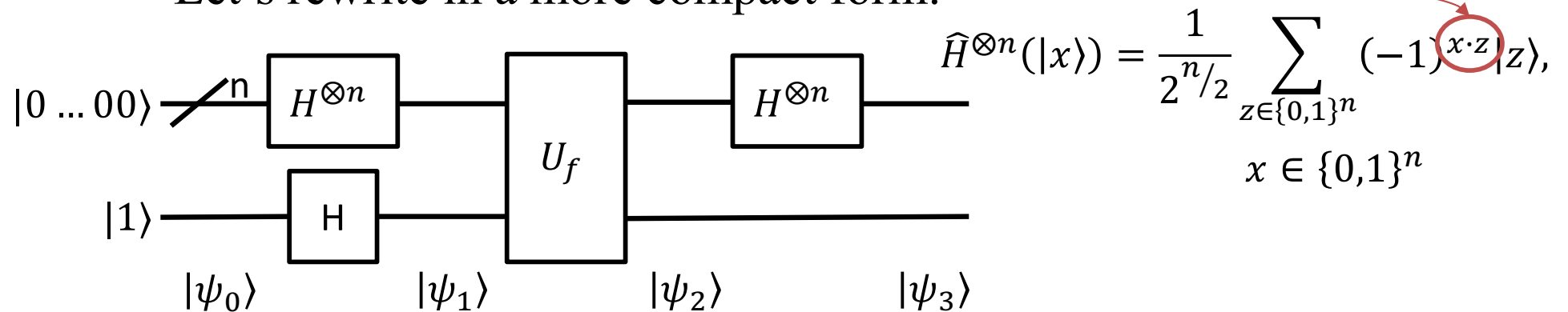
::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

bitwise scalar product of bit strings mod 2

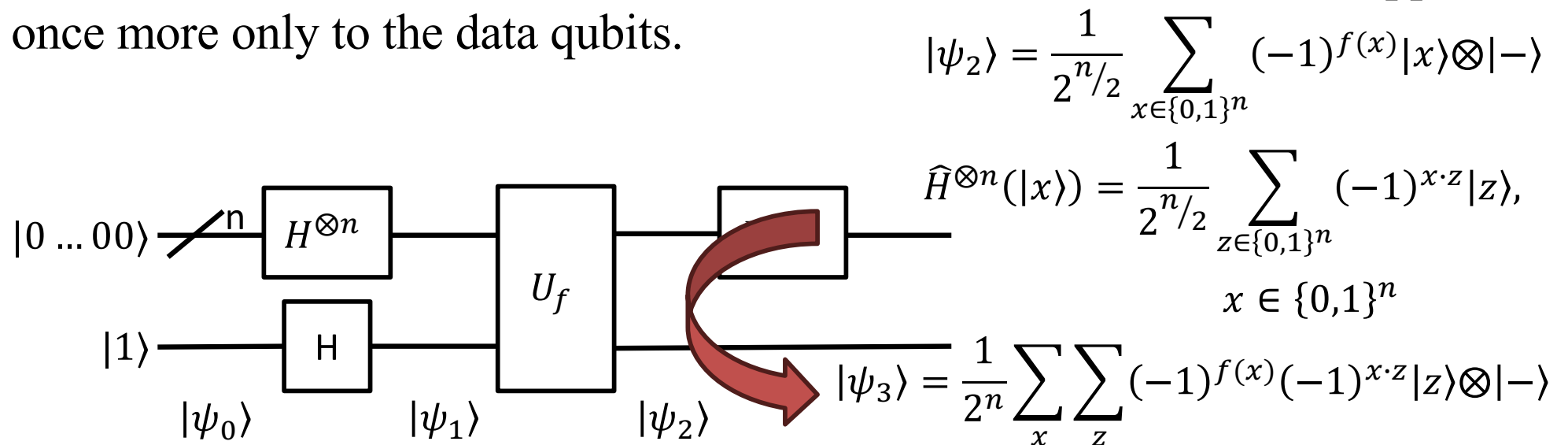
Let's rewrite in a more compact form:



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

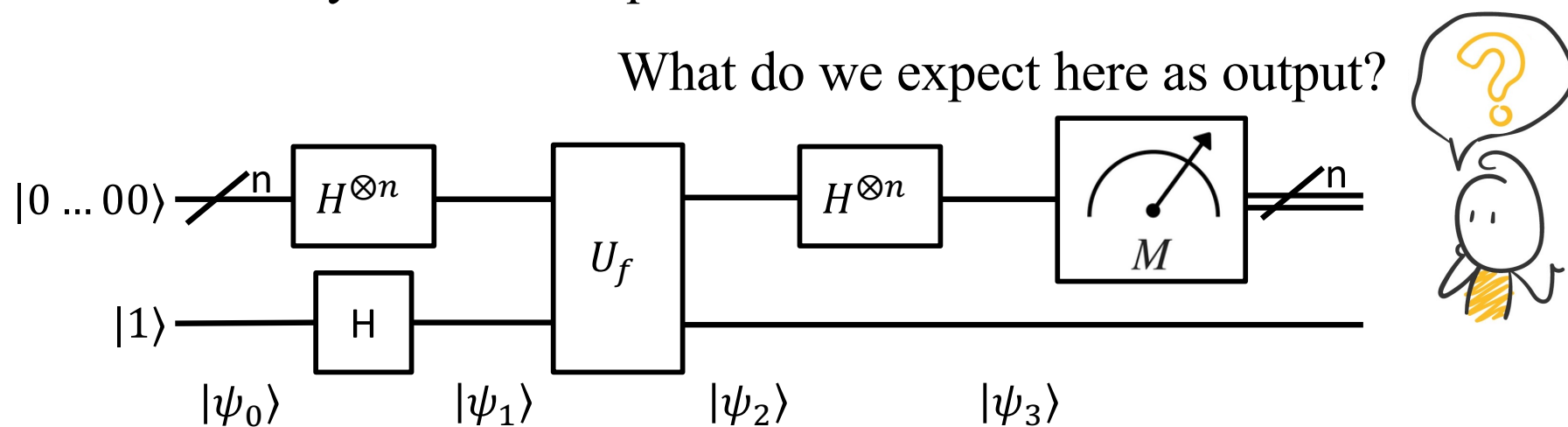
This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.



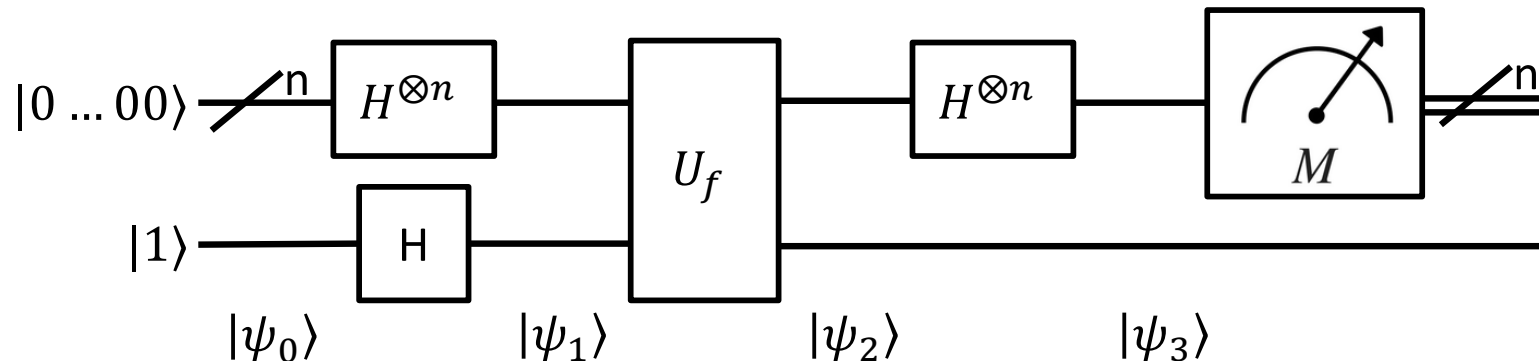
::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

$$|\psi_3\rangle = \frac{1}{2^n} \sum_x \sum_z (-1)^{f(x)} (-1)^{x \cdot z} |z\rangle \otimes |-\rangle$$

$(-1)^{f(x)+x \cdot z}$



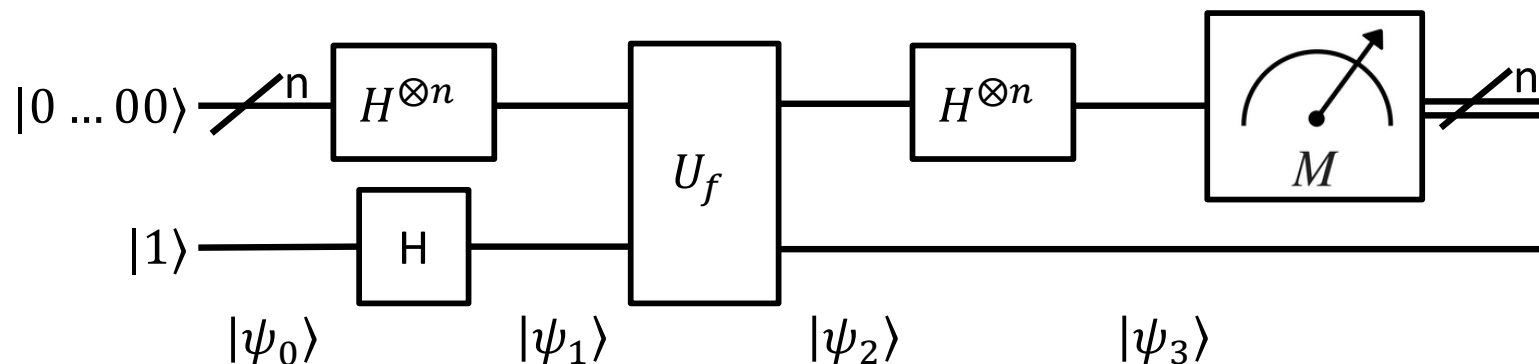
::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

$$|\psi_3\rangle = \frac{1}{2^n} \sum_x \sum_z (-1)^{f(x)+x \cdot z} |z\rangle \otimes |-\rangle$$

superposition of basis states



::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

This is the probability of having all 0 as measurements in standard basis for the upper input:


$$|\psi_3\rangle = \frac{1}{2^n} \sum_x \sum_z (-1)^{f(x)+x \cdot z} |z\rangle \otimes |-\rangle$$

superposition of basis states

$$\text{prob}(00\dots 0 | \psi_3) = \left| \frac{1}{2^n} \sum_x (-1)^{f(x)} \right|^2$$

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

This is the probability of having all 0 as measurements in standard basis for the upper input:

All $f(x)$ will be 0 or 1, so all the terms in the sum will add each other to 2^n as result.


$$|\psi_3\rangle = \frac{1}{2^n} \sum_x \sum_z (-1)^{f(x)+x \cdot z} |z\rangle \otimes |-\rangle$$

superposition of basis states

$$\text{prob}(00\dots 0 | \psi_3) = \begin{cases} 1 & f(0) = f(1) \\ 0 & f(0) \neq f(1) \end{cases}$$

::... Deutsch-Jozsa's Algorithm

The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

This is the probability of having all 0 as measurements in standard basis for the upper input:

Half of $f(x)$ will be 0 and half 1, so the terms in the sum will cancel each other.


$$|\psi_3\rangle = \frac{1}{2^n} \sum_x \sum_z (-1)^{f(x)+x \cdot z} |z\rangle \otimes |-\rangle$$

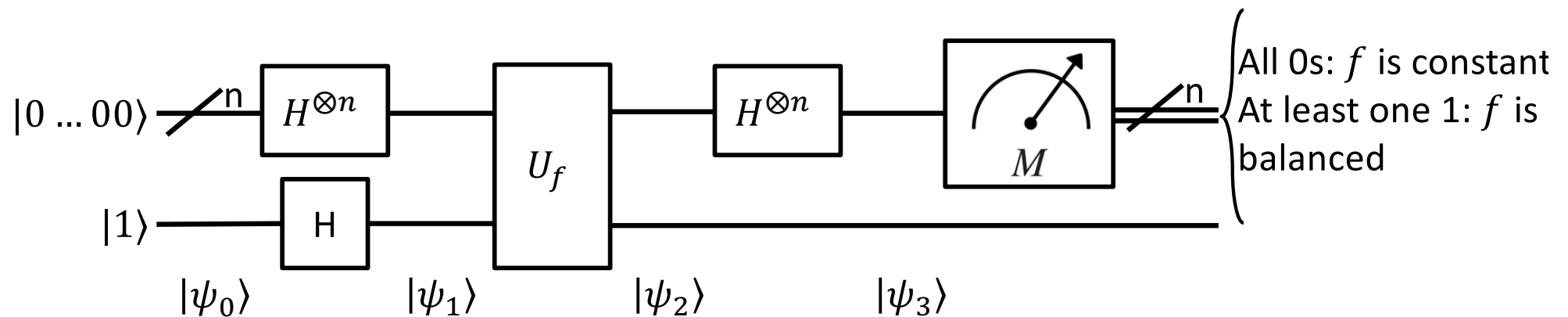
superposition of basis states

$$\text{prob}(00\dots 0 | \psi_3) = \begin{cases} 1 & f(0) = f(1) \\ 0 & f(0) \neq f(1) \end{cases}$$

::... Deutsch-Jozsa's Algorithm

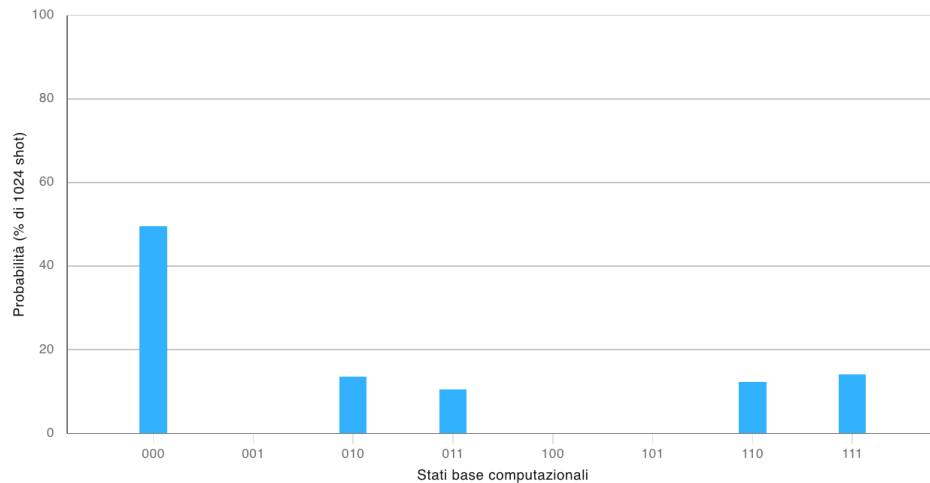
The Deutsch-Jozsa's algorithm allows to determine if f is constant or balanced with just one evaluation of the function. This algorithm uses the oracle gate, as the previous one, where the only difference is being that the upper input is a bit string of length n .

This algorithm proceeds following the same steps and the simpler previous version: the upper input has qubits all in the $|0\rangle$ basis state and the lower one in the $|1\rangle$ basis state. The Hadamard gate is applied to all the qubits, and after the oracle is called. Now, Hadamard transformations are applied once more only to the data qubits.

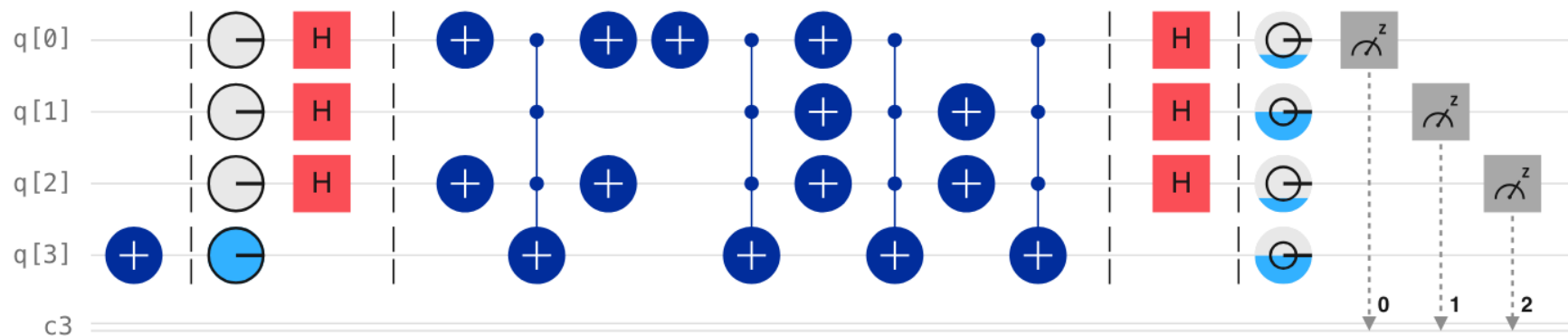


::... Deutsch-Jozsa's Algorithm - Example

Let us experiment with the realization of the circuit from the Deutsch-Jozsa's algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



The function is balanced.



::... Simon's Algorithm

This is one of the two algorithms that addresses a rather artificial problem , where there is no real application, but are stepping stones to actually realize those algorithms that have concrete utility. We consider a function on bit strings that maps to another bit string of the same length as the input:

$$f: \{0,1\}^n \rightarrow \{0,1\}^n$$

and we have advanced information on the properties of f : there exists a bit string s of length n such that $f(x) = f(y)$ if and only if y is x or $x \oplus s$:

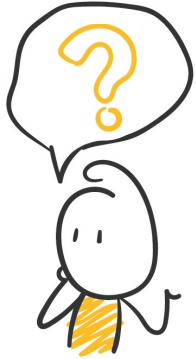
$$\exists s \in \{0,1\}^n: f(x) = f(y) \Leftrightarrow y \in \{x, x \oplus s\}$$

where $\oplus$ is a bitwise add mod 2. The task addressed by this algorithm is to find such an s .

This implies that the function f is either 2-to-1 or 1-to-1:

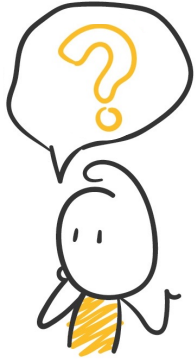
- in the special case where $s = 0$, we have $f(x) = f(y) \Leftrightarrow y = x$ and it represents a 1-to-1 function.
- if $s \neq 0$, we have two arguments x, y for which the function returns the same output, and here we have a 2-to-1 mapping.

::... Simon's Algorithm



How many function calls do we need in order to find s if we apply a classic approach?

::... Simon's Algorithm



How many function calls do we need in order to find s if we apply a classic approach?



We need to keep pooling the function for different arguments x until we find two distinct arguments x_1, x_2 for which the function has the same output. At this time, we can compute s from the found arguments x_1, x_2 .

$$x_2 = x_1 \oplus s \rightarrow x_1 \oplus x_2 = \underbrace{x_1 \oplus x_1}_0 \oplus s = s$$

If we don't find such two distinct arguments x_1, x_2 , we have to keep trying half of the possible arguments plus 1, and if we fail to find them, then we know we are in the first case and no distinct arguments exists.

So, the number of tries is up to $2^n/2 + 1$, which is the maximum number of function calls.

::... Simon's Algorithm

With quantum computing, we can do better than an exponentially-growing number of function calls.

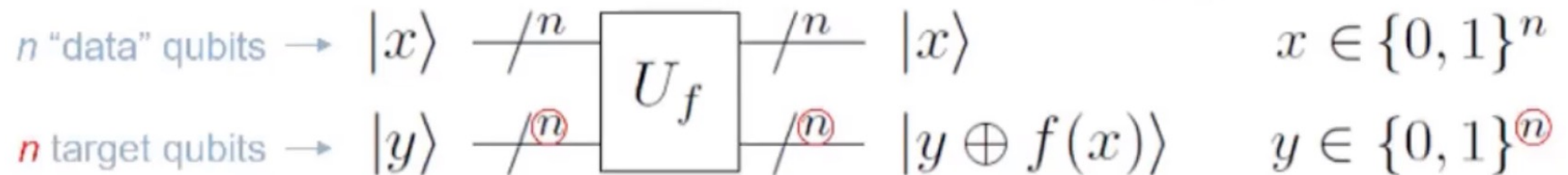
We have the oracle gate, looking very much the one in the previous algorithm, except for some small differences:

$$\begin{array}{ccccc} |x\rangle & \xrightarrow{\text{---}/n\text{---}} & \boxed{U_f} & \xrightarrow{\text{---}/n\text{---}} & |x\rangle \\ |y\rangle & \xrightarrow{\text{---}} & & \xrightarrow{\text{---}} & |y \oplus f(x)\rangle \end{array}$$

::... Simon's Algorithm

With quantum computing, we can do better than an exponentially-growing number of function calls.

We have the oracle gate, looking very much the one in the previous algorithm, except for some small differences:



We have n -data qubits as the upper input that remain unchanged in their basis states, and (in contrast to the Deutsch-Jozsa) a lower input of n -data qubits that is eventually flipped according to the function applied to the upper input, analogously to the Deutsch-Jozsa.

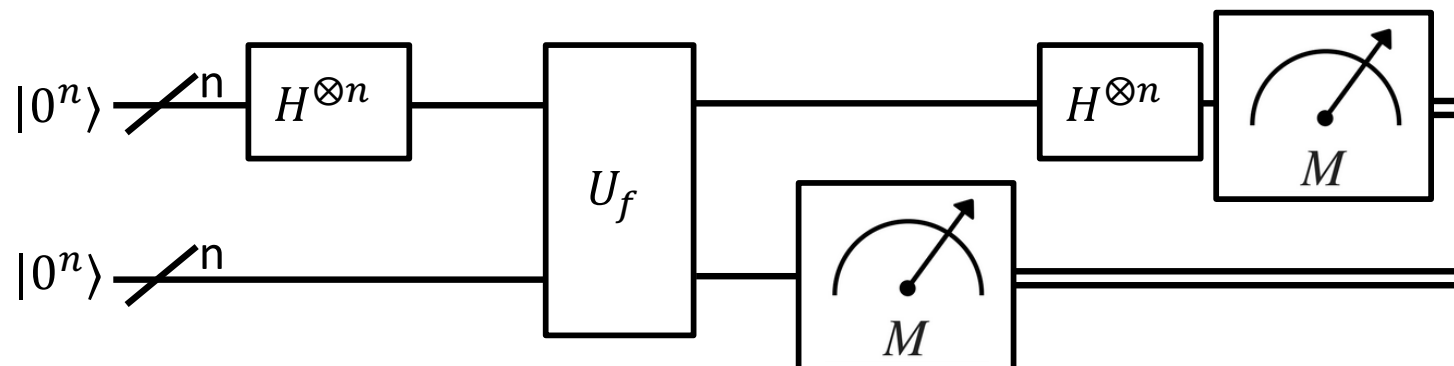
::... Simon's Algorithm

With quantum computing, we can do better than an exponentially-growing number of function calls.

We have the oracle gate, looking very much the one in the previous algorithm, except for some small differences:

$$\begin{array}{ll}
 n \text{ "data" qubits} \rightarrow |x\rangle & \xrightarrow{\text{---}/n\text{---}} \boxed{U_f} \xrightarrow{\text{---}/n\text{---}} |x\rangle & x \in \{0, 1\}^n \\
 n \text{ target qubits} \rightarrow |y\rangle & \xrightarrow{\text{---}/\textcircled{n}\text{---}} \boxed{U_f} \xrightarrow{\text{---}/\textcircled{n}\text{---}} |y \oplus f(x)\rangle & y \in \{0, 1\}^{\textcircled{n}}
 \end{array}$$

Here's the circuit, holding a lot of similarities with the Deutsch-Jozsa:

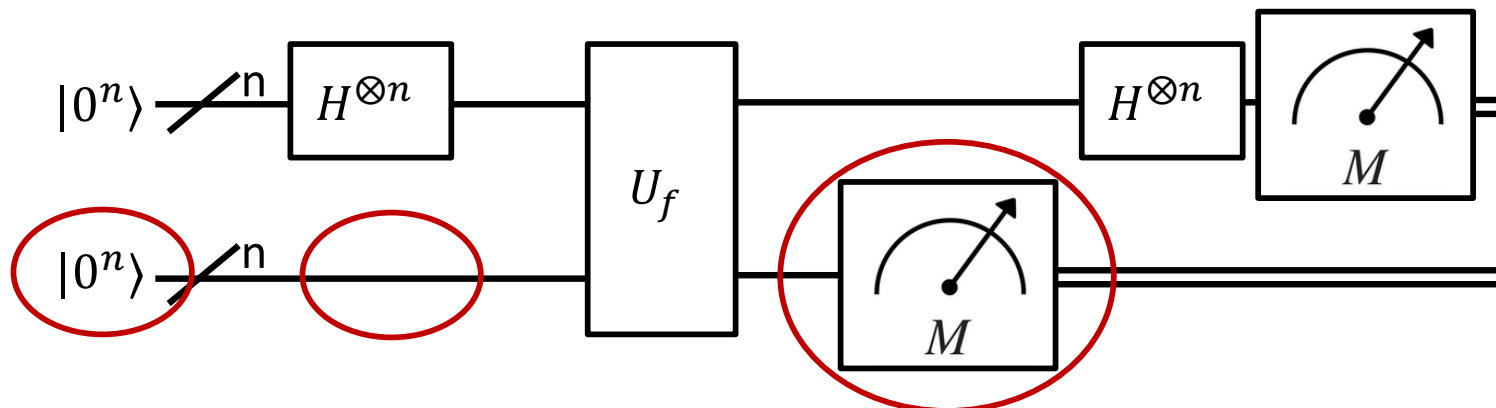


::... Simon's Algorithm

However, there are quite a few twists:

1. the target qubits all start in the basis state $|0\rangle$ (while in the Deutsch-Jozsa the single target qubit starts in the $|1\rangle$ state);
2. Before entering the oracle, we have Hadamards only on the data qubits and not on the target qubits;
3. Immediately after the oracle, we perform measurements on the target qubit and we keep going with the data qubits by making the hadamard (as in the Deutsch-Jozsa case) and then make measurements.

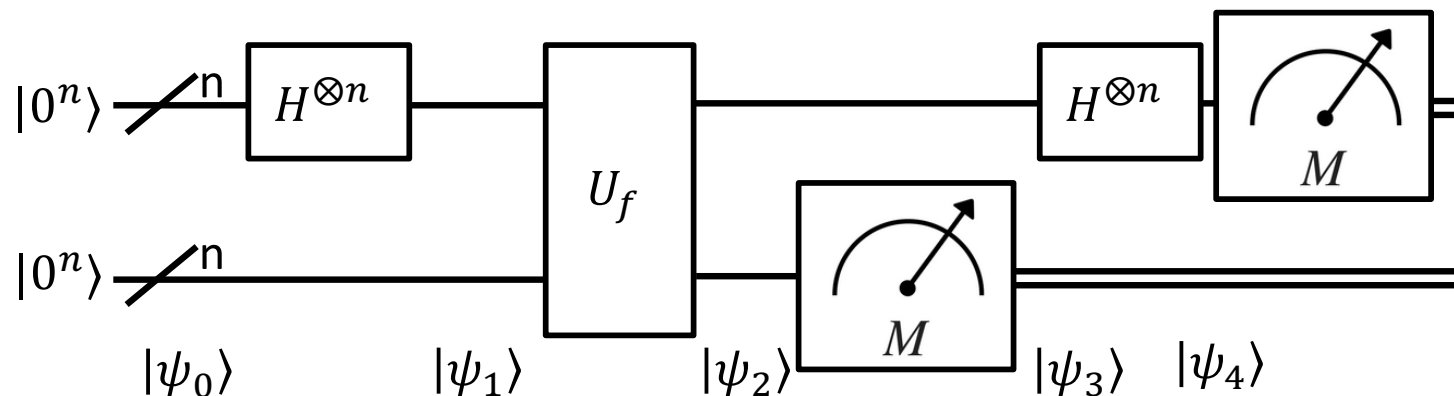
In red there are these three main differences.



::... Simon's Algorithm

Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle$ is equal to the Deutsch-Jozsa case and is $|\psi_1\rangle = \underbrace{\frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle}_{\text{Hadamard Transformation}} \otimes \underbrace{|0^n\rangle}_{\text{Unchanged target qubits}}.$



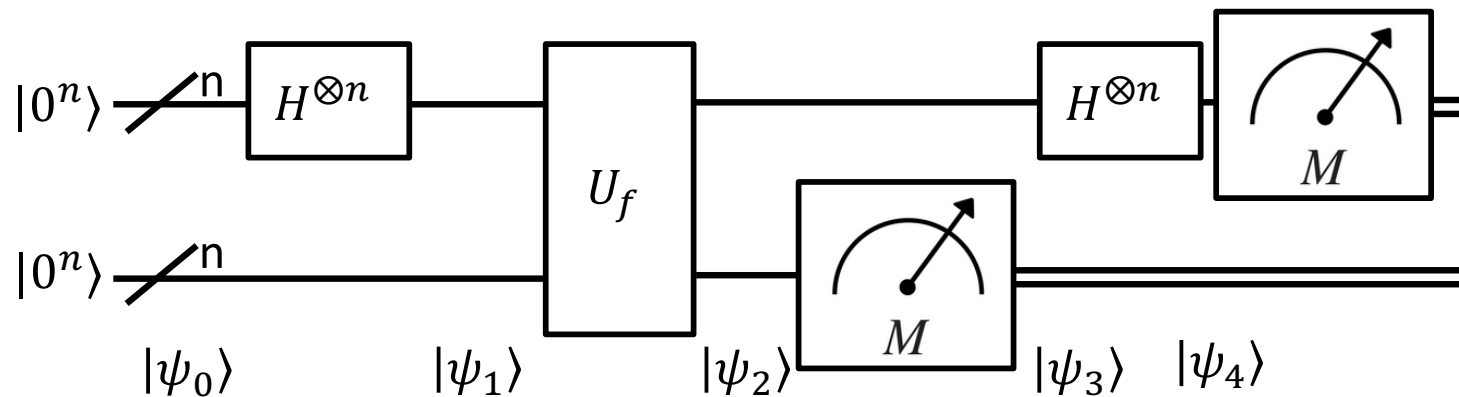
... Simon's Algorithm

Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0^n\rangle.$

2. Then the oracle comes and we have $|\psi_2\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes \underbrace{|f(x)\rangle}.$

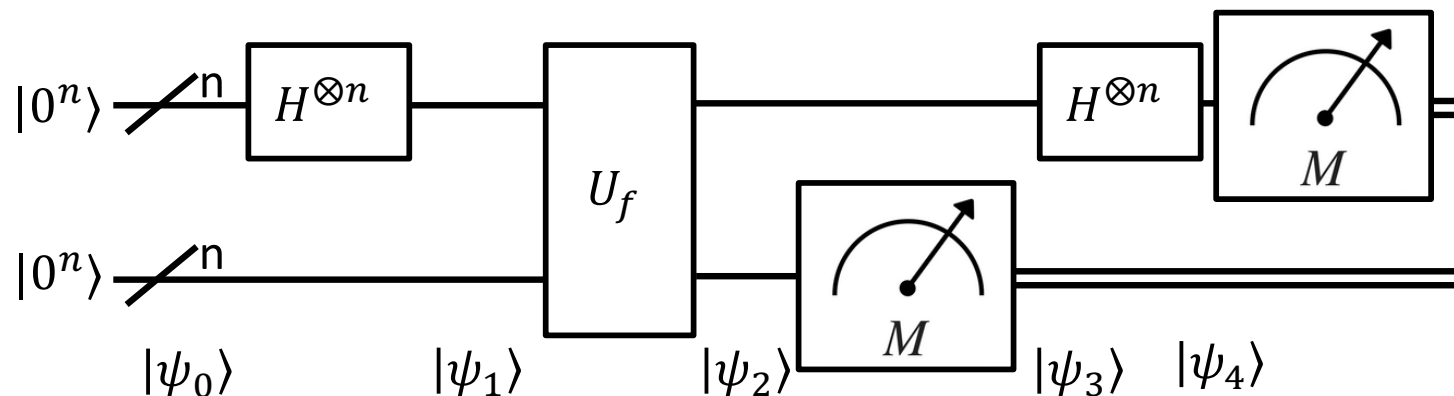
The oracle adds the evaluation of the function in x to a 0 state



::... Simon's Algorithm

Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0^n\rangle.$
2. $|\psi_2\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle.$
3. The target qubits ($|f(x)\rangle$) are measured and $|\psi_3\rangle$ is equal to $|\psi_2\rangle$ projected orthogonally onto the eigenspace associated with the a specific value of the function $f(x)$ that are assumed only for x and $x \oplus s$. $|\psi_3\rangle = \frac{1}{\sqrt{2}} (|x\rangle + |x \oplus s\rangle) \otimes |f(x)\rangle.$



... Simon's Algorithm

Let's go through the different stages of the calculation of the state evolution:

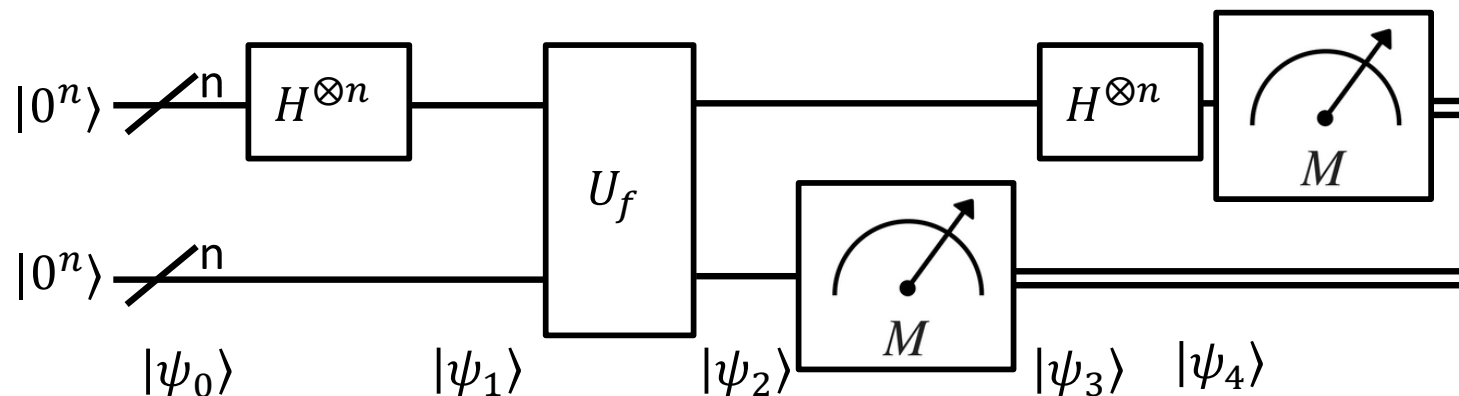
$$1. |\psi_1\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0^n\rangle.$$

$$2. |\psi_2\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle.$$

$$3. |\psi_3\rangle = \frac{1}{\sqrt{2}} (|x\rangle + |x \oplus s\rangle).$$

$$4. \text{ We apply Hadamard transformation on data qubit } |\psi_4\rangle \propto \sum_{z \in \{0,1\}^n} [(-1)^{x \cdot z} + (-1)^{(x \oplus s) \cdot z}] |z\rangle \propto \sum_{z \in \{0,1\}^n} [1 + (-1)^{s \cdot z}] |z\rangle.$$

$$\hat{H}^{\otimes n}(|x\rangle) = \frac{1}{2^{n/2}} \sum_{z \in \{0,1\}^n} (-1)^{x \cdot z} |z\rangle,$$

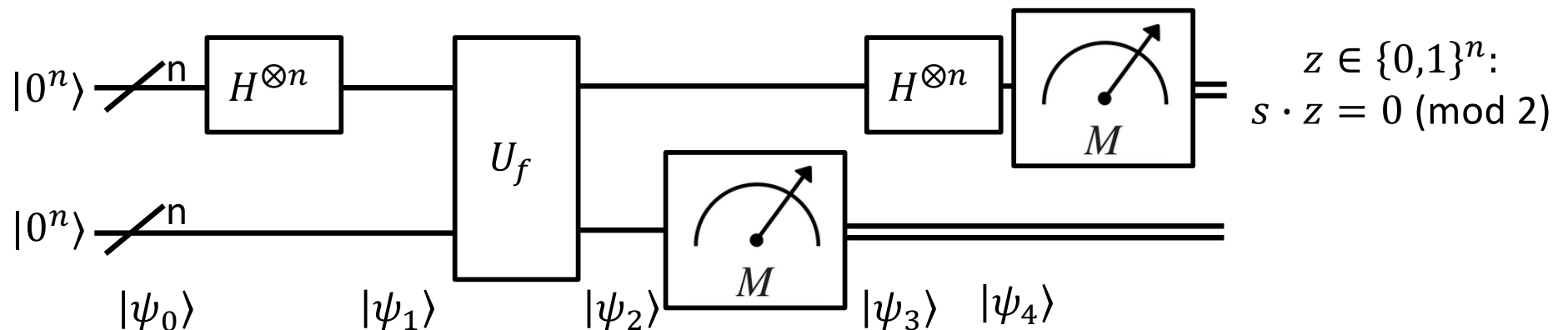


::... Simon's Algorithm

Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0^n\rangle.$
2. $|\psi_2\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle.$
3. $|\psi_3\rangle = \frac{1}{\sqrt{2}} (|x\rangle + |x \oplus s\rangle).$
4. $|\psi_4\rangle \propto \sum_{z \in \{0,1\}^n} \underbrace{[1 + (-1)^{s \cdot z}]}_{\text{Non-zero if and only if } s \cdot z = 0 \pmod{2}} |z\rangle$ and in this case the measure is z .

Non-zero if and only if
 $s \cdot z = 0 \pmod{2}$

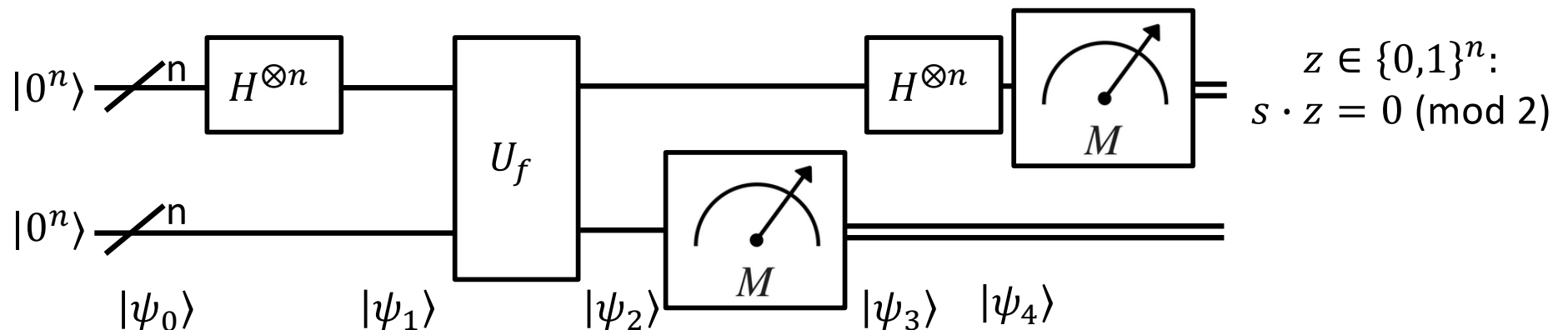


::... Simon's Algorithm

Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0^n\rangle.$
2. $|\psi_2\rangle = \frac{1}{2^{n/2}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle.$
3. $|\psi_3\rangle = \frac{1}{\sqrt{2}} (|x\rangle + |x \oplus s\rangle).$
4. $|\psi_4\rangle \propto \sum_{z \in \{0,1\}^n} \underbrace{[1 + (-1)^{s \cdot z}]}_{\text{Zero if and only if } s \cdot z = 1 \text{ mod } 2} |z\rangle$ and in this case the measure is 0^n .

Zero if and only if
 $s \cdot z = 1 \text{ mod } 2$

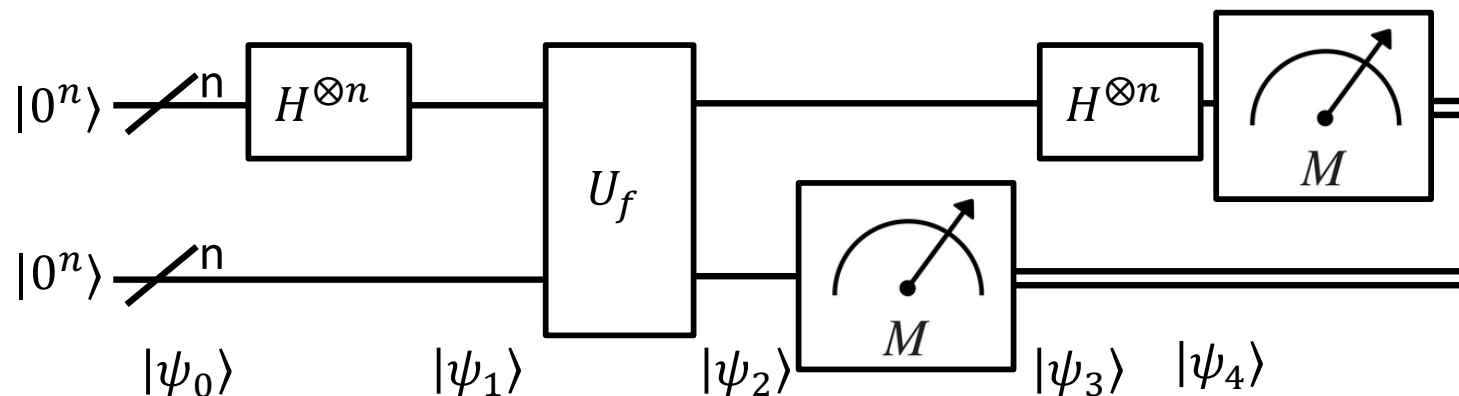


::... Simon's Algorithm

The final measurement on the data qubit produces some z that is the scalar product with the unknown s , that we are looking for. However this does not give the s , so the idea is to repeat the circuit, and each run of the circuit will produce another z , different than the previous.

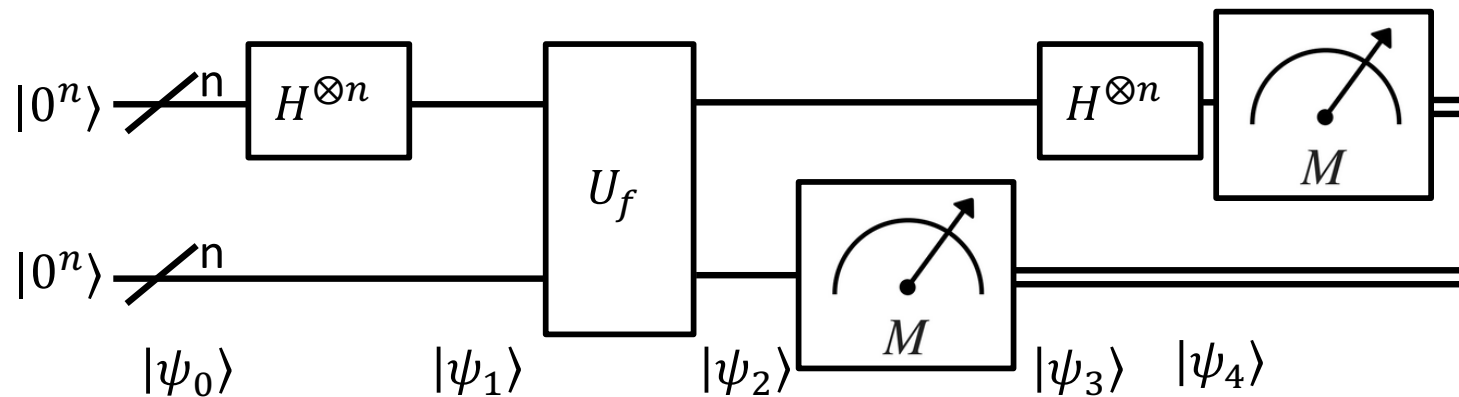
If we run the circuit often enough, we can obtain sufficiently many different z until enough z have been collected to have $(n-1)$ independent conditions for s to determine it uniquely.

This is a probabilistic algorithm, as it is not guaranteed that, after a finite number of repetitions, we will obtain the solution, as it might not be possible to have independent conditions.



::... Simon's Algorithm

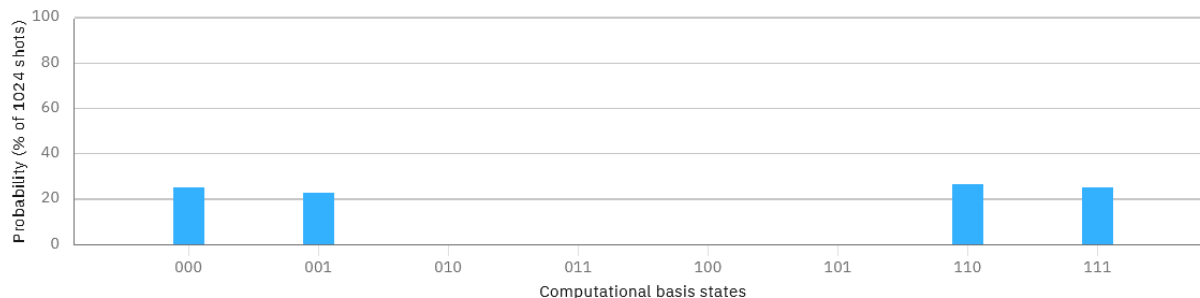
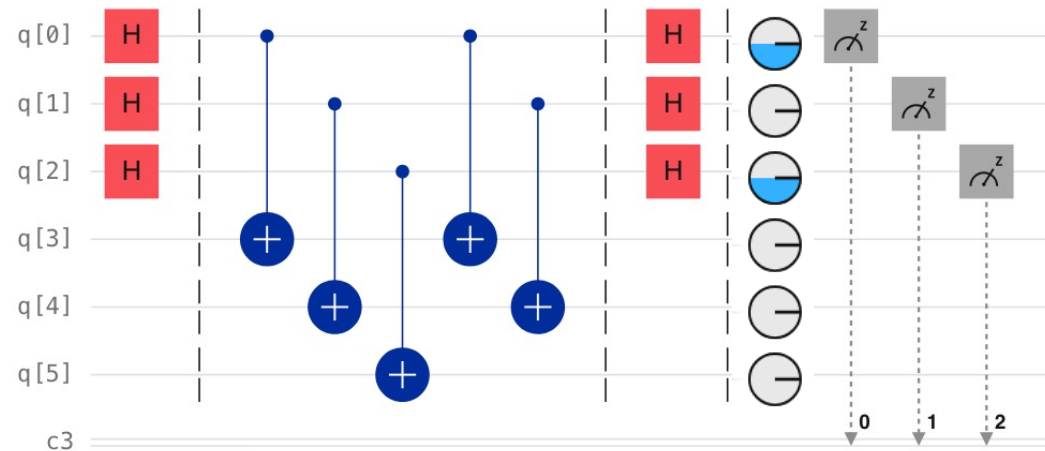
On average, the number of times to run the circuit until having a sufficient number of independent conditions scales linearly with the length of the bit strings, which is an improvement with respect to the classical case.



::... Simon's Algorithm - Example

x	$f(x \oplus s)$
000	101
001	010
010	011
011	100
100	011
101	100
110	101
111	010

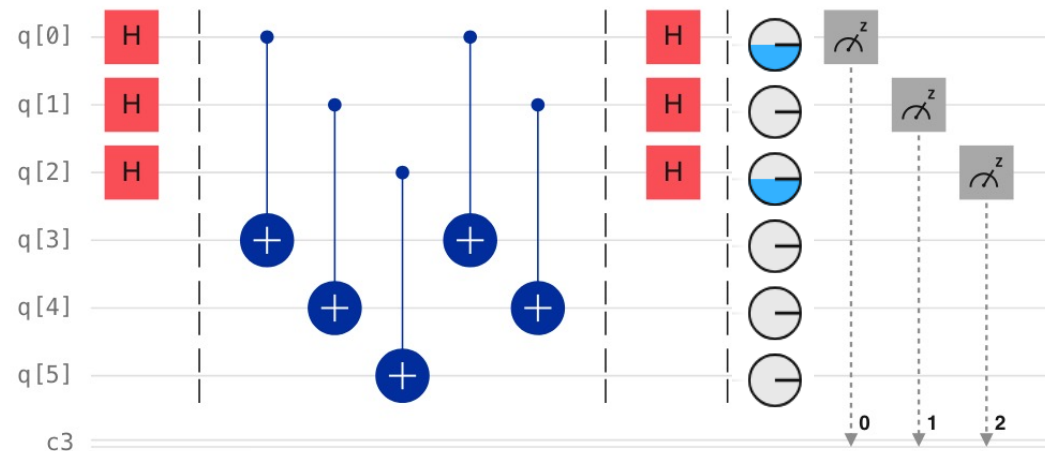
Consider the following simple example with $n = 3$ and the secret string $s = 110$. Notice that inputs $x = 000$ and $x = 110$ will produce the same output $f(x \oplus s) = 101$. This produces a *collision*, i.e. two inputs map to the same output. The same relationship exists for $x = 011 = 111$, $x = 010 = 100$, and $x = 011 = 101$.



::... Simon's Algorithm - Example

x	$f(x \oplus s)$
000	101
001	010
010	011
011	100
100	011
101	100
110	101
111	010

Consider the following simple example with $n = 3$ and the secret string $s = 110$. Notice that inputs $x = 000$ and $x = 110$ will produce the same output $f(x \oplus s) = 101$. This produces a *collision*, i.e. two inputs map to the same output. The same relationship exists for $x = 011 = 111$, $x = 010 = 100$, and $x = 011 = 101$.



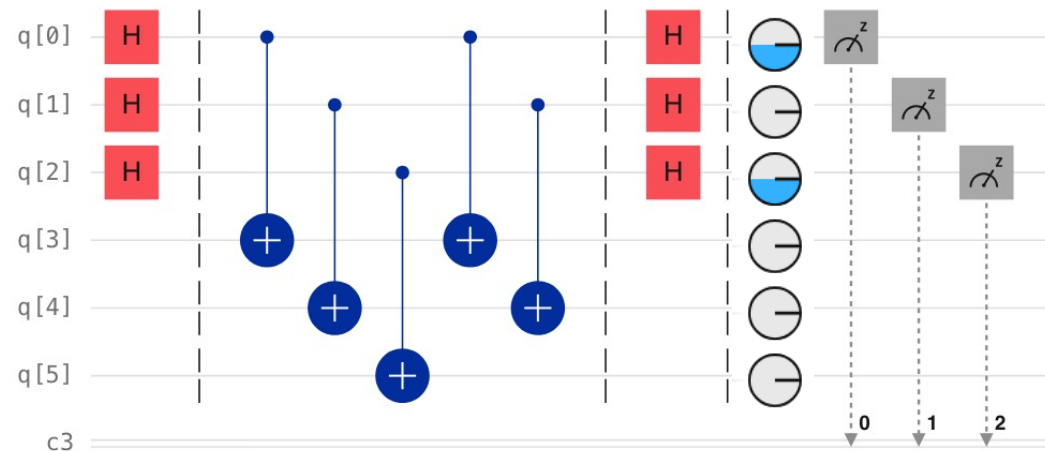
We will use the results in the probability distribution to classically find s . The results must satisfy:

$$s \cdot z = 0 \pmod{2}$$

::... Simon's Algorithm - Example

x	$f(x \oplus s)$
000	101
001	010
010	011
011	100
100	011
101	100
110	101
111	010

Consider the following simple example with $n = 3$ and the secret string $s = 110$. Notice that inputs $x = 000$ and $x = 110$ will produce the same output $f(x \oplus s) = 101$. This produces a *collision*, i.e. two inputs map to the same output. The same relationship exists for $x = 011 = 111$, $x = 010 = 100$, and $x = 011 = 101$.



We will use the results in the probability distribution to classically find s . The results must satisfy:

$$s \cdot z = 0 \pmod{2}$$

... Simon's Algorithm - Example

x	$f(x \oplus s)$
000	101
001	010
010	011
011	100
100	011
101	100
110	101
111	010

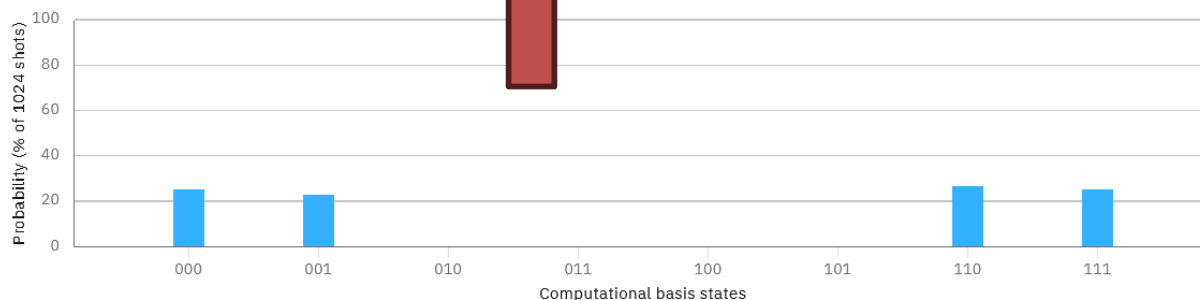
Consider the following simple example with $n = 3$ and the secret string $s = 110$. Notice that inputs $x = 000$ and $x = 110$ will produce the same output $f(x \oplus s) = 101$. This produces a *collision*, i.e. two inputs map to the same output. The same relationship exists for $x = 011 = 111$, $x = 010 = 100$, and $x = 011 = 101$.

$$s \cdot 000 = (s_2 \cdot 0) + (s_1 \cdot 0) + (s_0 \cdot 0) = 0 \pmod{2}$$

$$s \cdot 001 = (s_2 \cdot 0) + (s_1 \cdot 0) + (s_0 \cdot 1) = 0 \pmod{2}$$

$$s \cdot 110 = (s_2 \cdot 1) + (s_1 \cdot 1) + (s_0 \cdot 0) = 0 \pmod{2}$$

$$s \cdot 111 = (s_2 \cdot 1) + (s_1 \cdot 1) + (s_0 \cdot 1) = 0 \pmod{2}$$



::... Simon's Algorithm - Example

x	$f(x \oplus s)$
000	101
001	010
010	011
011	100
100	011
101	100
110	101
111	010

Consider the following simple example with $n = 3$ and the secret string $s = 110$. Notice that inputs $x = 000$ and $x = 110$ will produce the same output $f(x \oplus s) = 101$. This produces a *collision*, i.e. two inputs map to the same output. The same relationship exists for $x = 011 = 111$, $x = 010 = 100$, and $x = 101 = 101$.

$$s \cdot 000 = (s_2 \cdot 0) + (s_1 \cdot 0) + (s_0 \cdot 0) = 0 \pmod{2}$$

$$s \cdot 001 = (s_2 \cdot 0) + (s_1 \cdot 0) + (s_0 \cdot 1) = 0 \pmod{2}$$

$$s \cdot 110 = (s_2 \cdot 1) + (s_1 \cdot 1) + (s_0 \cdot 0) = 0 \pmod{2}$$

$$s \cdot 111 = (s_2 \cdot 1) + (s_1 \cdot 1) + (s_0 \cdot 1) = 0 \pmod{2}$$

From the second equation, we can see $s_0 = 0$.

From the last 2 equations, we can see $s_2 = s_1 = 0$ ($s = 000$) or $s_2 = s_1 = 1$ ($s = 110$). The non-trivial solution gives us $s = 110$, which is our hidden string.

::... Quantum Search

We are looking for a solution to a given problem, and finding it is difficult, but if we have a possible candidate, checking if it is a solution is simple.

More formally, given a boolean function $f(x)$ such that

$$f(x): \{0,1\}^n \rightarrow \{0,1\}, \quad f(x) = \begin{cases} 1, & \text{if } x = x^* \\ 0, & \text{if } x \neq x^* \end{cases}$$

We have to find the value of x such that $f(x) = 1$, which is equivalent of finding x^* .

So, solving a problem via trial and error is more feasible: we have n possible solutions, and checking which one is correct is straightforward. We go through all of them until we find the correct one.

This scales linearly with the number of possibilities, and on average, we must have $(n+1)/2$ attempts. We want a quantum algorithm that does this better than a classical one.

::... Quantum Search

Let's assume we have four possibilities, so $n = 4$, and all of them are evenly likely, so with a classic method we have to try them all in the worst case or you're lucky that the first one you try is the right one. Here's a proposal to go faster with a quantum algorithm.

The previous quantum algorithms worked with an oracle gate, also in this formulation we have such a gate, but with a slight different formulation:

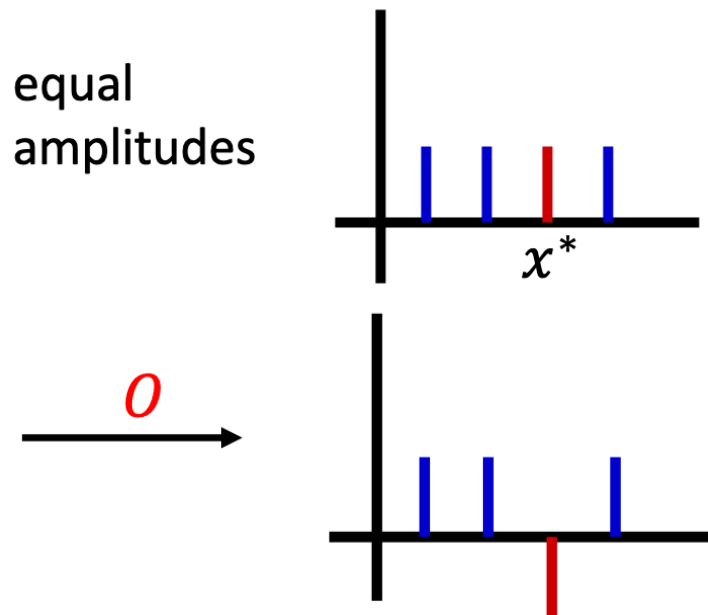


We set up the initial state in superposition so as to model all the possible solutions to the problem represented by $f(x)$, including x^* .

::... Quantum Search

Let's assume we have four possibilities, so $n = 4$, and all of them are evenly likely, so with a classic method we have to try them all in the worst case or you're lucky that the first one you try is the right one. Here's a proposal to go faster with a quantum algorithm.

The previous quantum algorithms worked with an oracle gate, also in this formulation we have such a gate, but with a slight different formulation:



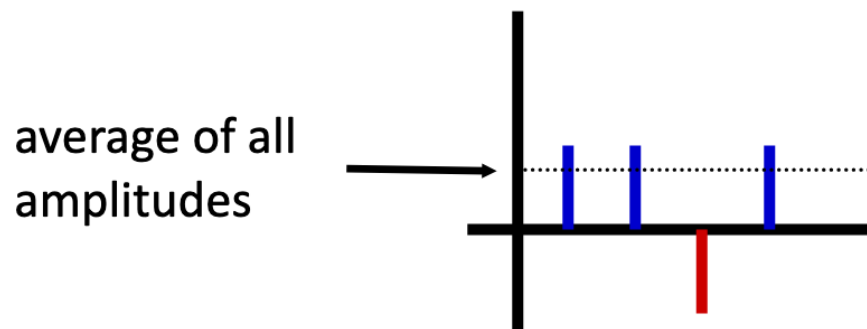
We set up the initial state in superposition so as to model all the possible solutions to the problem represented by $f(x)$, including x^* .

Then, we apply a proper oracle that marks $|x^*\rangle$ with a minus sign.

::... Quantum Search

Let's assume we have four possibilities, so $n = 4$, and all of them are evenly likely, so with a classic method we have to try them all in the worst case or you're lucky that the first one you try is the right one. Here's a proposal to go faster with a quantum algorithm.

The previous quantum algorithms worked with an oracle gate, also in this formulation we have such a gate, but with a slight different formulation:

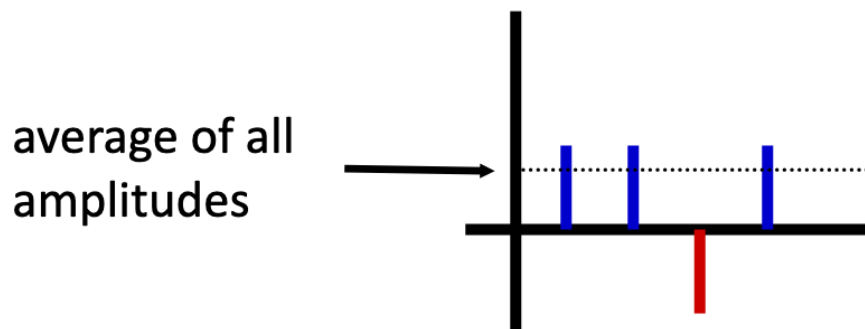


If we measure now, then all the states have equal probability.

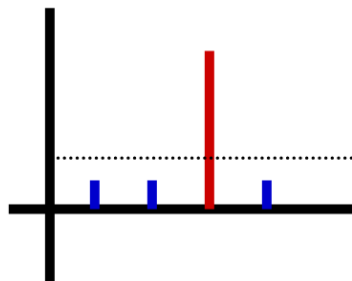
::... Quantum Search

Let's assume we have four possibilities, so $n = 4$, and all of them are evenly likely, so with a classic method we have to try them all in the worst case or you're lucky that the first one you try is the right one. Here's a proposal to go faster with a quantum algorithm.

The previous quantum algorithms worked with an oracle gate, also in this formulation we have such a gate, but with a slight different formulation:

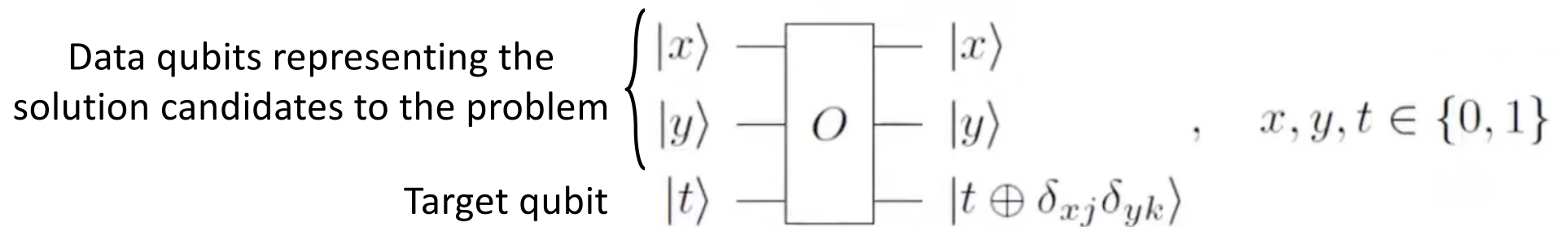


If we measure now, then all the states have equal probability.



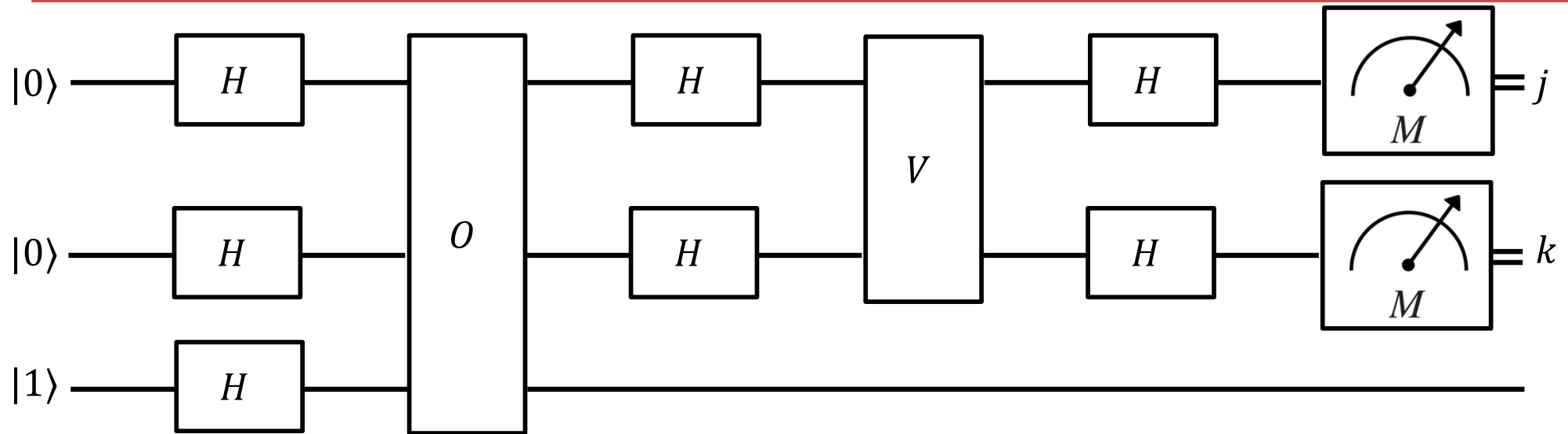
We apply a proper operator Inversion about the mean to amplify the state of interest, i.e., the one related to the solution $|x^*\rangle$.

::... Quantum Search



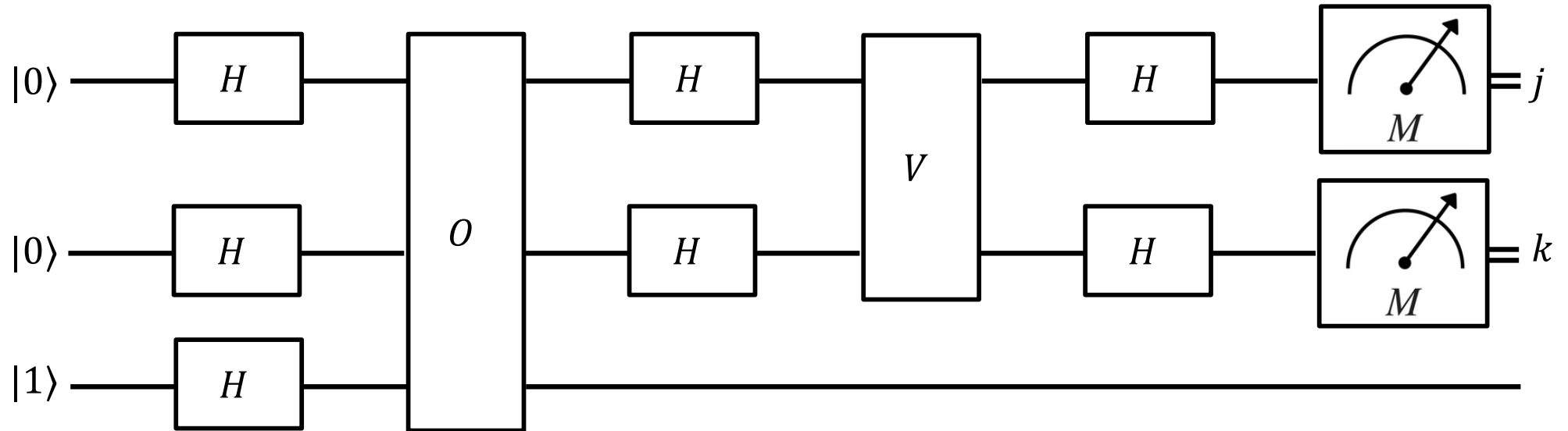
It is a three-qubit gate: two data qubits encode the query, in the sense that there are four possibilities to check, which can be labeled by the four different basis states of these two qubits. As in the previous cases, if the data qubits are in their basis states, they remain unchanged when output by the gate. Then we have the target qubit, and the oracle is designed so that the target qubit is flipped if the solution encoded by the data qubit is the correct one for the problem. Such a flipping is denoted by addition modulo 2, as in the previous algorithms. jk indicates the right solution and xy the current solution in evaluation, and if they coincide, the product is 1, and the target qubit is flipped.

... Quantum Search



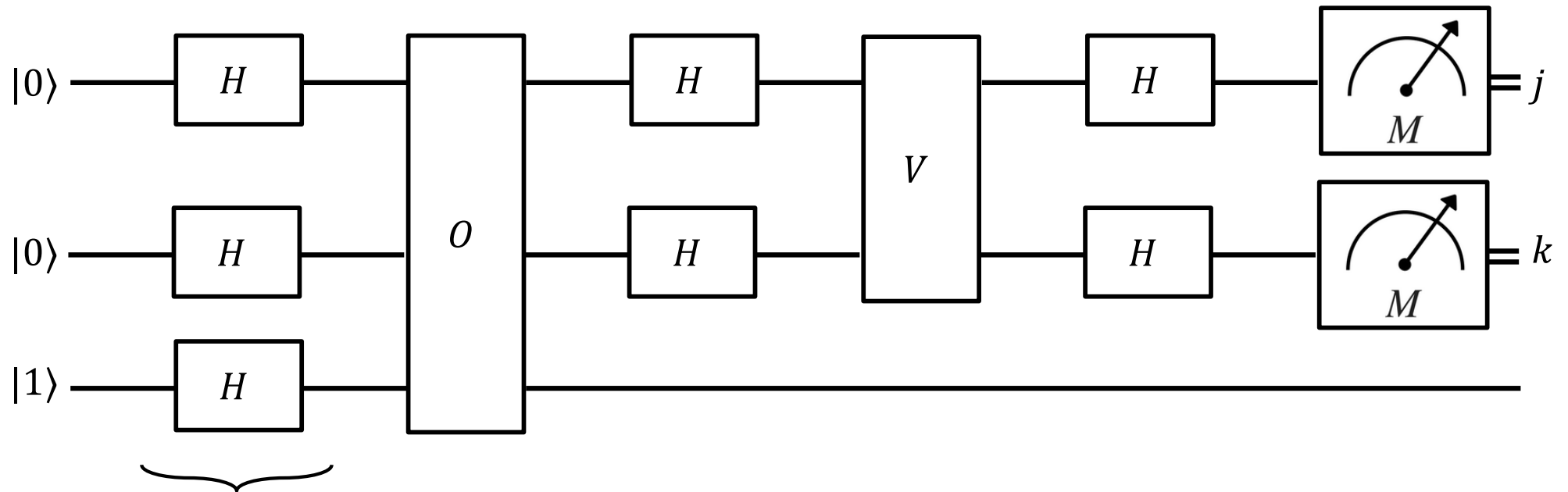
The quantum circuit that searches for the correct solution consists of three qubits coming in, the first two in the basis state $|0\rangle$ and the last one in $|1\rangle$. They all go through a Hadamard gate, and they enter the oracle. This is similar to the previous algorithm: we do not enter the oracle in the basis states; instead, we proceed in the superposition state, thanks to the application of the Hadamard gate. The last qubit after exiting the oracle is untouched, and the rest of the circuit operates on the data qubits, which undergo another Hadamard. After passing a given two-qubit gate, they are again transformed by an Hadamard and measured in the standard basis.

... Quantum Search

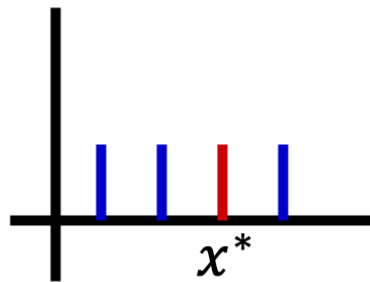


The claim is that the measurements at the end of the circuit will return the label of the correct solution. We have to run it once, and we get immediately the solution to the problem.

::... Quantum Search

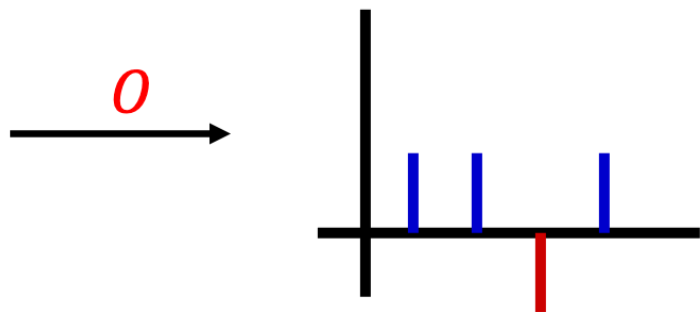
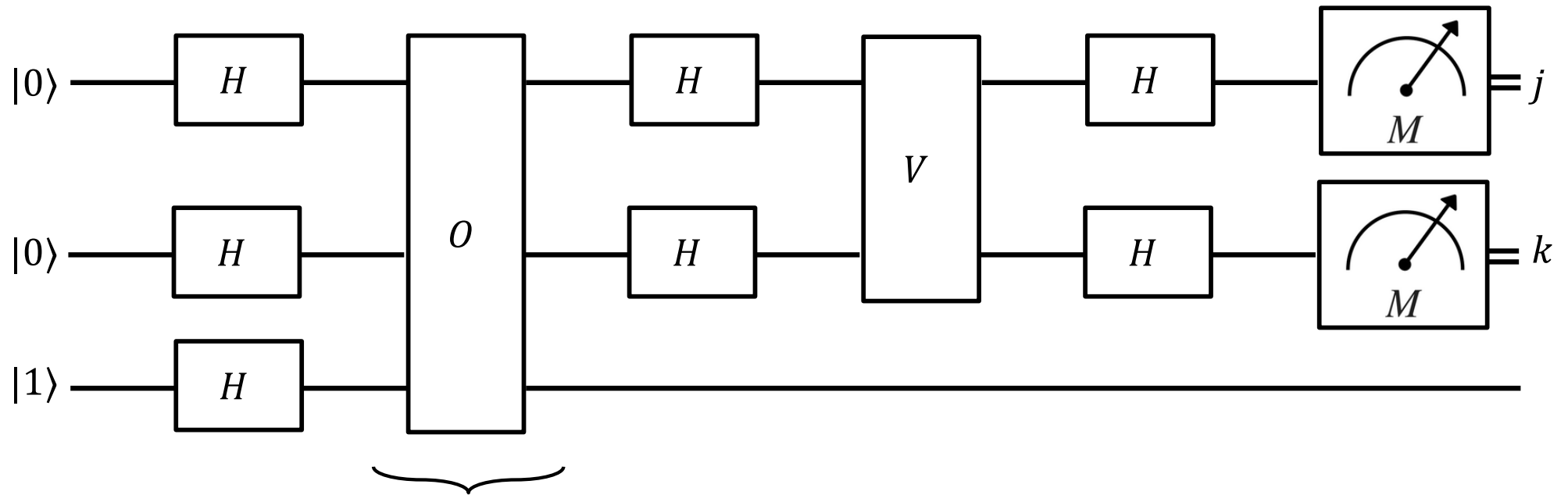


equal
amplitudes



The first stage of the circuit is needed to make the initial state in superposition of all the possible solutions in the two upper qubits.

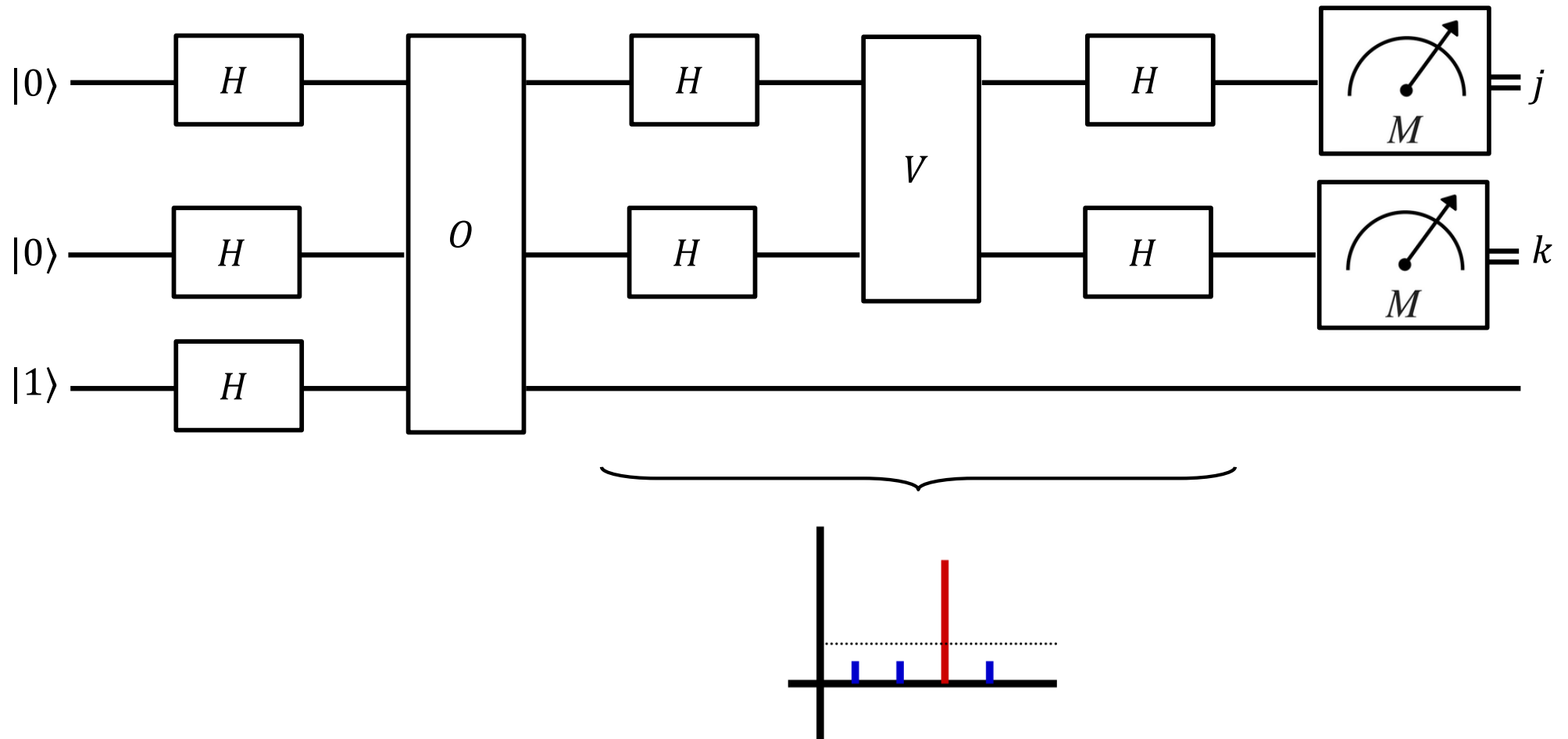
... Quantum Search



The second stage of the circuit is to have the solution state $|x^*\rangle$ being marked with a minus sign.

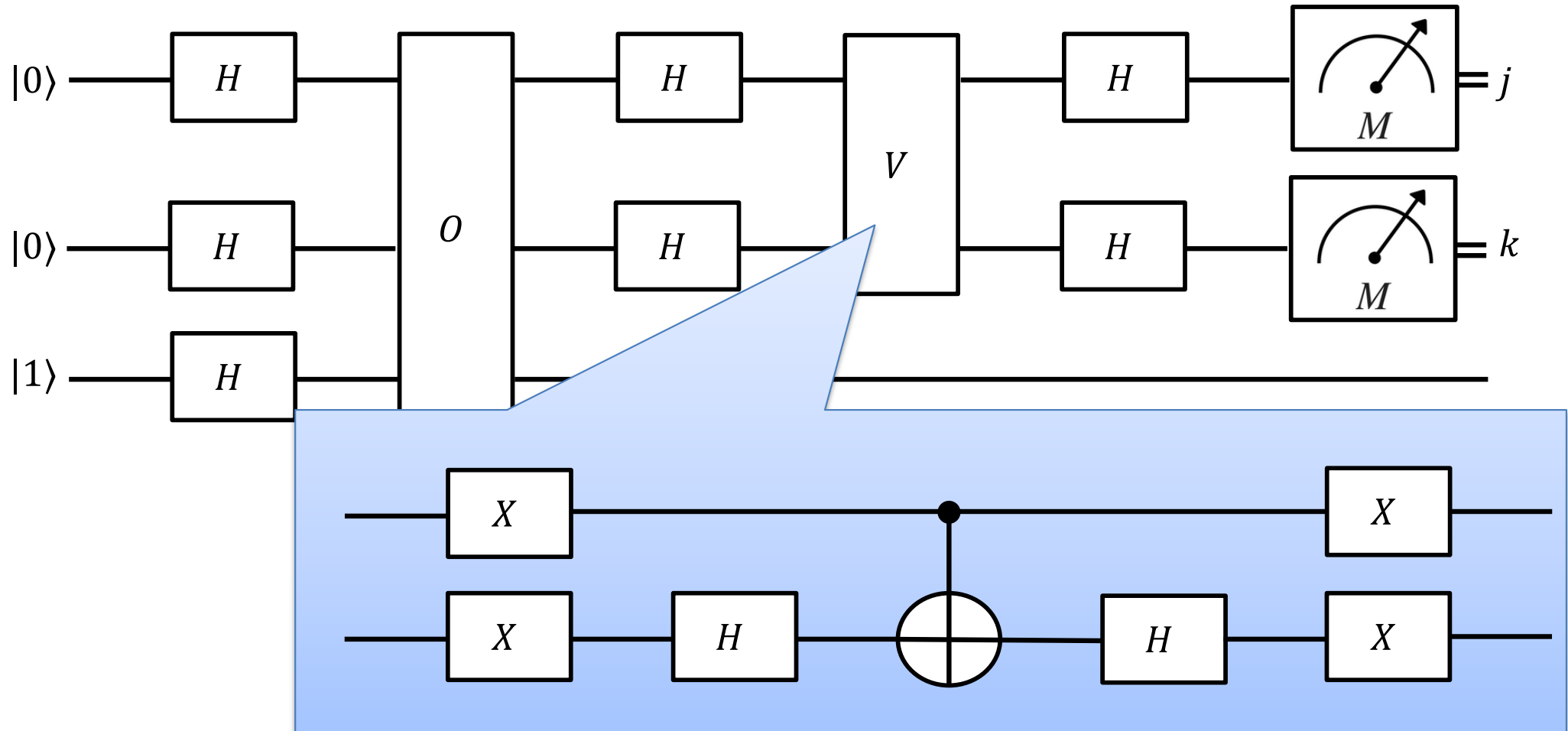
$$O|x\rangle := (-1)^{f(x)}|x\rangle, \forall x \in \{0,1\}^n$$

::... Quantum Search



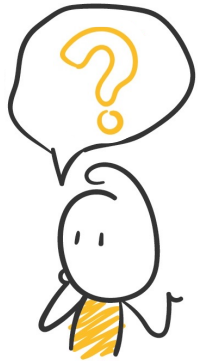
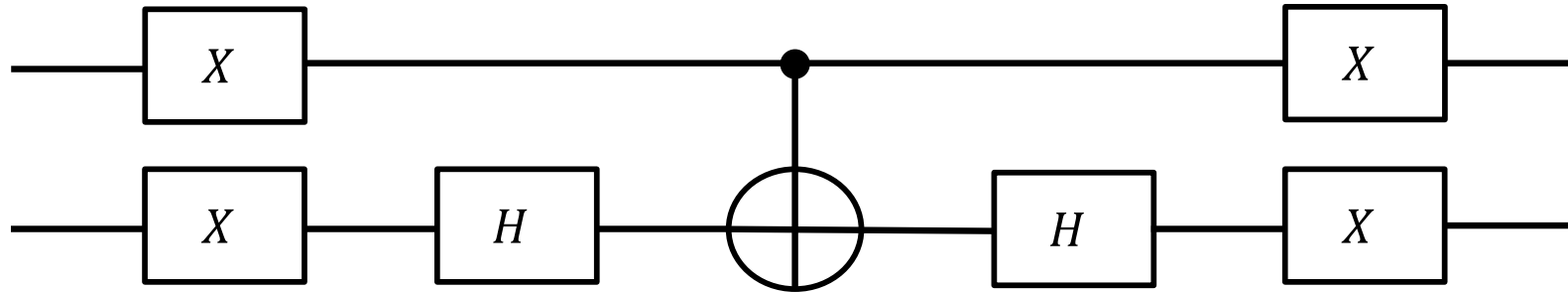
The last stage of the circuit is to have the solution state $|x^*\rangle$ with the highest amplitude so as to be the one measured.

::... Quantum Search



Both qubits go through the Pauli-X, the lower one is transformed with Hadamard. The C-NOT is applied on the lower qubit controlled by the upper one. Then, the gates in the first part are placed in the opposing order: first Hadamard on the lower and Pauli X on both qubits.

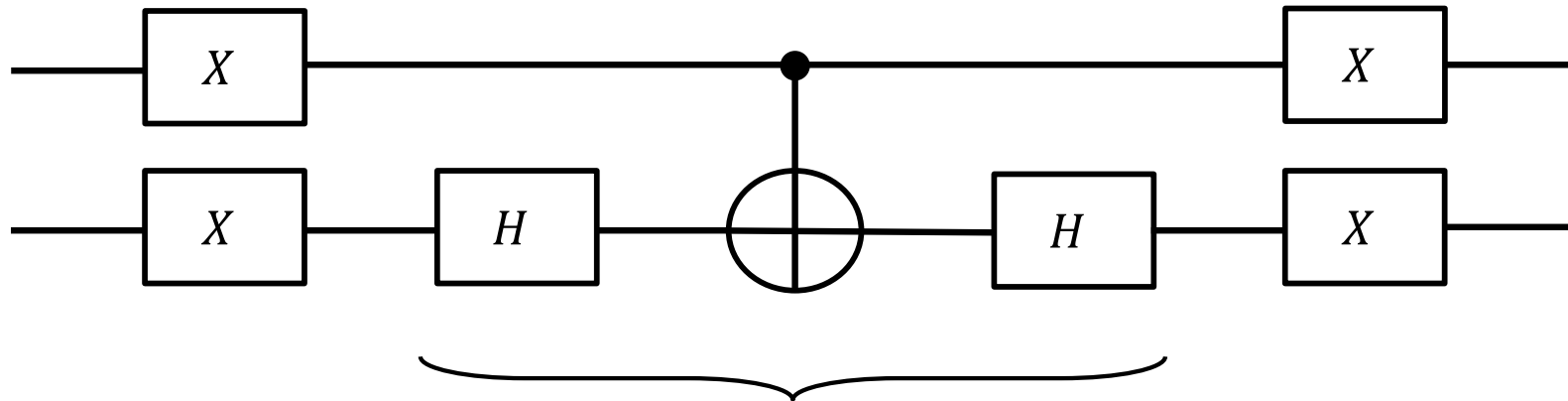
::... Quantum Search



What is the output of this gate?

Based on the states of the input qubits,
what kind of states will the output qubits have?

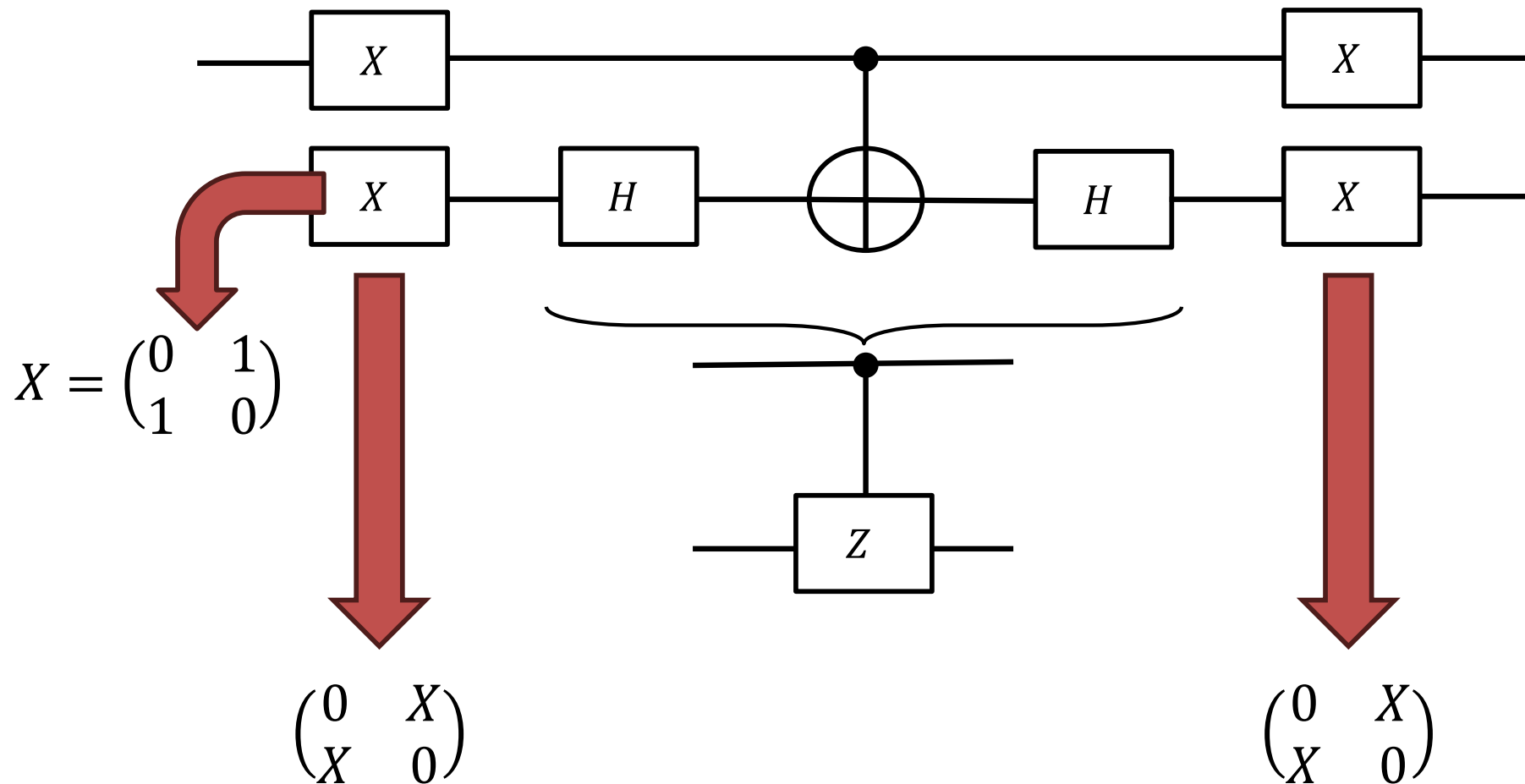
::... Quantum Search



$$\begin{aligned}
 (I \otimes H)CX(I \otimes H) &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix} \\
 &= \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix} \\
 &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}
 \end{aligned}$$

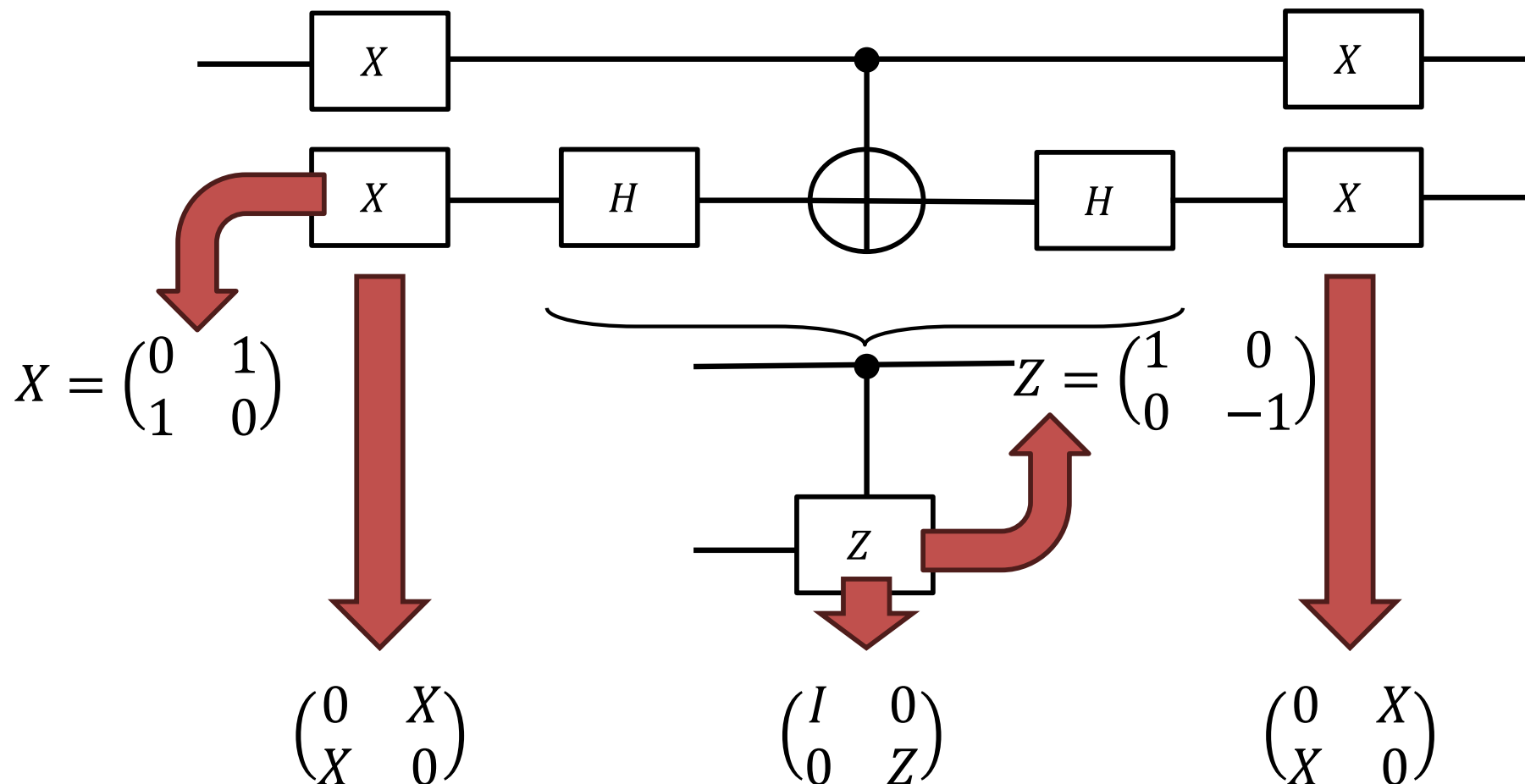
The central part of the circuit can be reduced to a controlled Pauli-Z, by working with the matrix representation of the three gates, this is evident.

::... Quantum Search



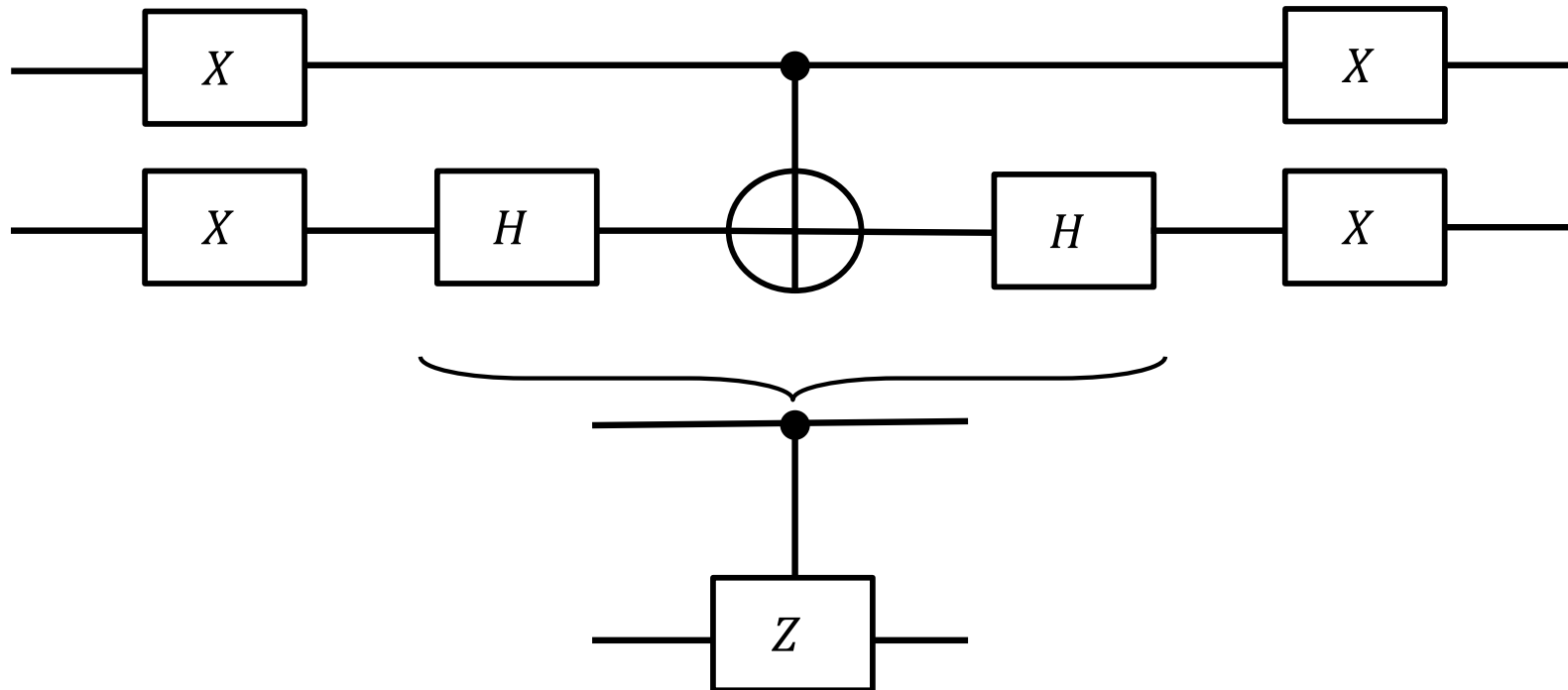
To understand what this circuit does, we need to look at the matrix representation of the various involved gates. Each gate has a 2x2 matrix representation, while the parallel processing of 2 qubits has a 4x4 matrix.

... Quantum Search



To understand what this circuit does, we need to look at the matrix representation of the various involved gates. Each gate has a 2x2 matrix representation, while the parallel processing of 2 qubits has a 4x4 matrix.

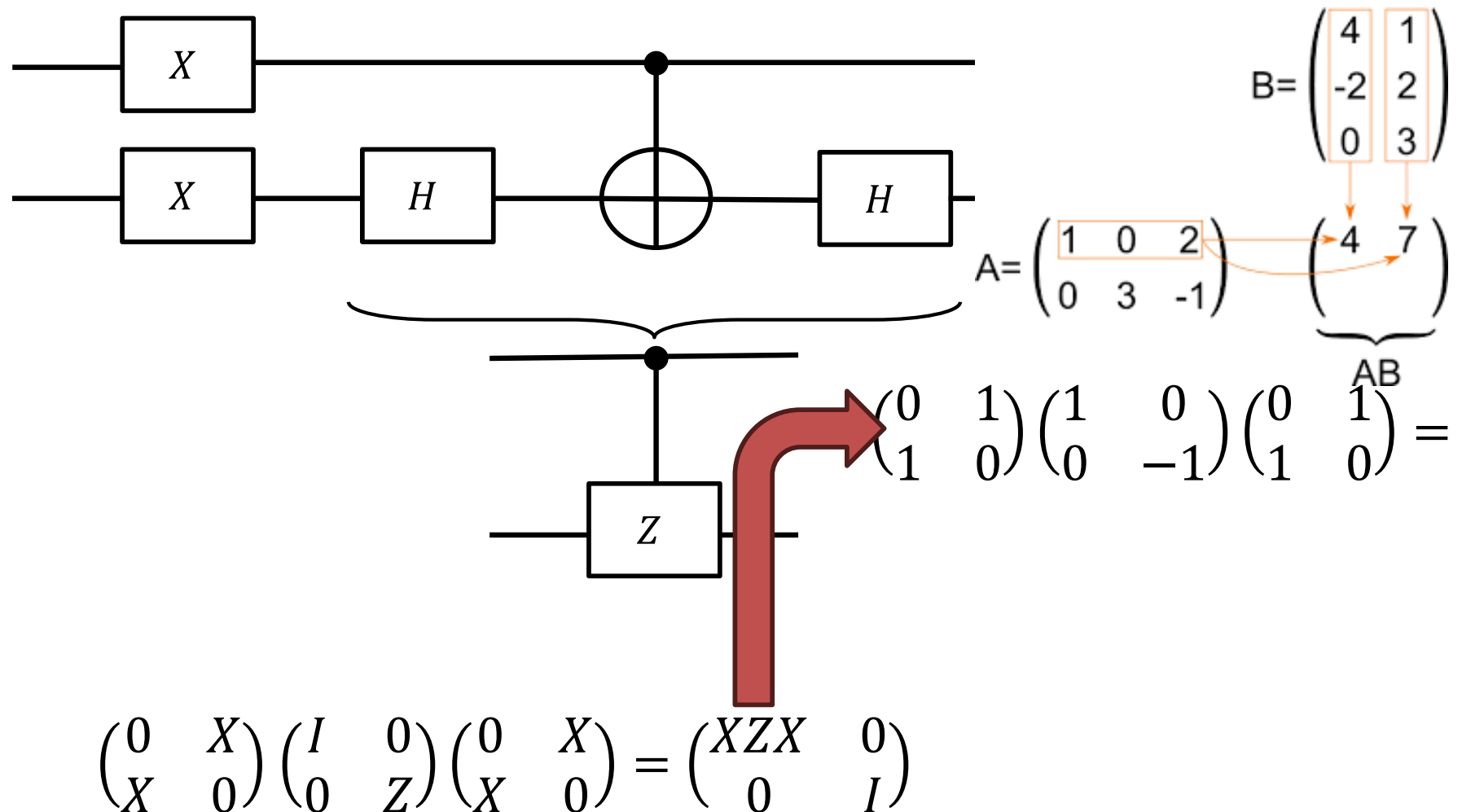
::... Quantum Search



$$\begin{pmatrix} 0 & X \\ X & 0 \end{pmatrix} \begin{pmatrix} I & 0 \\ 0 & Z \end{pmatrix} \begin{pmatrix} 0 & X \\ X & 0 \end{pmatrix}$$

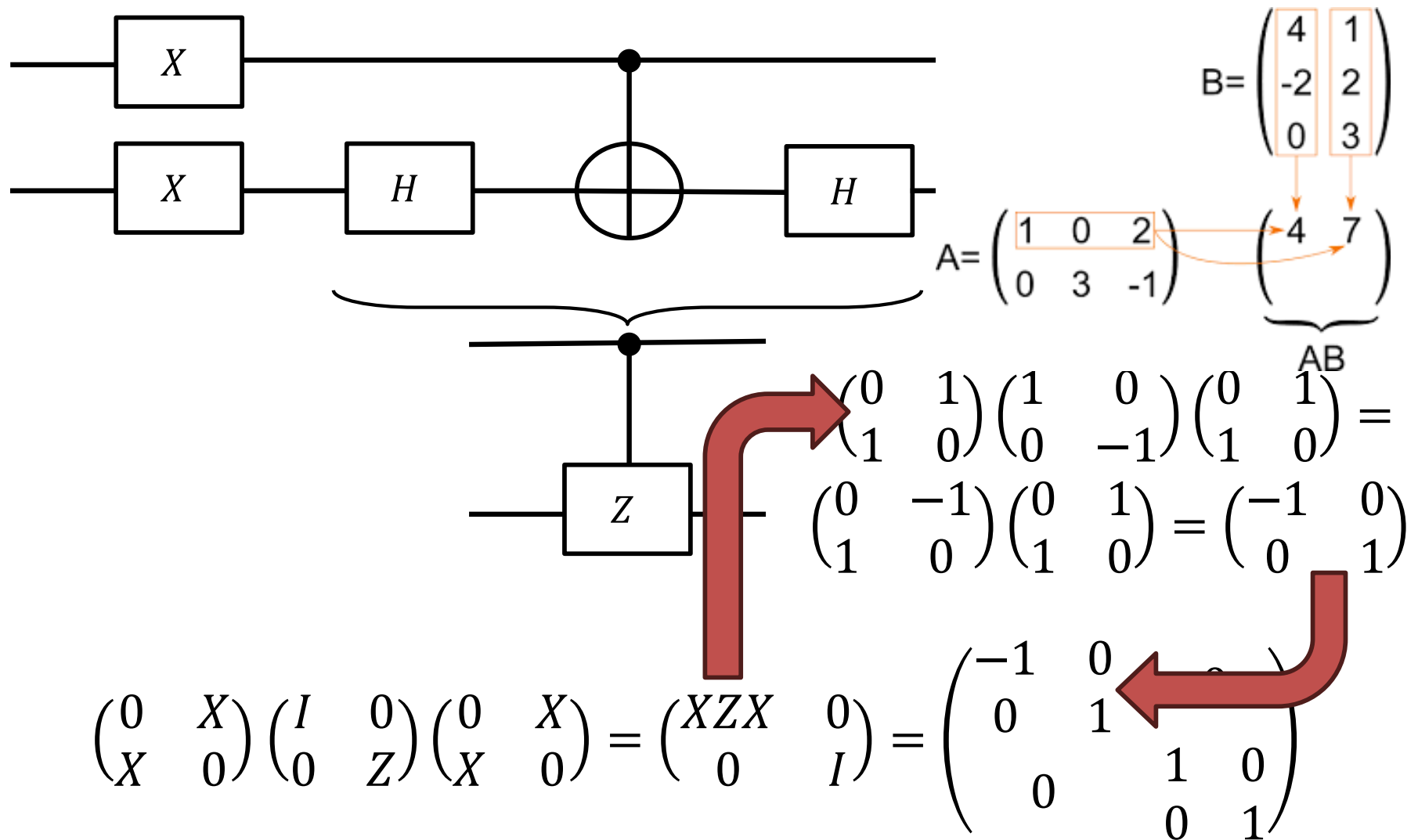
We have to multiply these matrix in order to have the matricial representation of the transformation realised by the overall circuit.

... Quantum Search



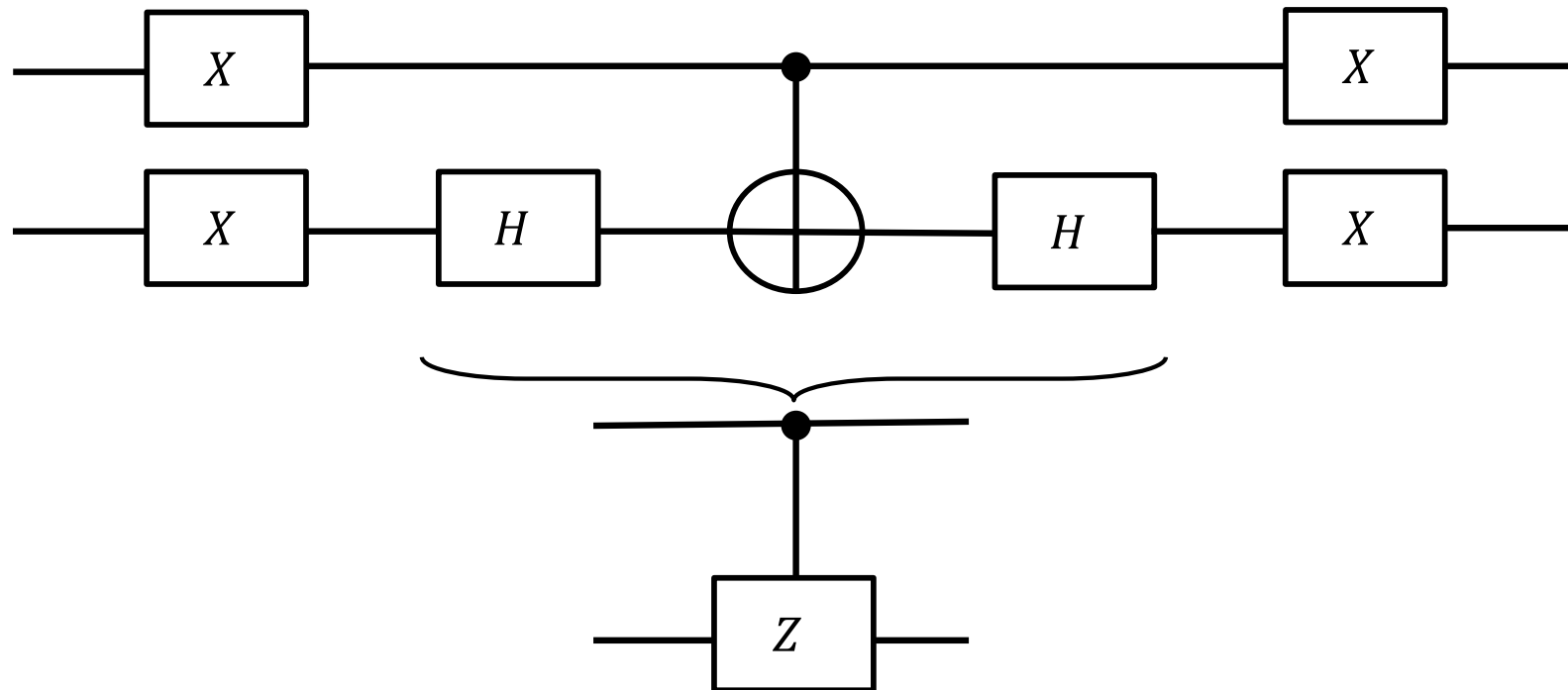
We have to multiply these matrix in order to have the matricial representation of the transformation realised by the overall circuit.

... Quantum Search



We have to multiply these matrix in order to have the matricial representation of the transformation realised by the overall circuit.

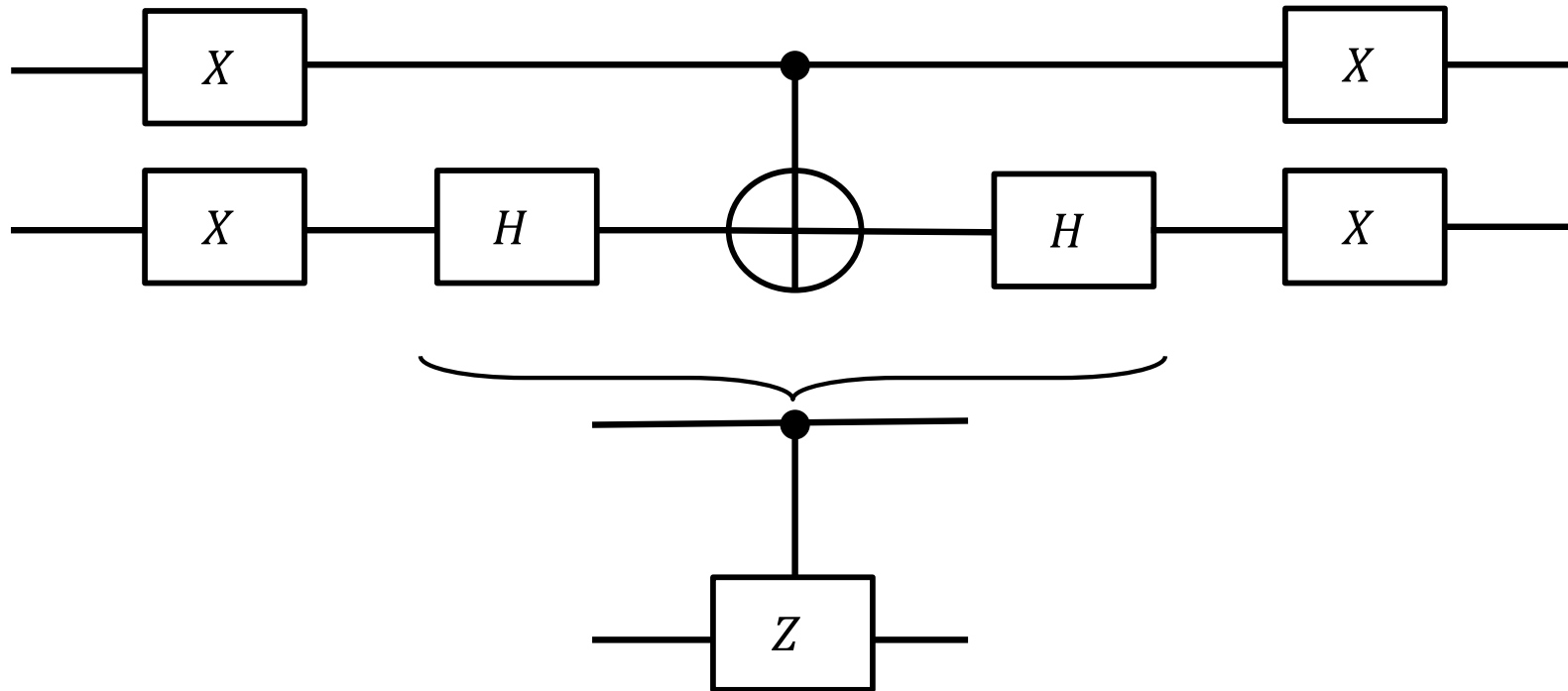
... Quantum Search



$$\begin{pmatrix} 0 & X \\ X & 0 \end{pmatrix} \begin{pmatrix} I & 0 \\ 0 & Z \end{pmatrix} \begin{pmatrix} 0 & X \\ X & 0 \end{pmatrix} = \begin{pmatrix} XZX & 0 \\ 0 & I \end{pmatrix} = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

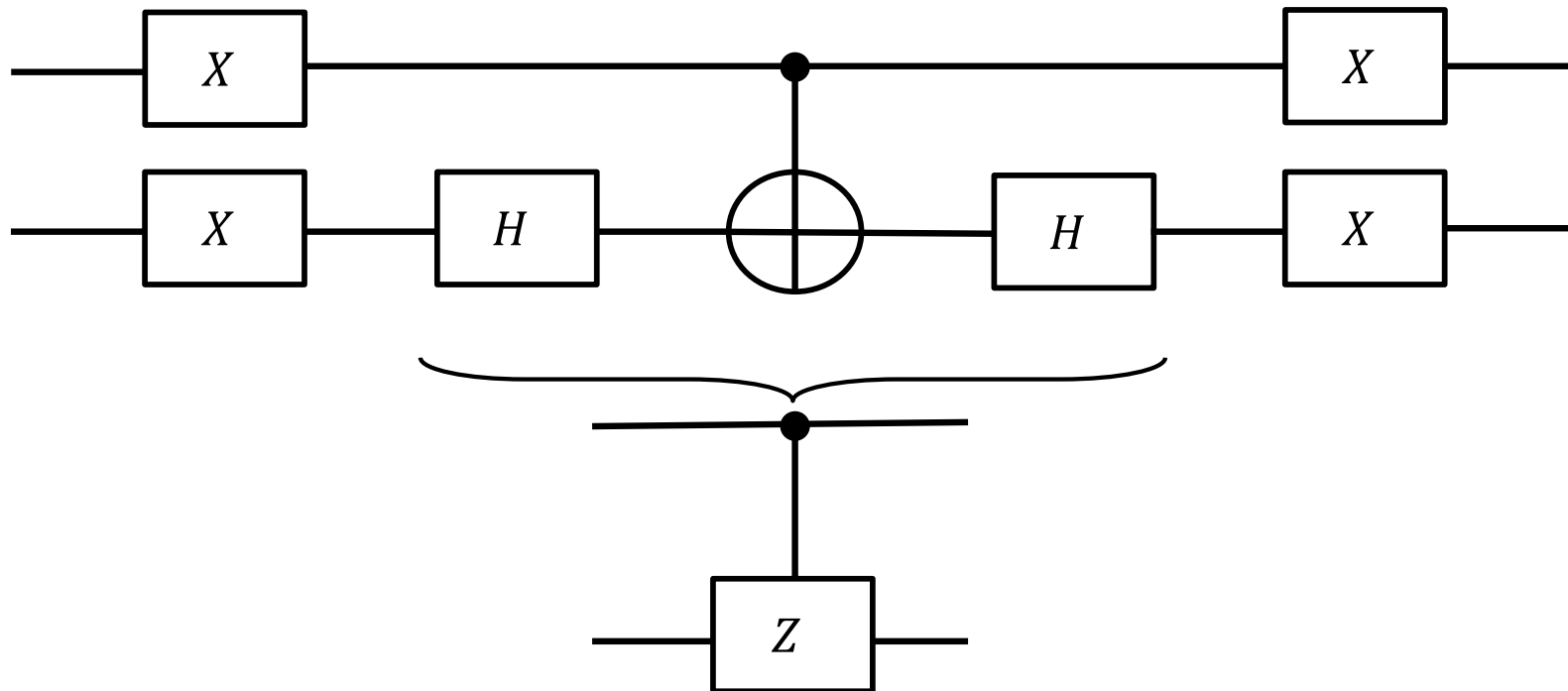
We have to multiply these matrix in order to have the matricial representation of the transformation realised by the overall circuit.

::... Quantum Search



$$\begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix} \propto \begin{pmatrix} 2 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

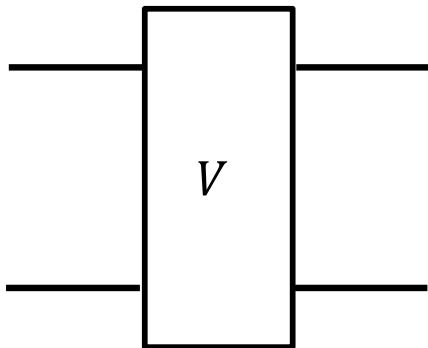
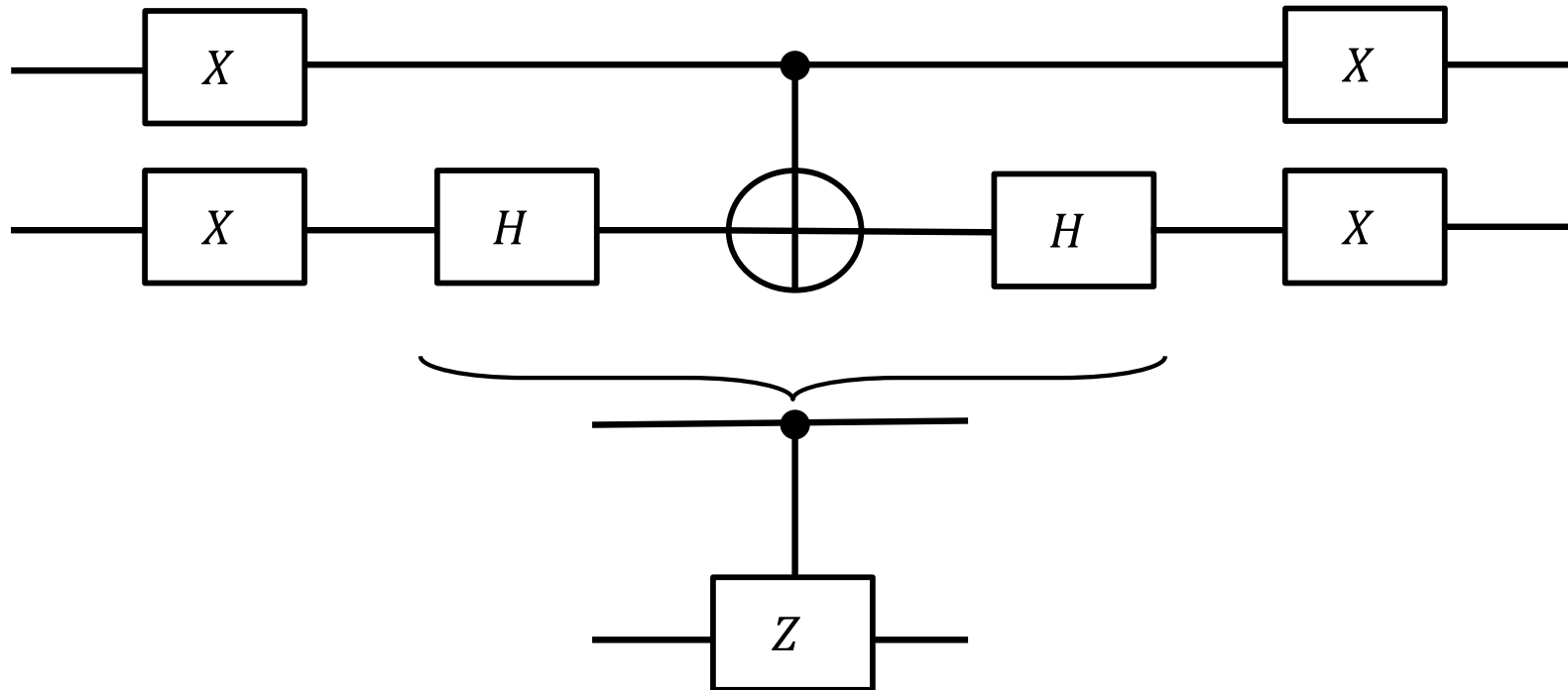
::... Quantum Search



$$\propto \underbrace{\begin{pmatrix} 2 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}}_{2|00\rangle\langle 00|} - \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_I$$

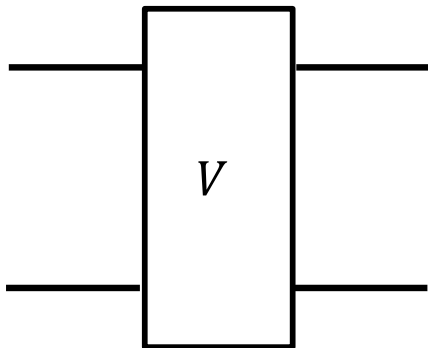
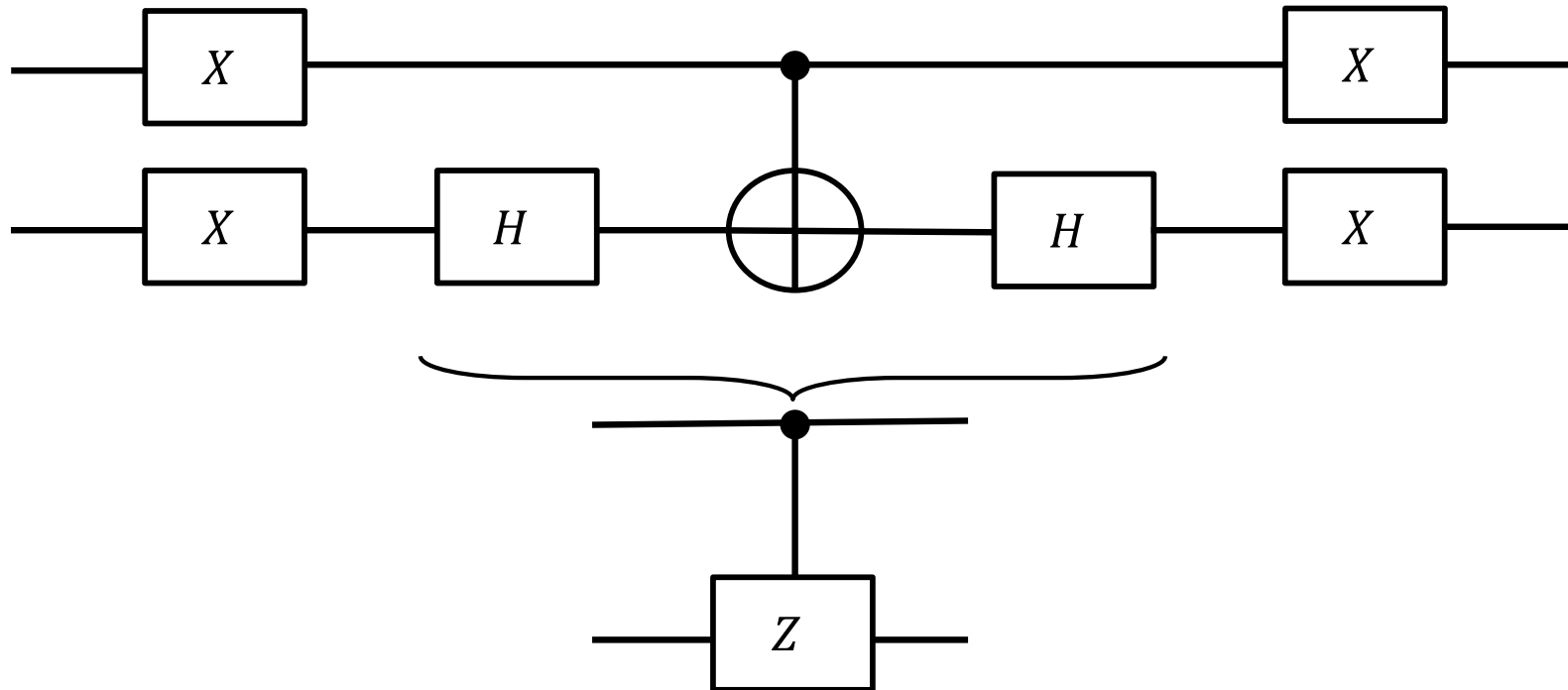
The first is the projection onto the $|00\rangle$ state multiplied by 2; the second is the identity.

::... Quantum Search



$$\hat{V} = 2|00\rangle\langle 00| - \hat{I}$$

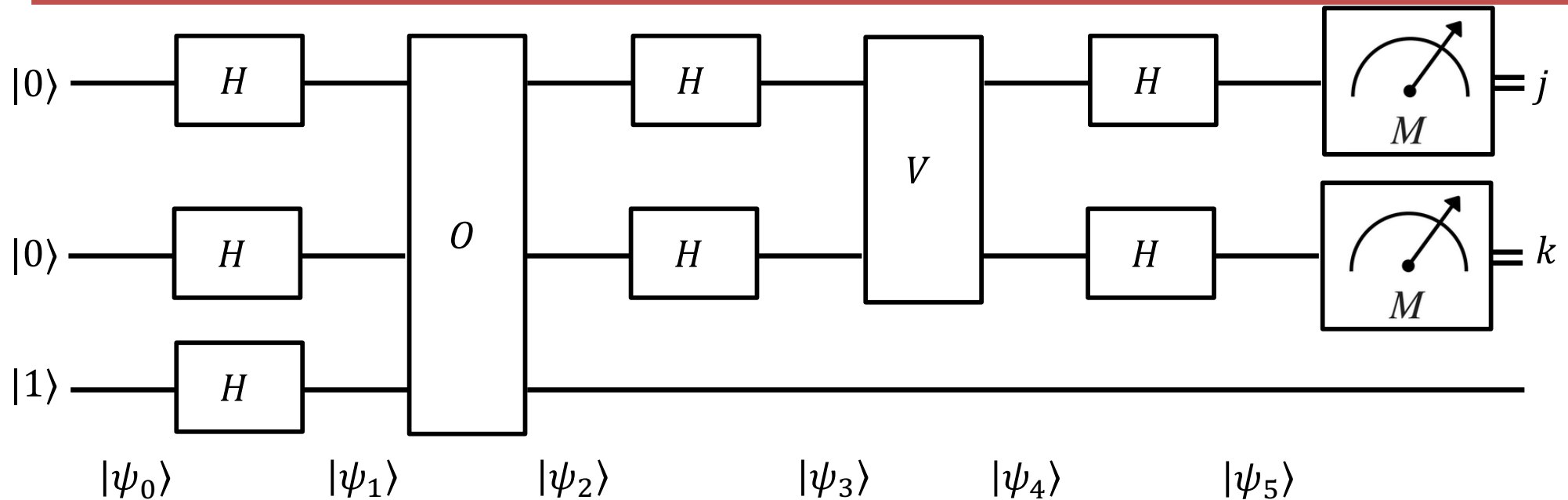
::... Quantum Search



$$\hat{V} = 2|00\rangle\langle 00| - \hat{I}$$

This is Reflection Gate R_{00} as it reflects a quantum state across the $|00\rangle$ state, and applying the phase shift to everything except the $|00\rangle$ state.

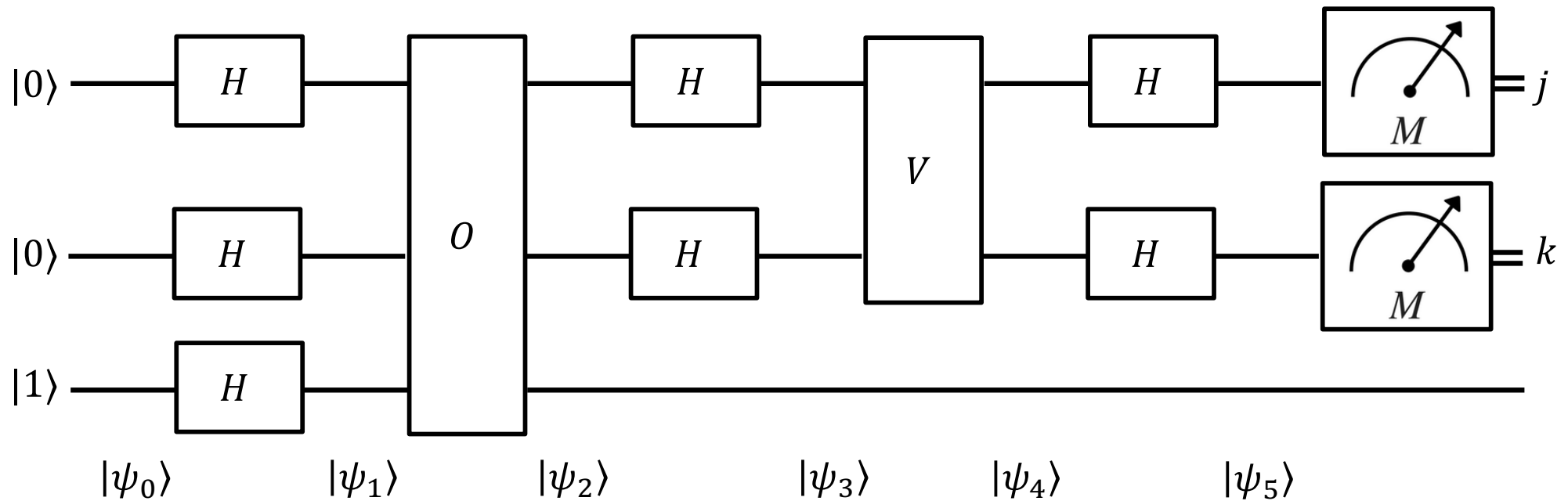
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

1. In the first stage we have Hadamard applied to all of the three input qubits: $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}$.

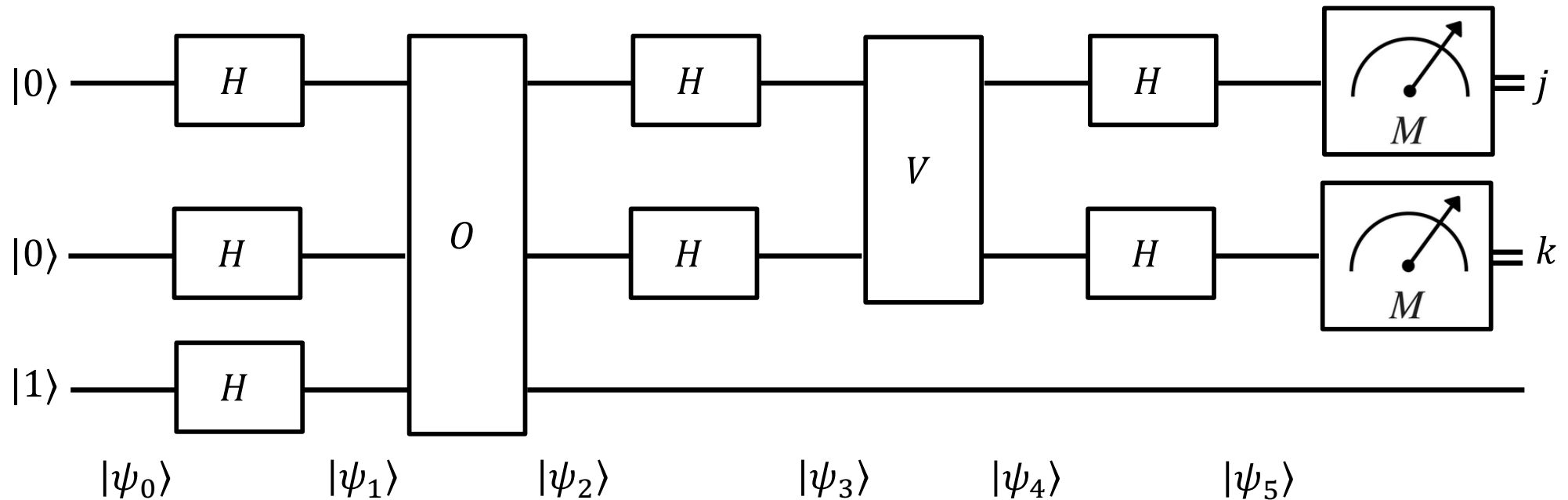
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$
2. The three qubits enter the oracle and the first two qubits are unchanged if in the basis states while the last one is flipped if the two upper qubits represent the correct solution, namely $x_0 \in \{0,1\}^2$:
 $|\psi_2\rangle = \frac{1}{2\sqrt{2}} [|x_0\rangle \otimes (|1\rangle - |0\rangle) + \sum_{x \neq x_0} |x\rangle \otimes (|0\rangle - |1\rangle)].$

... Quantum Search



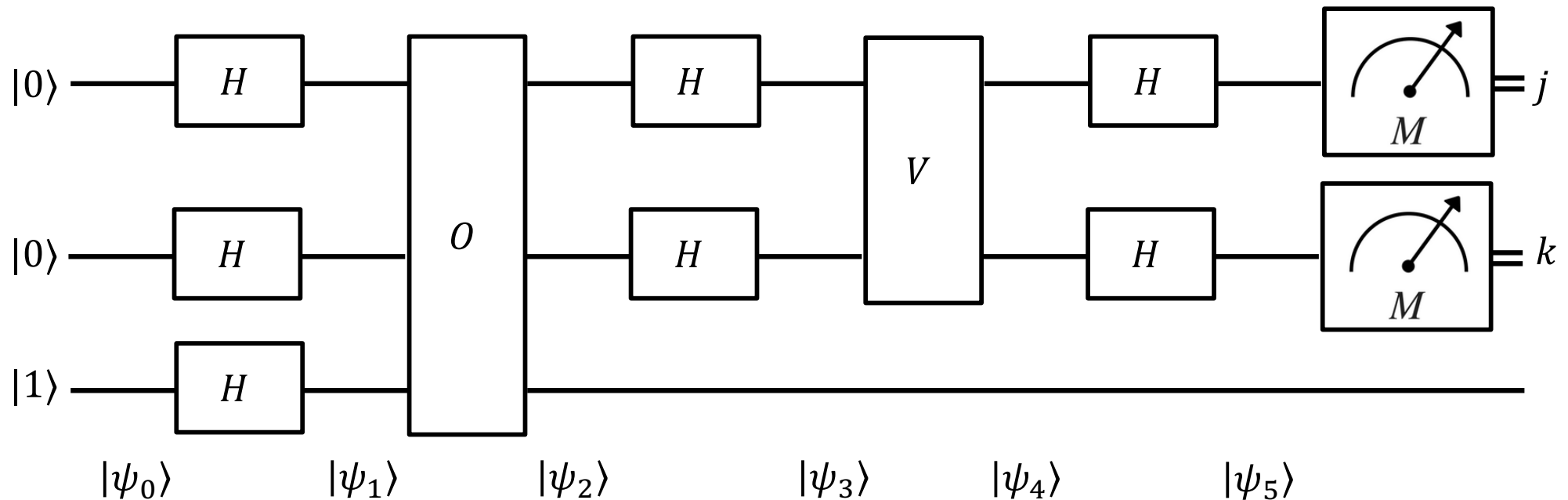
Let's go through the different stages of the calculation of the state evolution:

$$1. |\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

2. The three qubits enter the oracle and the first two qubits are unchanged if in the basis states while the last one is flipped if the two upper qubits represent the correct solution, namely $x_0 \in \{0,1\}^2$:

$$|\psi_2\rangle = \frac{1}{2\sqrt{2}} \left[|x_0\rangle \otimes (|1\rangle - |0\rangle) + \sum_{x \neq x_0} |x\rangle \otimes (|0\rangle - |1\rangle) \right].$$

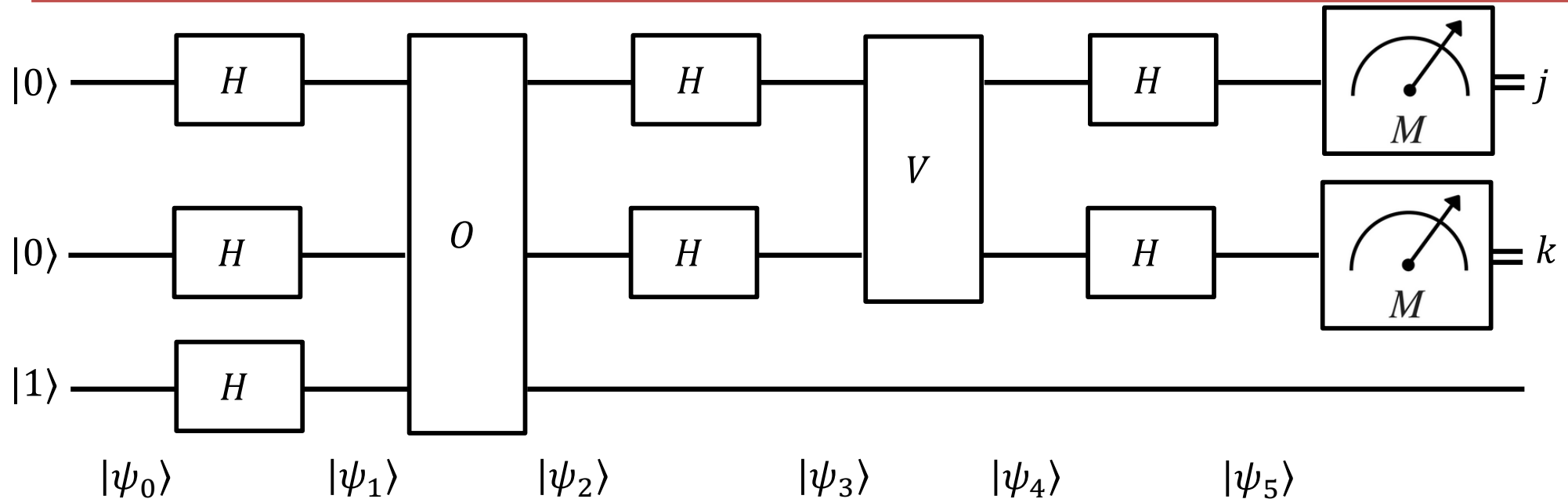
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}$.
2. $|\psi_2\rangle \propto [|x_0\rangle - \sum_{x \neq x_0} |x\rangle] \otimes |-\rangle$, now the target qubit has done its work and we can ignore it in the rest of the computation.

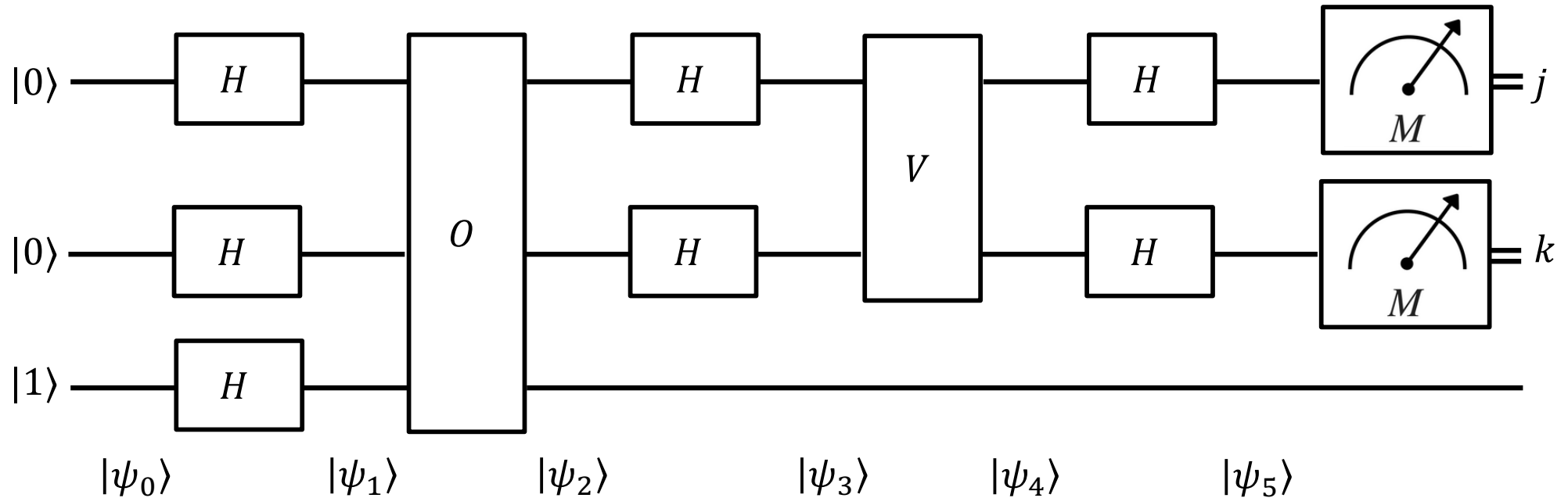
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$
2. $|\psi_2\rangle \propto [|x_0\rangle - \sum_{x \neq x_0} |x\rangle] \otimes |-\rangle.$
3. The qubits pass through Hadamard again, and we have the following output $|\psi_3\rangle \propto \sum_{z \in \{0,1\}^2} (-1)^{z \cdot x_0} |z\rangle - \sum_{\substack{z \in \{0,1\}^2 \\ z \neq x_0}} (-1)^{z \cdot x} |z\rangle$

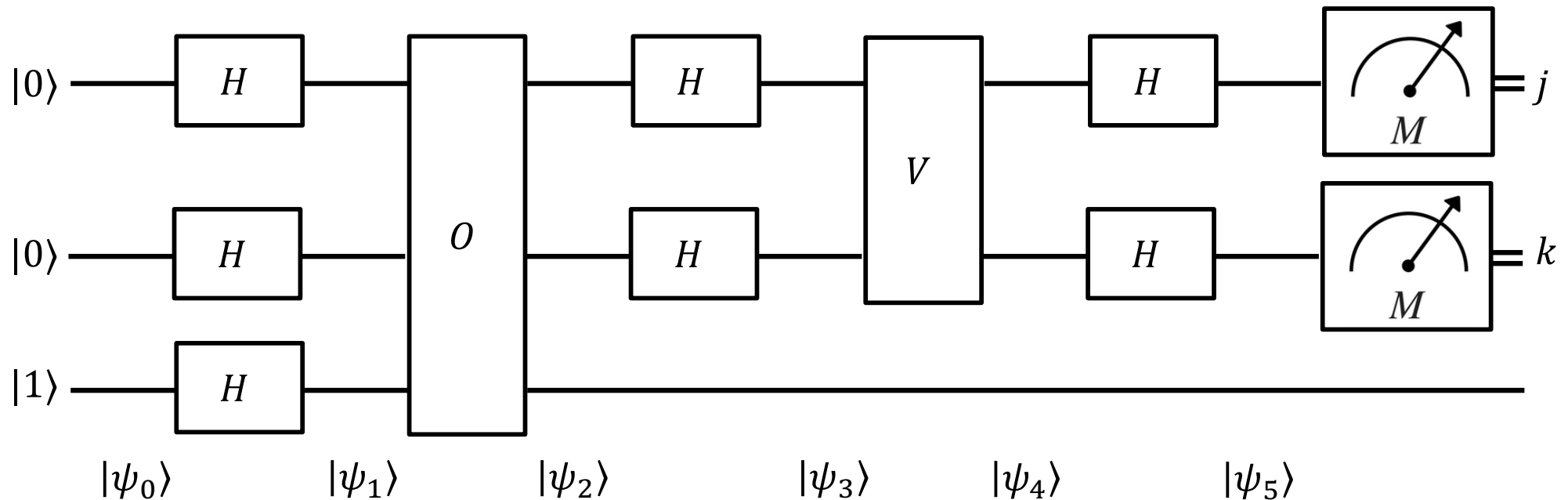
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$
2. $|\psi_2\rangle \propto [|x_0\rangle - \sum_{x \neq x_0} |x\rangle] \otimes |-\rangle.$
3. $|\psi_3\rangle \propto \sum_{z \in \{0,1\}^2} [(-1)^{z \cdot x_0} - \sum_{x \neq x_0} (-1)^{z \cdot x}] |z\rangle.$

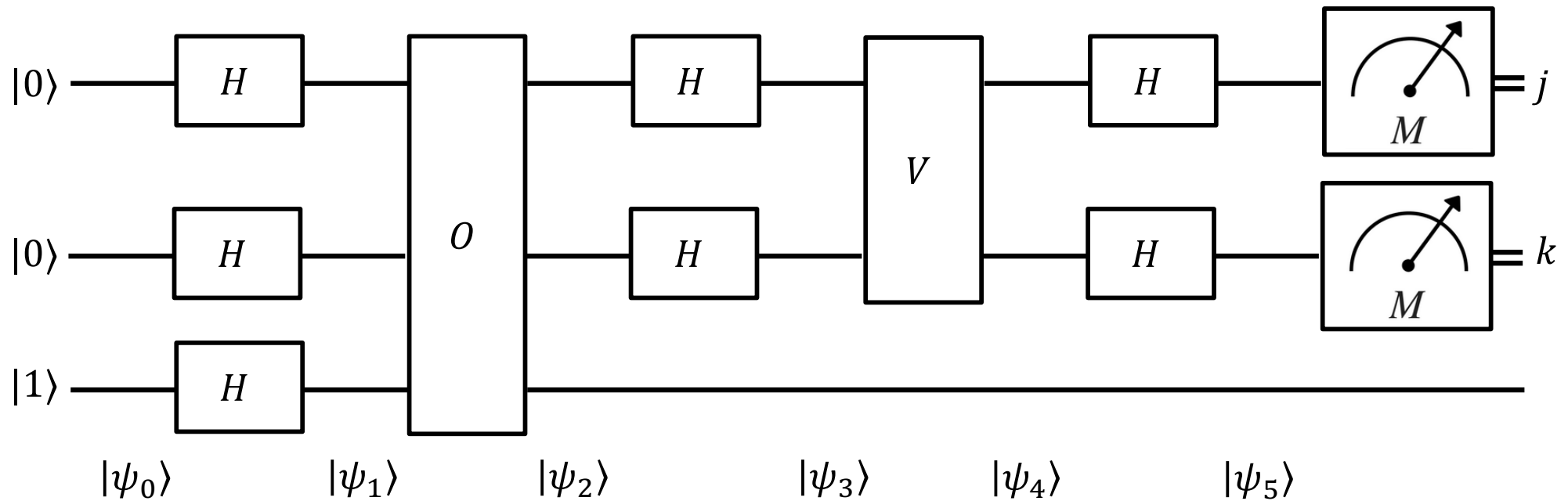
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

1. $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$
2. $|\psi_2\rangle \propto [|x_0\rangle - \sum_{x \neq x_0} |x\rangle] \otimes |-\rangle.$
3. $|\psi_3\rangle \propto \sum_{z \in \{0,1\}^2} [(-1)^{z \cdot x_0} - \sum_{x \neq x_0} (-1)^{z \cdot x}] |z\rangle.$
4. $|\psi_4\rangle = \hat{V}|\psi_3\rangle = 2|00\rangle\langle 00|\psi_3\rangle - |\psi_3\rangle$

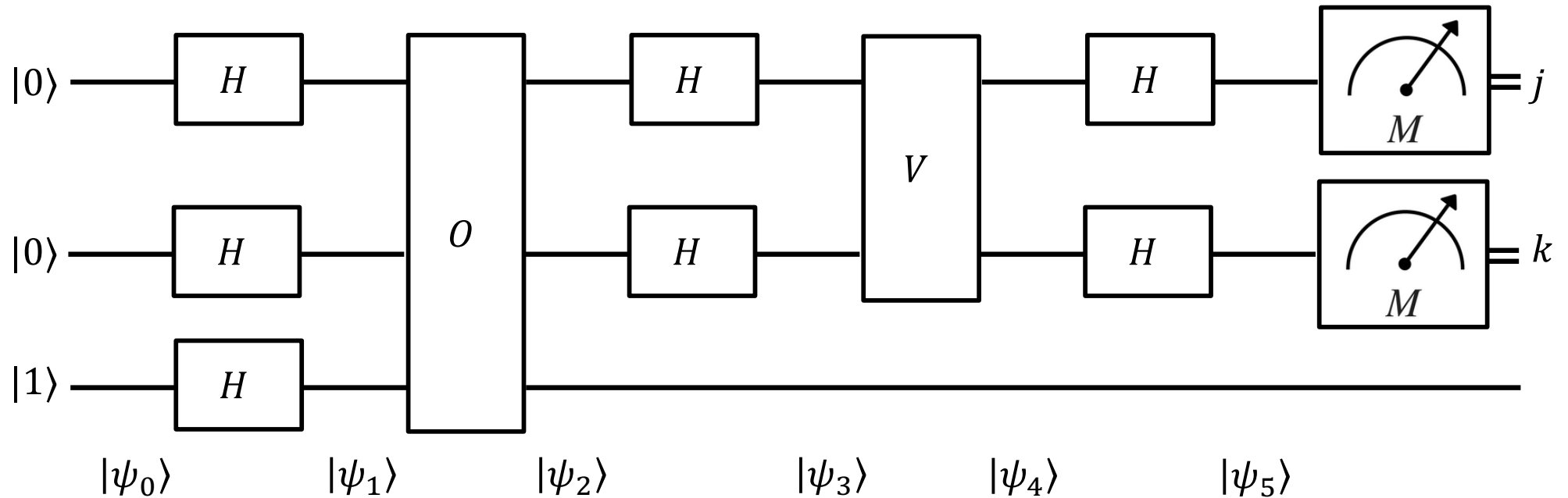
... Quantum Search



Let's go through the different stages of the calculation of the state evolution:

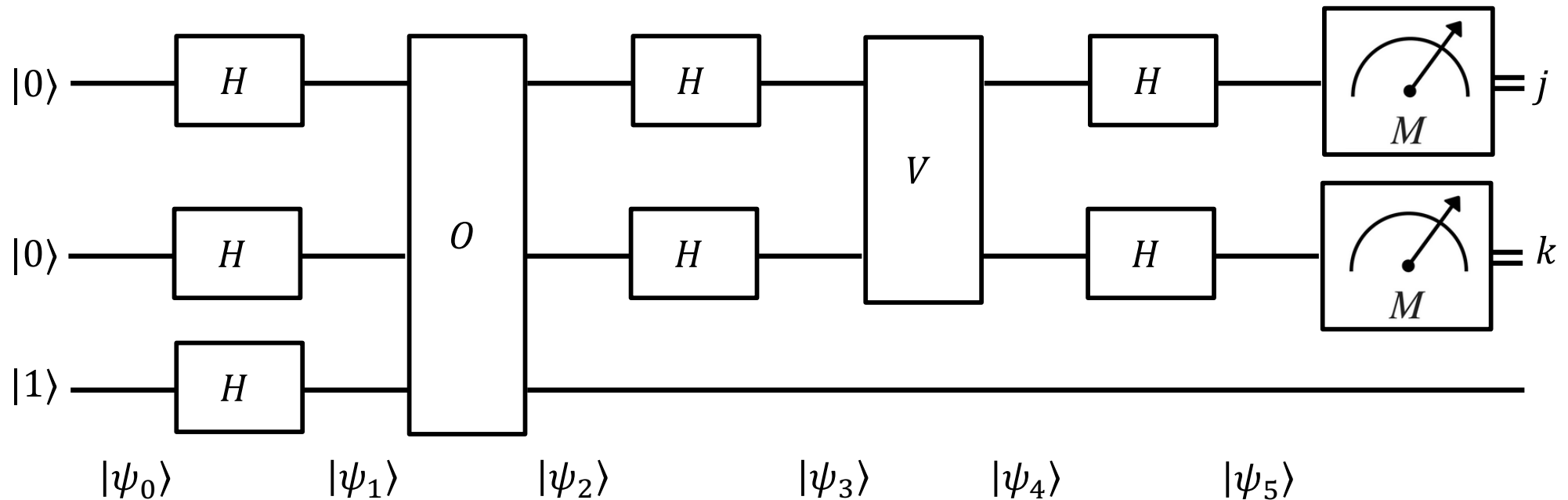
1. $|\psi_1\rangle = \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$
2. $|\psi_2\rangle \propto [|x_0\rangle - \sum_{x \neq x_0} |x\rangle] \otimes |-\rangle.$
3. $|\psi_3\rangle \propto \sum_{z \in \{0,1\}^2} [(-1)^{z \cdot x_0} - \sum_{x \neq x_0} (-1)^{z \cdot x}] |z\rangle.$
4. $|\psi_4\rangle = \hat{V}|\psi_3\rangle = \frac{2}{-4} |00\rangle \langle 00| \psi_3\rangle - |\psi_3\rangle$
 $\propto \boxed{2[1 - 3]} |00\rangle - \sum_{z \in \{0,1\}^2} [(-1)^{z \cdot x_0} - \sum_{x \neq x_0} (-1)^{z \cdot x}] |z\rangle.$

... Quantum Search



4. $|\psi_4\rangle = 2[1 - 3]|00\rangle - \sum_{z \in \{0,1\}^2} [(-1)^{z \cdot x_0} - \sum_{x \neq x_0} (-1)^{z \cdot x}] |z\rangle$.
5. Now we apply again Hadamard, remembering that the second term of $|\psi_4\rangle$ was the result of an Hadarmard, so now we are undoing it:
 $|\psi_5\rangle \propto -4 \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle - 2|x_0\rangle + \sum_{x \neq x_0} 2|x\rangle$

... Quantum Search



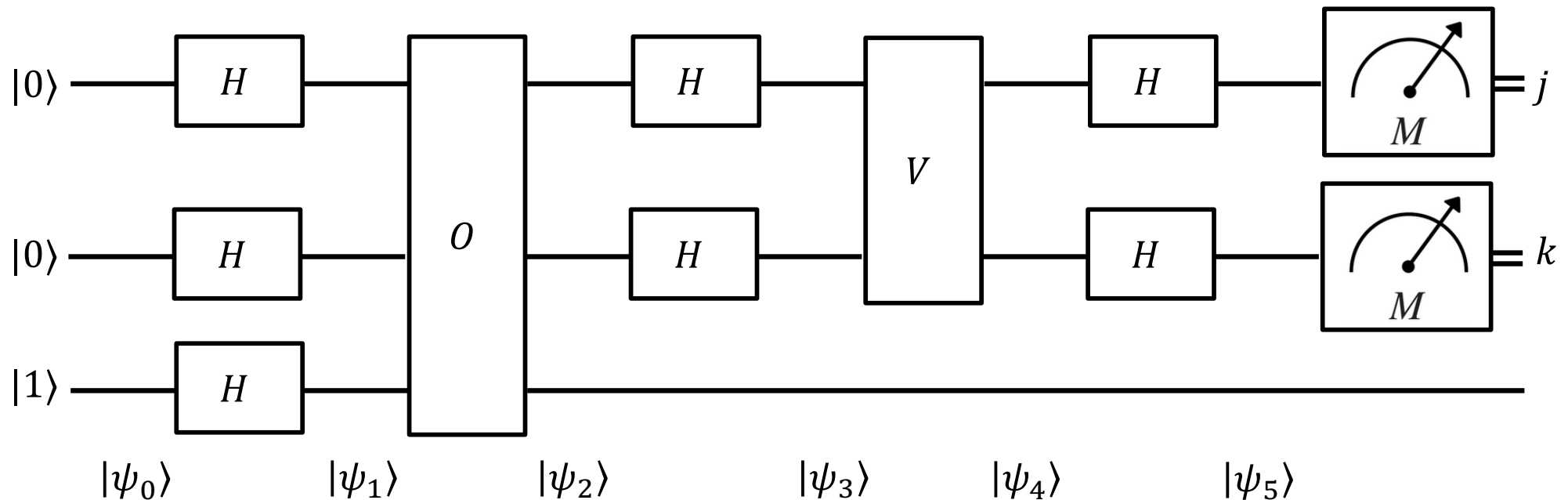
4. $|\psi_4\rangle = 2[1 - 3]|00\rangle - \sum_{z \in \{0,1\}^2} [(-1)^{z \cdot x_0} - \sum_{x \neq x_0} (-1)^{z \cdot x}] |z\rangle.$

5. Now we apply again Hadamard, remembering that the second term of $|\psi_4\rangle$ was the result of an Hadarmard, so now we are undoing it:

$$|\psi_5\rangle \propto -4 \frac{1}{2} \sum_{x \in \{0,1\}^2} |x\rangle - 2|x_0\rangle + \sum_{x \neq x_0} 2|x\rangle$$

$$\propto -2|x_0\rangle - 2 \sum_{x \neq x_0} |x\rangle - 2|x_0\rangle + \sum_{x \neq x_0} 2|x\rangle \propto |x_0\rangle$$

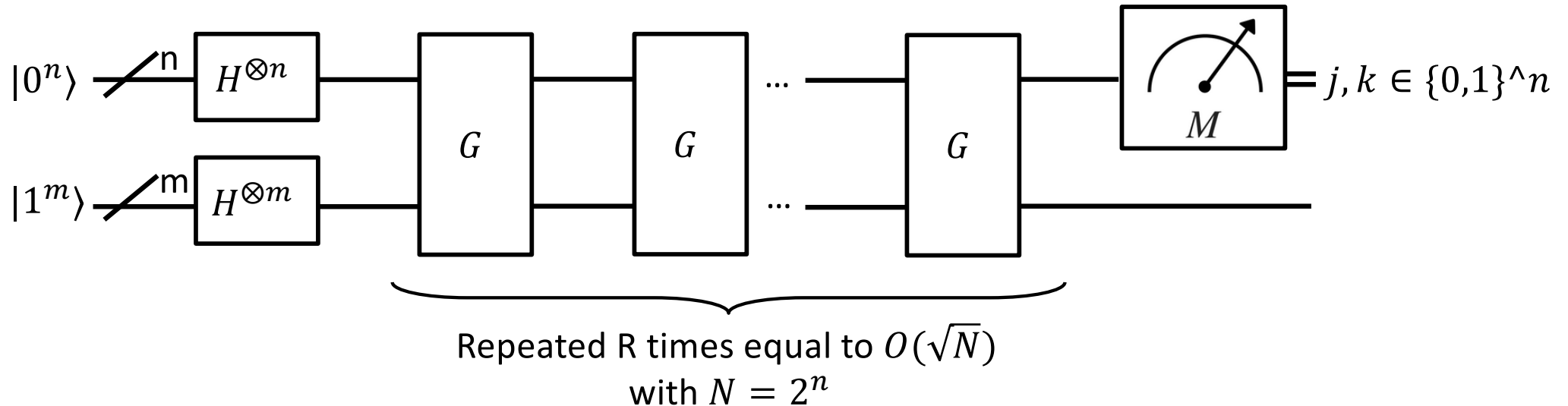
... Quantum Search



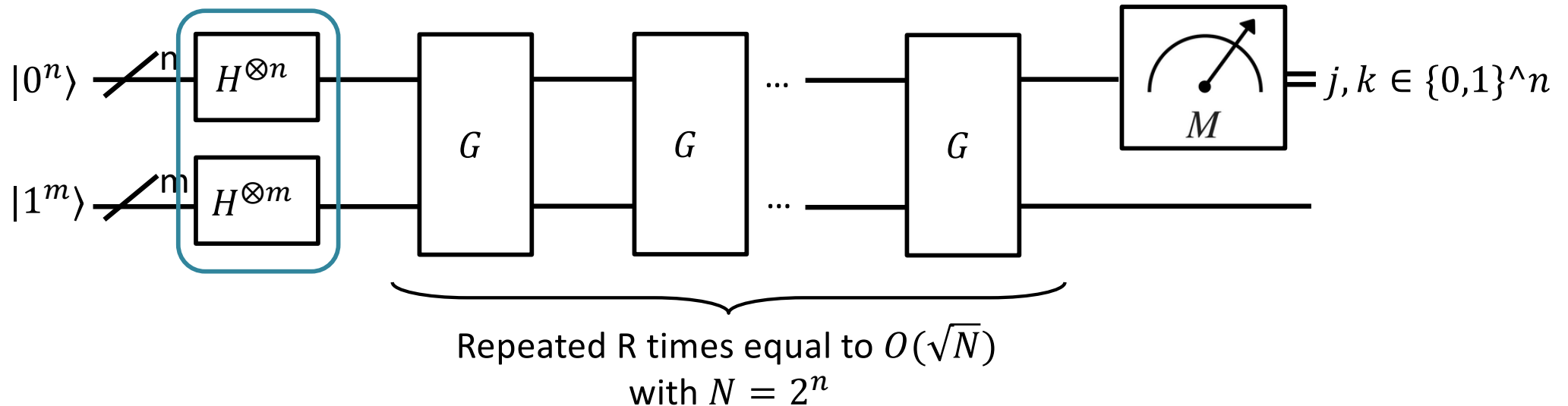
We have shown that with one run and one call to the query gate, we are able to obtain the correct solution to the problem.

This is similar to the Deutsch-Jozsa circuit, with the addition of the two-qubit gate V , a feature found in a more generalization of this algorithm called the Grover algorithm.

::... Grover's Algorithm

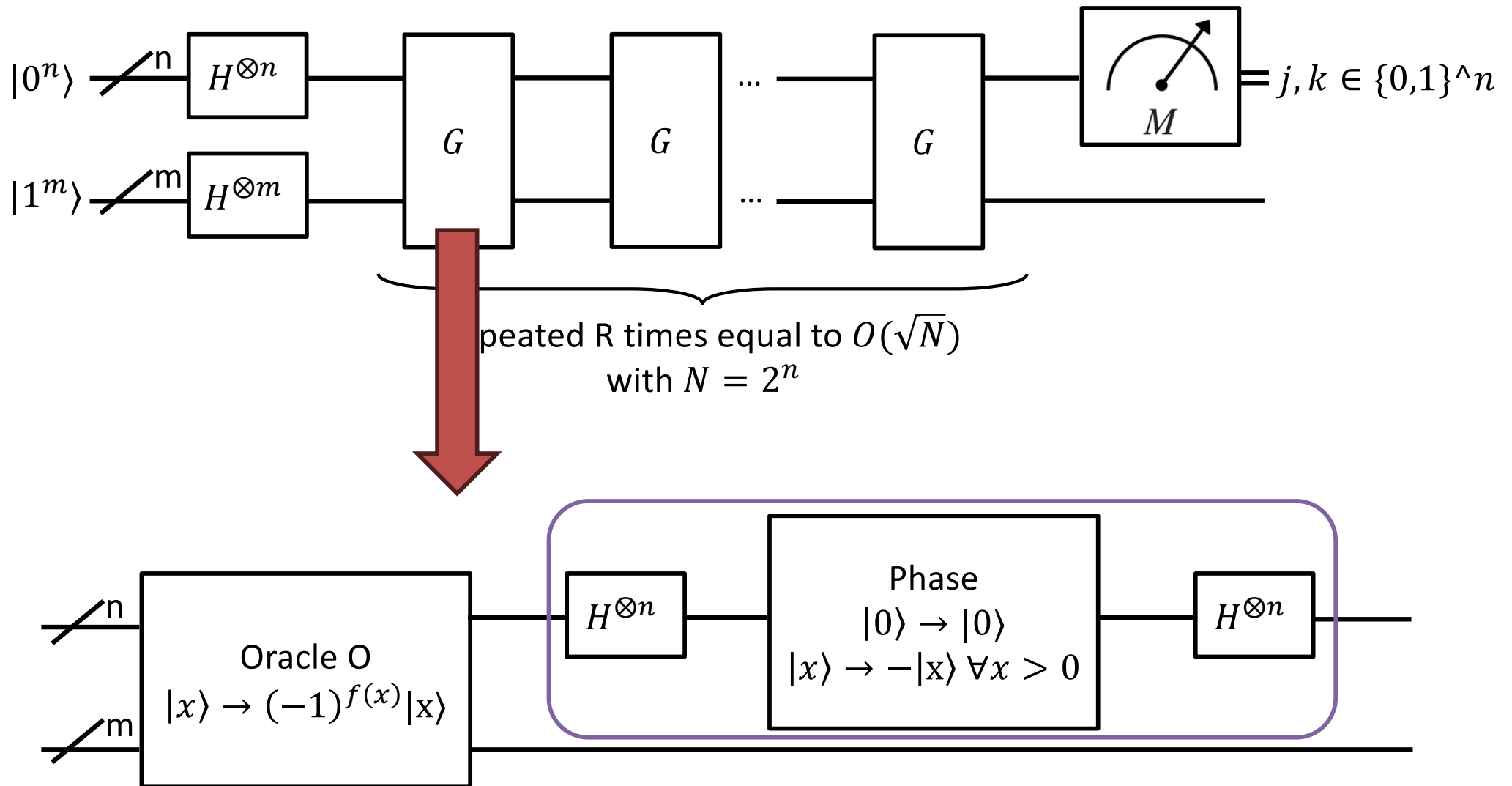


::... Grover's Algorithm



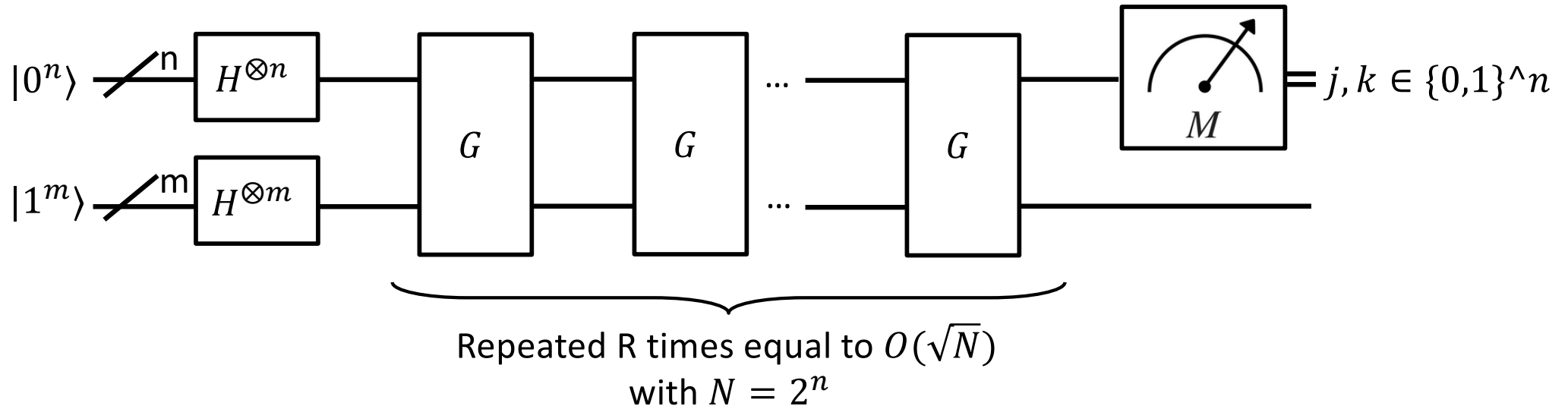
This gate prepares the state by putting all the data qubits in superposition. The target qubits (whose number $m < n$) are untouched.

... Grover's Algorithm



G is the Grover Operator, and it is made by the Oracle and the Inversion about the mean $U_s = 2H^{\otimes n}|0^n\rangle\langle 0^n|H^{\otimes n} - I$.

::... Grover's Algorithm

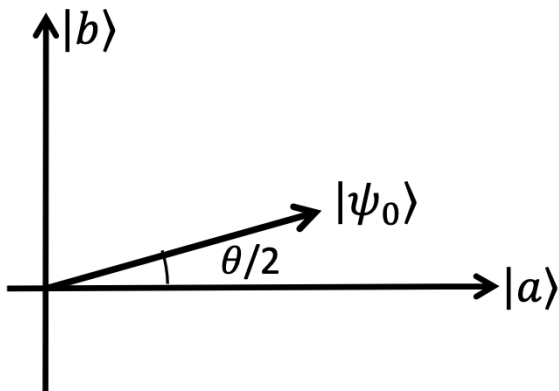


The Grover Operator is repeated $O(\sqrt{N})$ times before performing the measurement because we have to increase the amplitude of the state representing the solution before measuring it.

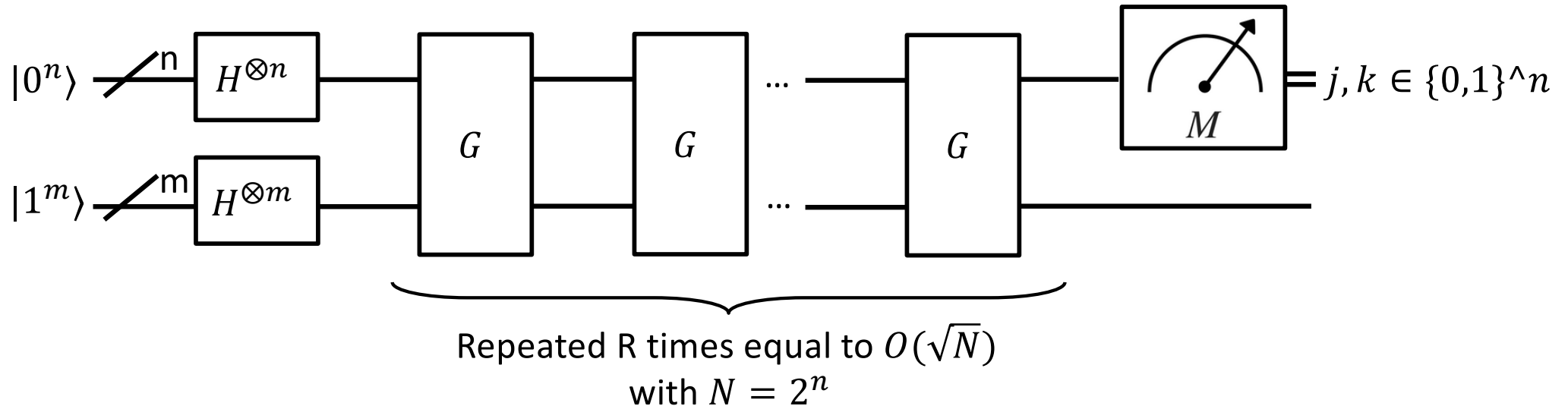
Let us consider two specific states:

$$|\alpha\rangle = \frac{1}{\sqrt{N-M}} \sum_{x, f(x)=0} |x\rangle$$

where N is the total number of inputs and M the number of solutions to the problem.



::... Grover's Algorithm

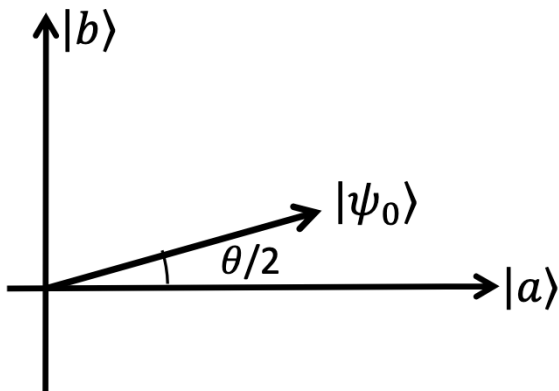


The Grover Operator is repeated $O(\sqrt{N})$ times before performing the measurement because we have to increase the amplitude of the state representing the solution before measuring it.

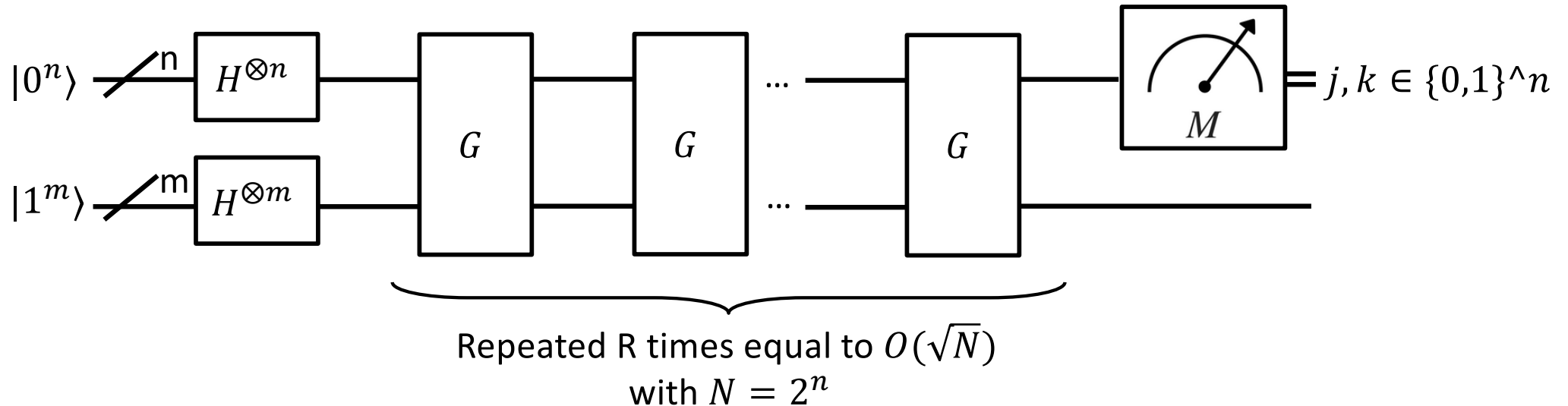
$|\alpha\rangle$ and $|\beta\rangle$ generate a 2-dimensional subspace S and $|\psi_0\rangle$ belongs to it:

$$|\psi_0\rangle = \sqrt{\frac{N-M}{N}} |\alpha\rangle + \sqrt{\frac{M}{N}} |\beta\rangle$$

$$|\psi_0\rangle = \cos\frac{\theta}{2} |\alpha\rangle + \sin\frac{\theta}{2} |\beta\rangle$$

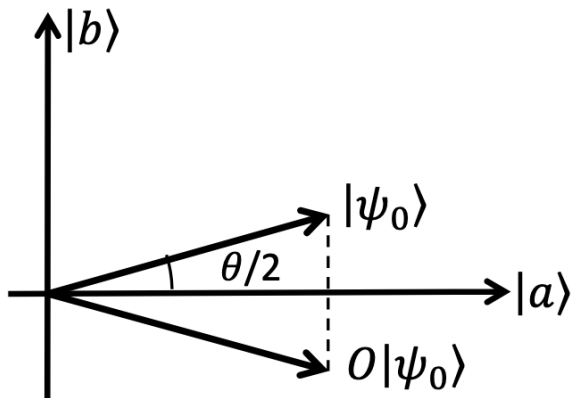


... Grover's Algorithm

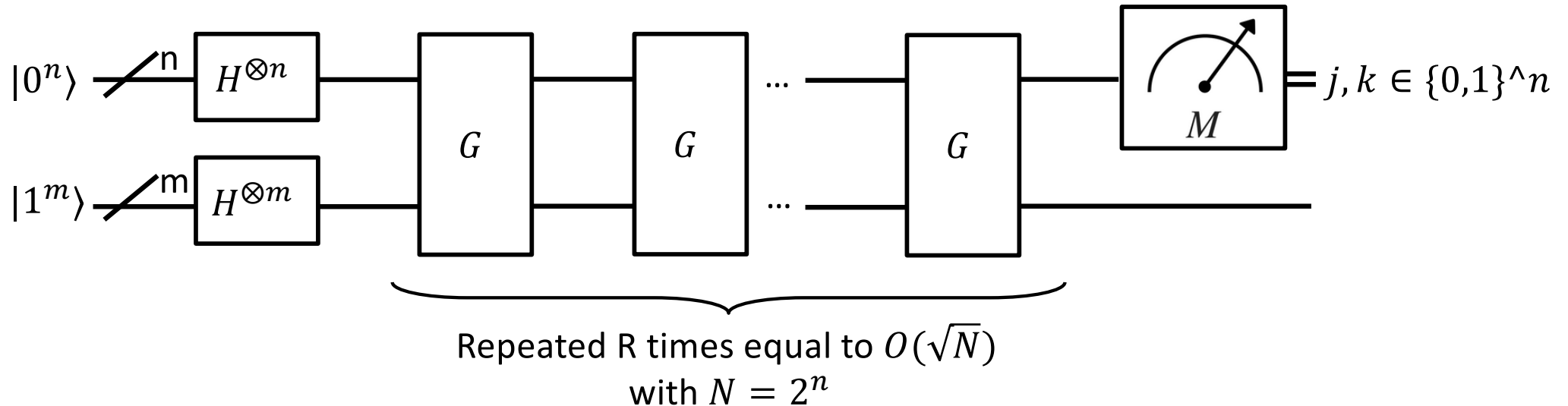


When we apply the oracle O to $|\psi_0\rangle$ the sign of the $|\beta\rangle$ component changes:

$$O|\psi_0\rangle = \cos\frac{\theta}{2}|\alpha\rangle - \sin\frac{\theta}{2}|\beta\rangle$$

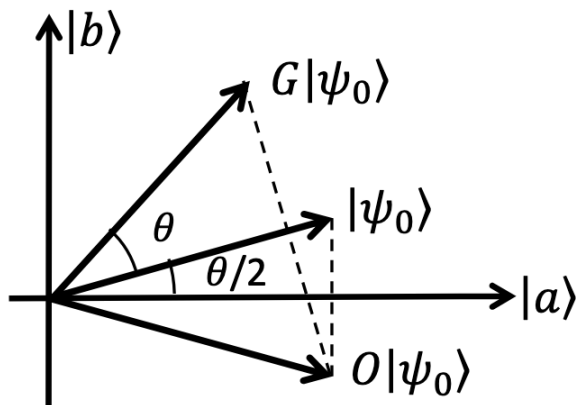


... Grover's Algorithm



When we apply the Oracle O to $|\psi_0\rangle$, the sign of the $|\beta\rangle$ component changes by being flipped:

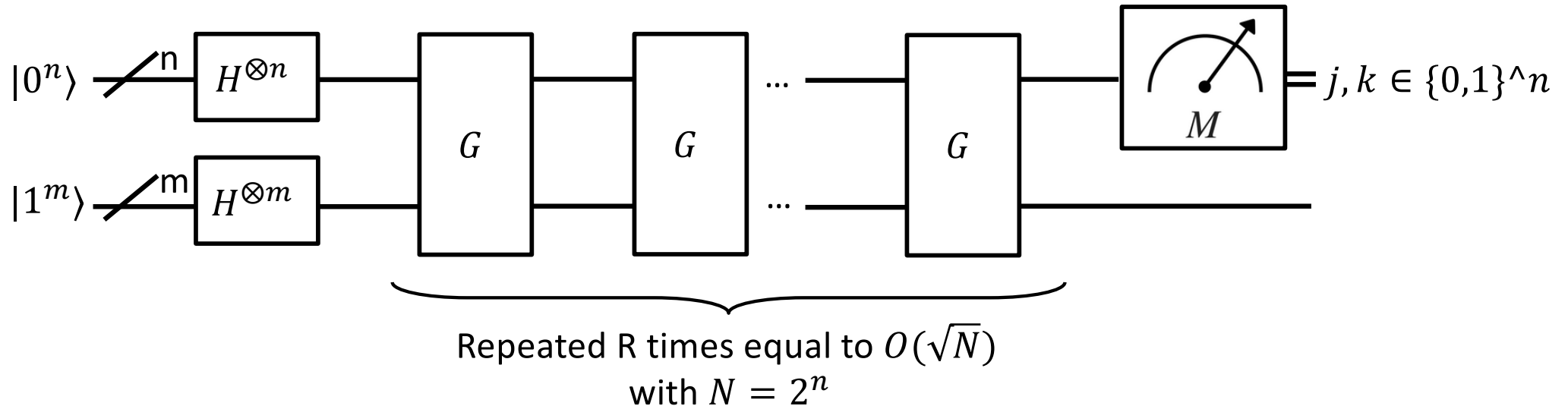
$$|\psi_1\rangle = O|\psi_0\rangle = \cos\frac{\theta}{2}|\alpha\rangle - \sin\frac{\theta}{2}|\beta\rangle$$



By applying the inversion about the mean U_s , we add an angle θ and change the sign back:

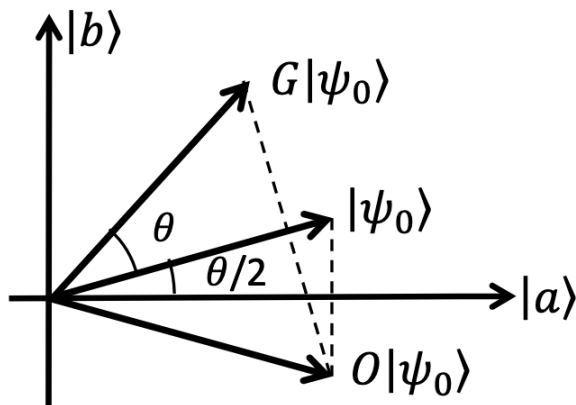
$$\begin{aligned} G|\psi_0\rangle &= U_s|\psi_1\rangle \\ &= \cos\left(\frac{\theta}{2} + \theta\right)|\alpha\rangle + \sin\left(\frac{\theta}{2} + \theta\right)|\beta\rangle \end{aligned}$$

... Grover's Algorithm

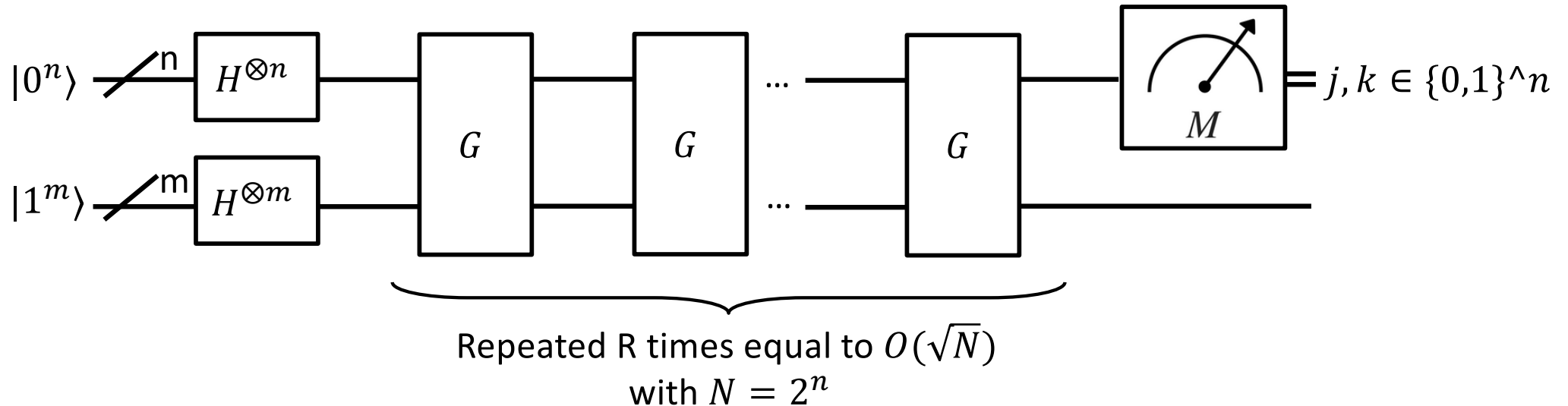


The pair of the Oracle and Inversion about the mean is repeatedly applied k time so that $G^k |\psi_0\rangle$ is as closer to $|b\rangle$ as possible:

$$G^k |\psi_0\rangle = \cos\theta_k |\alpha\rangle + \sin\theta_k |\beta\rangle, \theta_k = \left(k + \frac{1}{2}\right) \theta$$

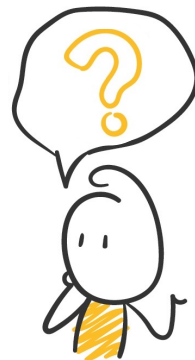
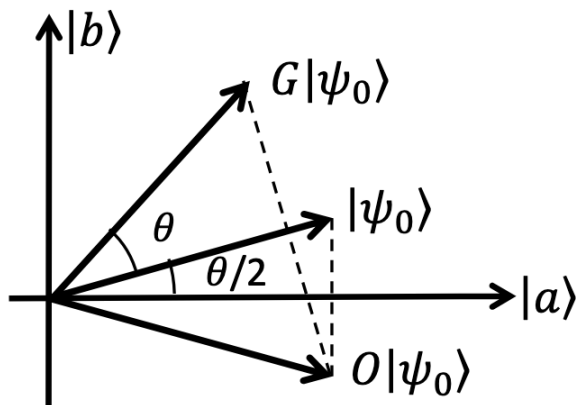


::... Grover's Algorithm



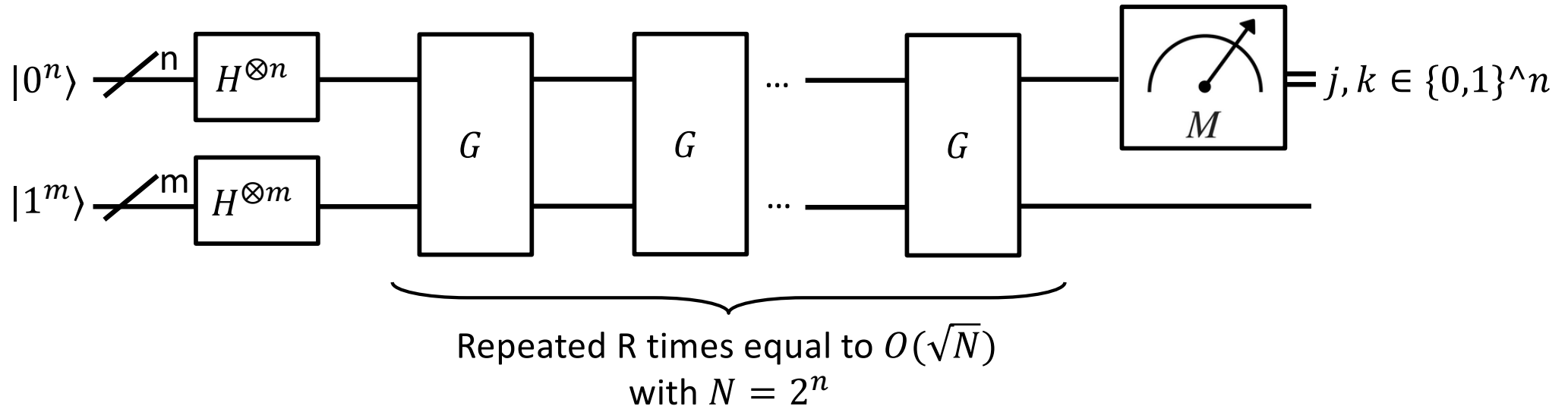
The pair of the Oracle and Inversion about the mean is repeatedly applied k time so that $G^k |\psi_0\rangle$ is as closer to $|b\rangle$ as possible:

$$G^k |\psi_0\rangle = \cos\theta_k |\alpha\rangle + \sin\theta_k |\beta\rangle, \theta_k = \left(k + \frac{1}{2}\right) \theta$$



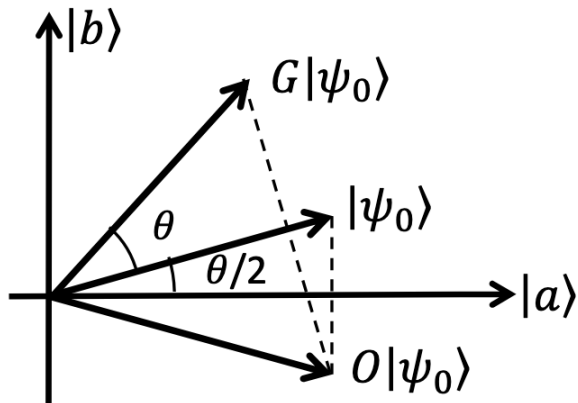
What is the optimal number of iterations k ?

... Grover's Algorithm



The pair of the Oracle and Inversion about the mean is repeatedly applied k times so that $G^k |\psi_0\rangle$ is as close to $|b\rangle$ as possible:

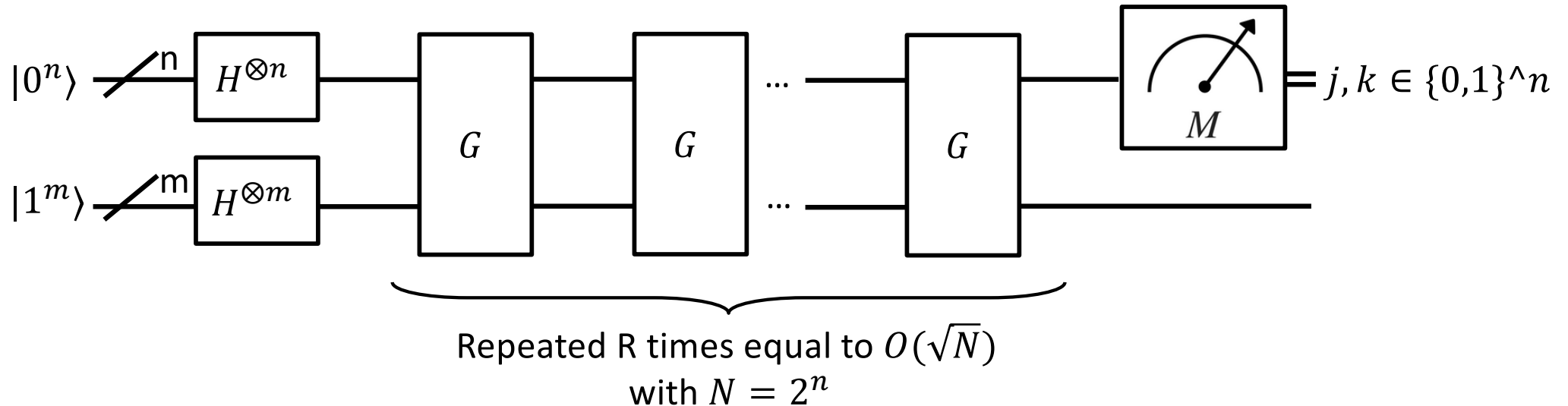
$$G^k |\psi_0\rangle = \cos\theta_k |\alpha\rangle + \sin\theta_k |\beta\rangle, \theta_k = \left(k + \frac{1}{2}\right) \theta$$



$$\sin\theta = \frac{2\sqrt{M(N-M)}}{N} \text{ for cases where } N \gg M \text{ this}$$

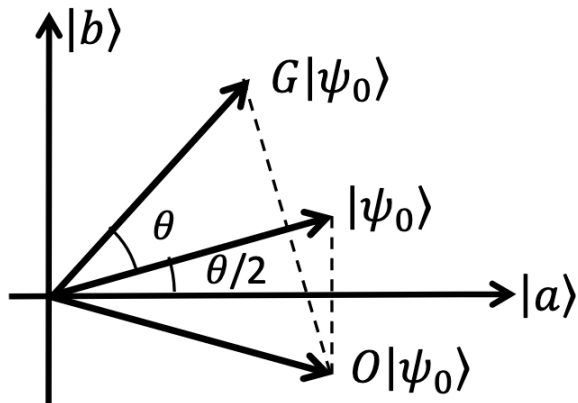
simplifies as $\theta \approx 2\sqrt{\frac{M}{N}}$

... Grover's Algorithm



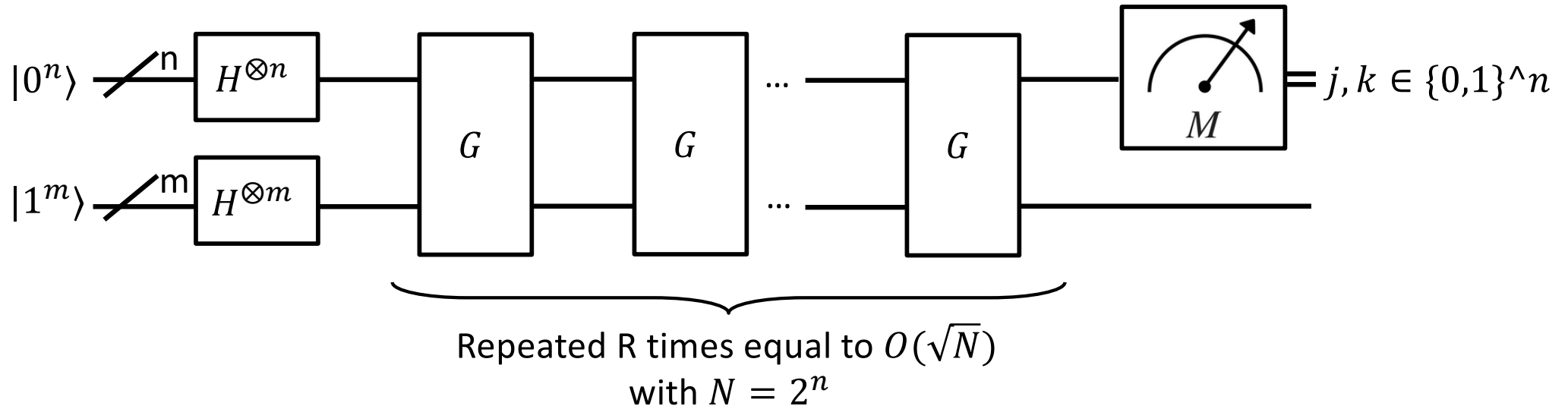
The pair of the Oracle and Inversion about the mean is repeatedly applied k times so that $G^k |\psi_0\rangle$ is as close to $|b\rangle$ as possible:

$$G^k |\psi_0\rangle = \cos\theta_k |\alpha\rangle + \sin\theta_k |\beta\rangle, \theta_k = \left(k + \frac{1}{2}\right) \theta$$

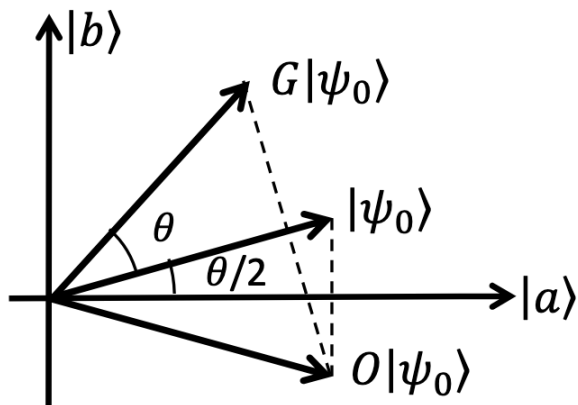


$$\theta_k \cong \frac{\pi}{2} \rightarrow k \cong \frac{1}{2} \left(\frac{\pi}{\theta} - 1 \right) \approx \frac{\pi}{4} \sqrt{\frac{N}{M}}$$

... Grover's Algorithm



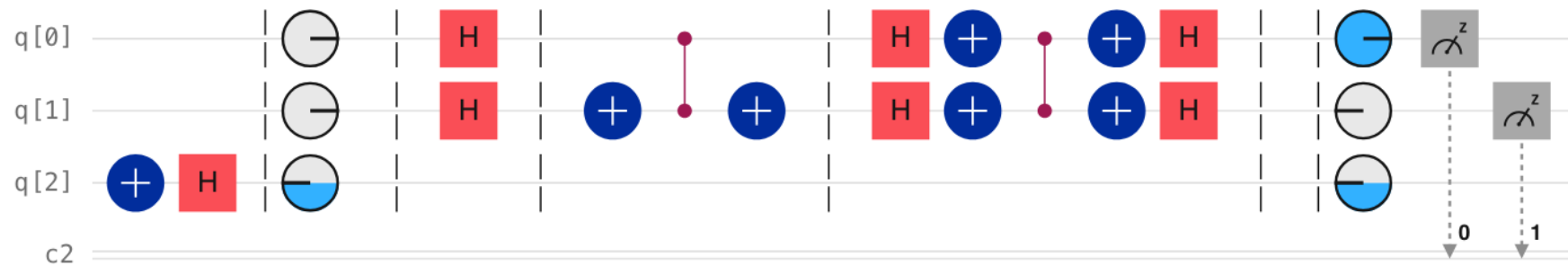
The pair of the Oracle and Inversion about the mean is repeatedly applied k time so that $G^k |\psi_0\rangle$ is as closer to $|b\rangle$ as possible:



Search Space (N)	Solutions (M)	Approx. Iterations (k)
100	1	8
1,000	1	25
1,000,000	1	785

::... Grover's Algorithm - Example

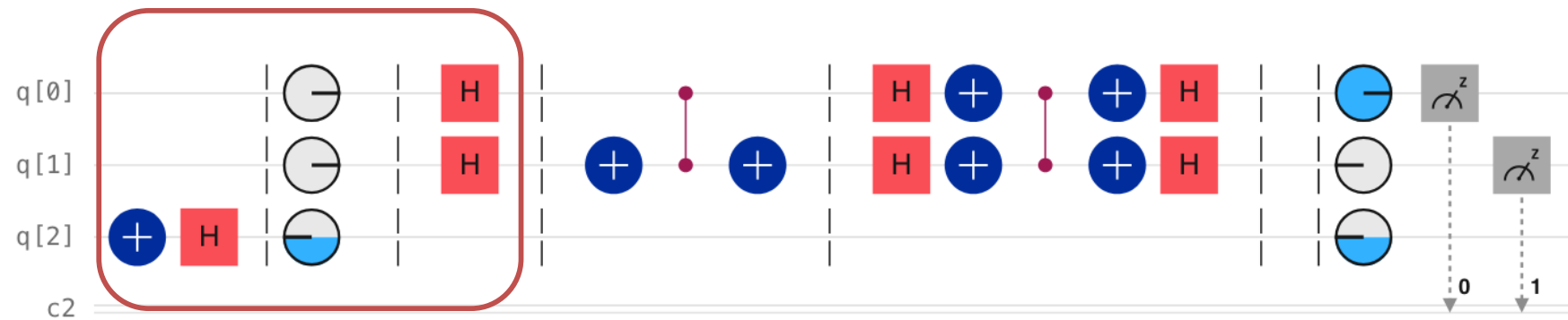
Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



We have modeled the oracle's circuit to the solution $|10\rangle$.

::... Grover's Algorithm - Example

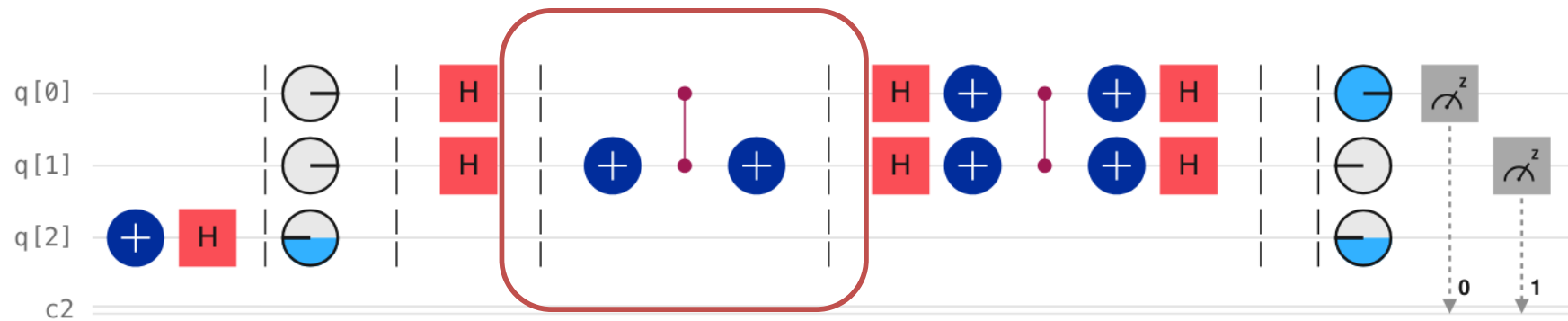
Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



Gates for the initial state

::... Grover's Algorithm - Example

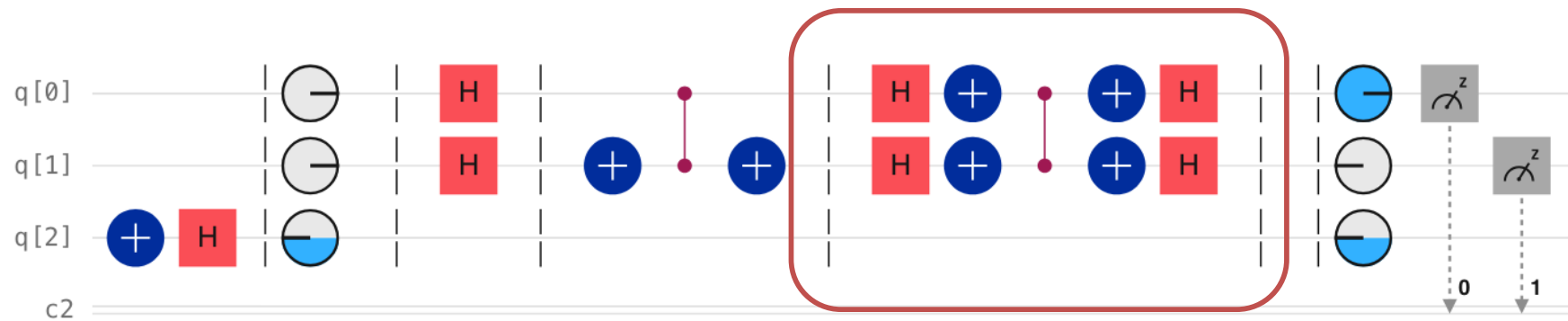
Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



Oracle for which the solution is $|10\rangle$

::... Grover's Algorithm - Example

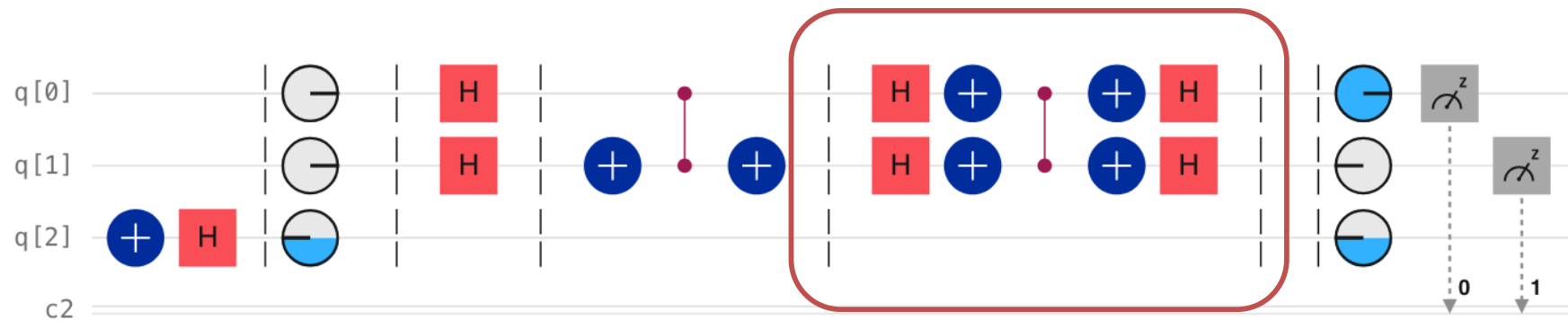
Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



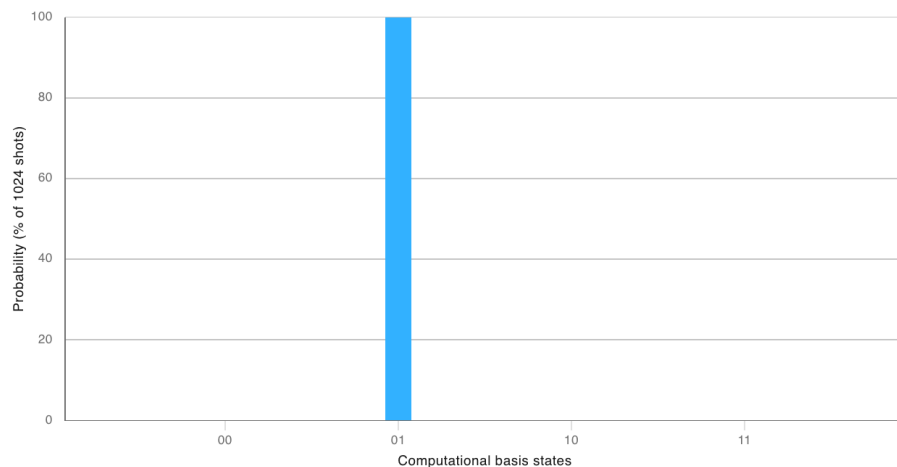
Reflection Gate for Inversion about the mean

::... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



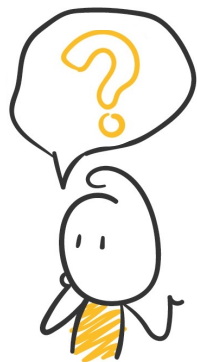
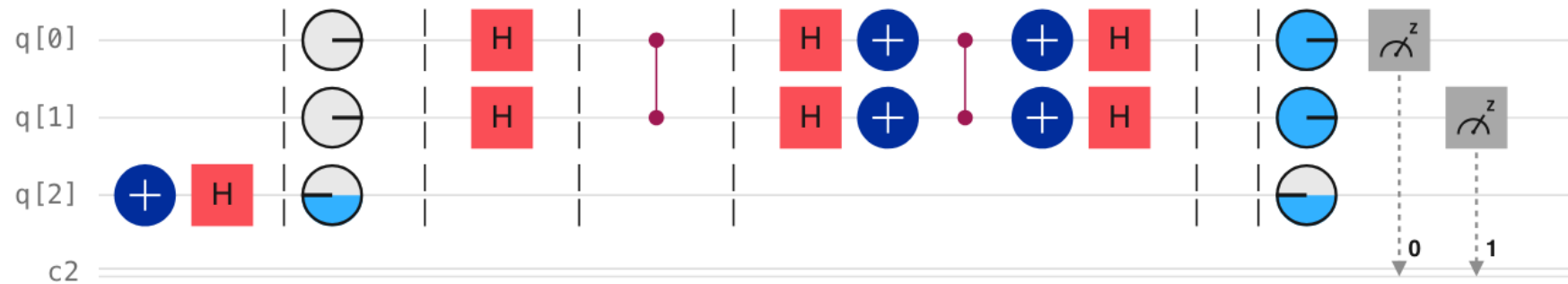
Reflection Gate for Inversion about the mean



The circuit found the solution we assumed to have.

::... Grover's Algorithm - Example

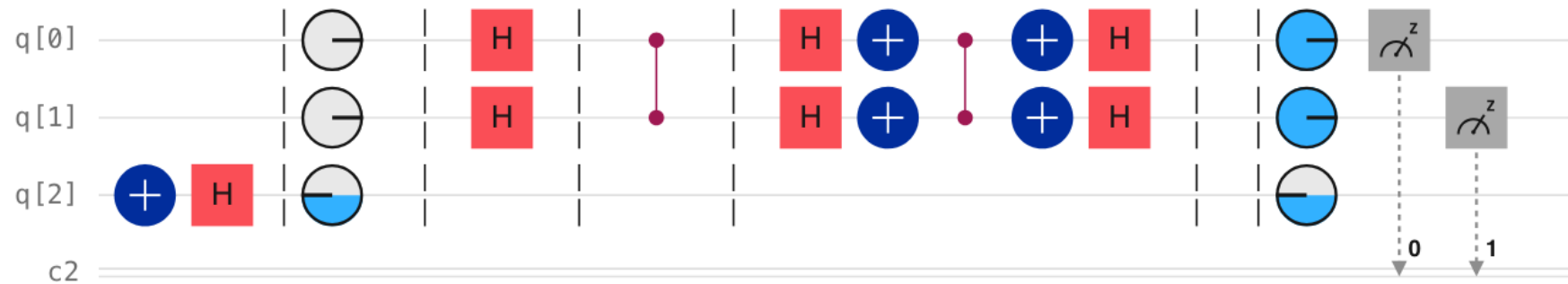
Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



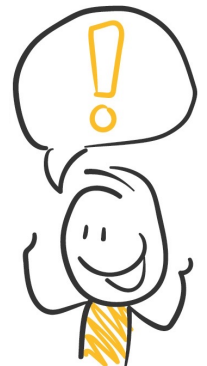
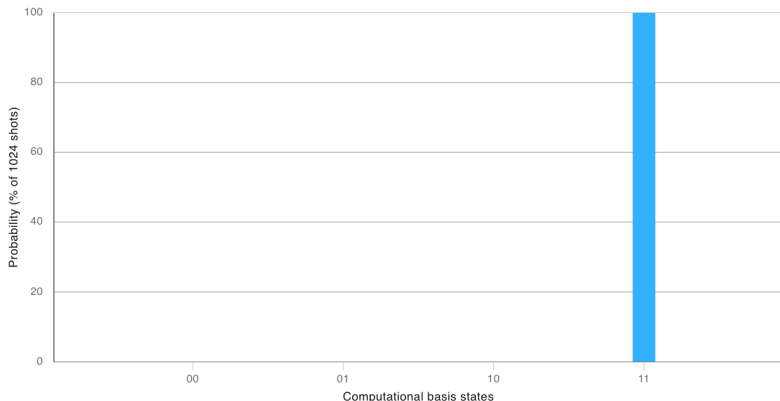
Now the solution is $|11\rangle$, will the circuit find the solution?

::... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.



Now the solution is $|11\rangle$, will the circuit find the solution? Yes, it was returned by the measurements.



::... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.

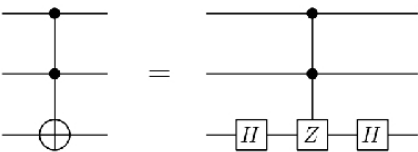
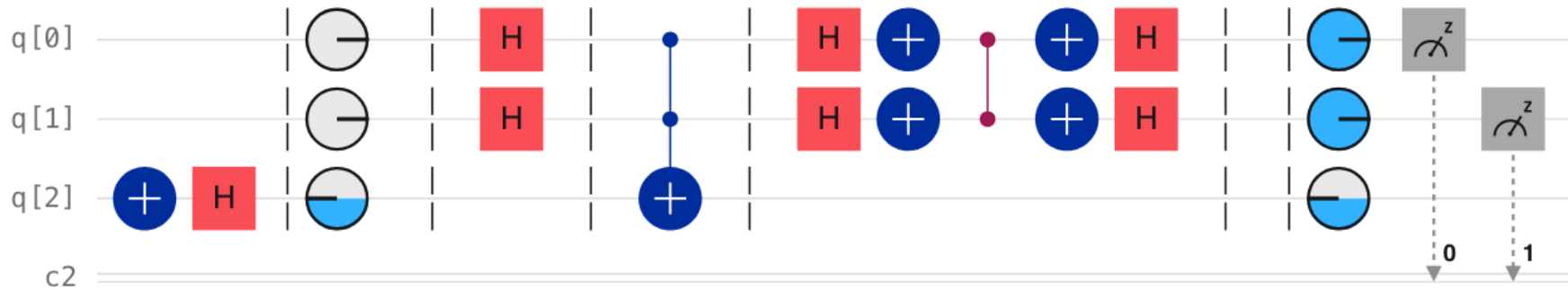
AB	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
00	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
01	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
10	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
11	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Let us the consider the case we do not know the solution and we have to model the binary function.

... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.

AB	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
00	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
01	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
10	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
11	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1



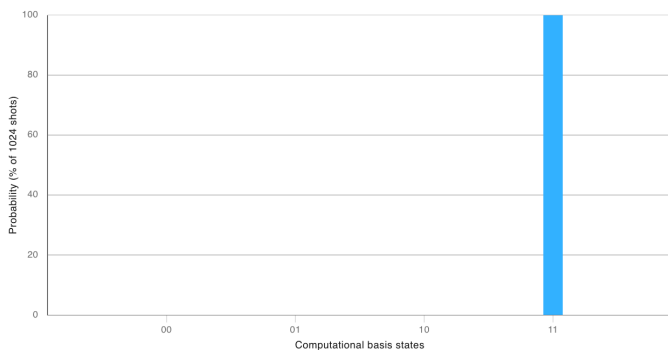
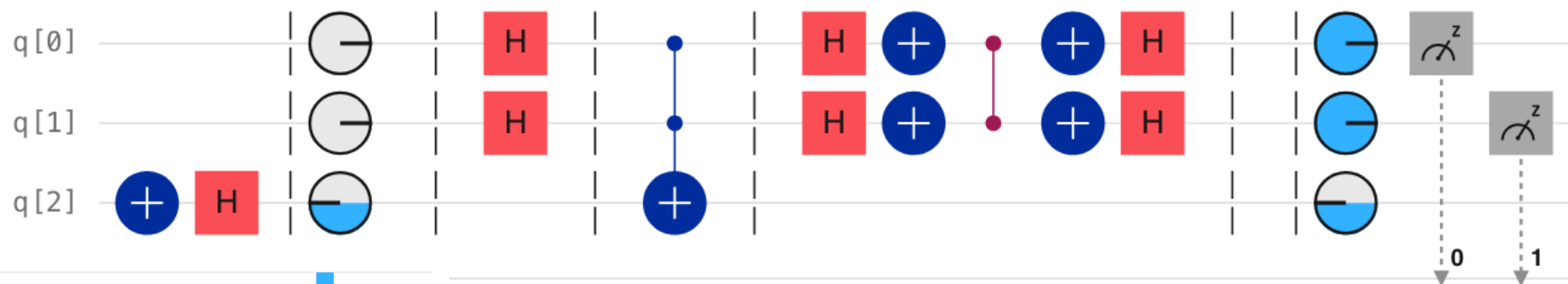
Toffoli gate through alteration of basis

The function is modelled by a Toffoli Gate on the ancilla qubit (as target one), which is filled when both data qubits are $|1\rangle$.

::... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.

AB	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
00	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
01	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
10	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
11	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

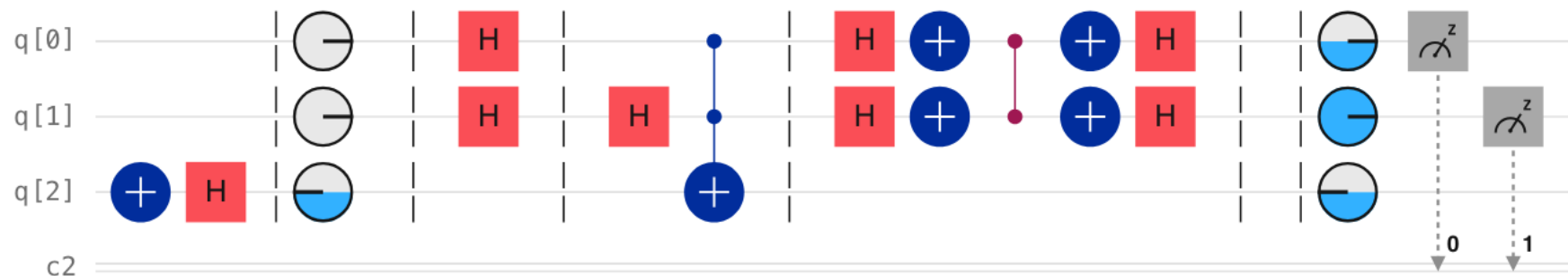


The circuit computes the solution equal to $|11\rangle$.

::... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.

AB	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
00	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
01	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
10	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
11	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

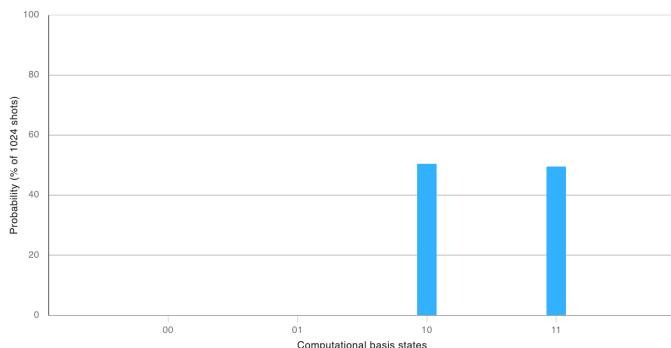
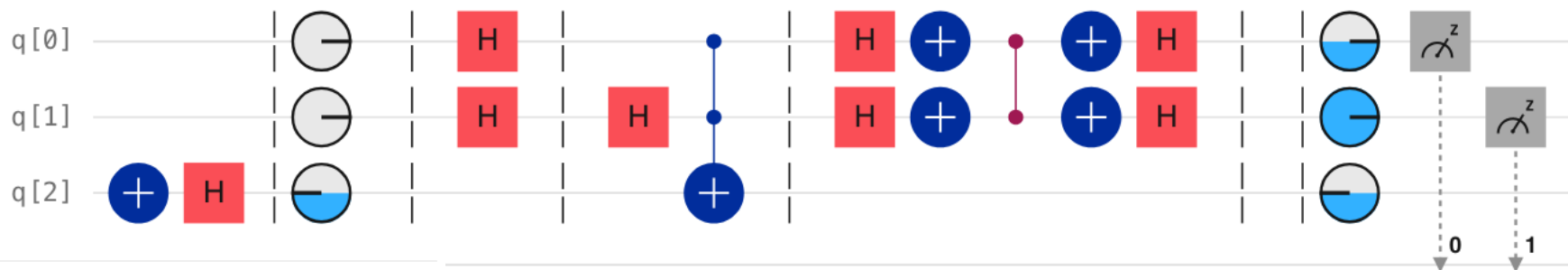


Now the oracle models another function.

::... Grover's Algorithm - Example

Let us experiment with the realization of the circuit from the Grover's Algorithm by using IBM Composer at <https://quantum.cloud.ibm.com/composer>.

AB	f_0	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}
00	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
01	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
10	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
11	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1



Now the oracle models another function, which has two solutions $|10\rangle$ and $|11\rangle$.

::... Grover's Algorithm - Applications

Grover's algorithm is a powerhouse for unstructured search and combinatorial optimization. Here is how that theoretical speedup translates into real-world applications.

1. Cryptanalysis (Cracking Codes) - The Problem: To crack a 128-bit AES key classically, you'd need to try 2^{128} combinations. The Grover Solution: Grover's algorithm can find the correct key in roughly 2^{64} operations. Impact: This effectively halves the security strength of symmetric encryption, and motivates post-quantum security standards to recommend doubling key sizes (e.g., moving from AES-128 to AES-256) to maintain the same level of protection.
2. Optimization in Logistics and Planning - Pattern Matching: Finding specific sequences in massive datasets, such as genomic strings in bioinformatics. Constraint Satisfaction: Identifying a valid schedule for thousands of flights where no two pilots are overworked and all planes are maintained. Grover's algorithm can "amplify" the probability of measuring the correct schedule.

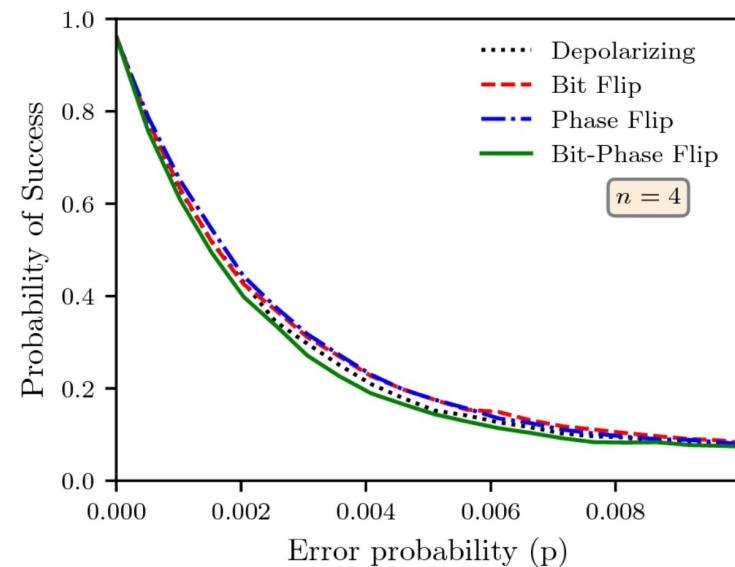
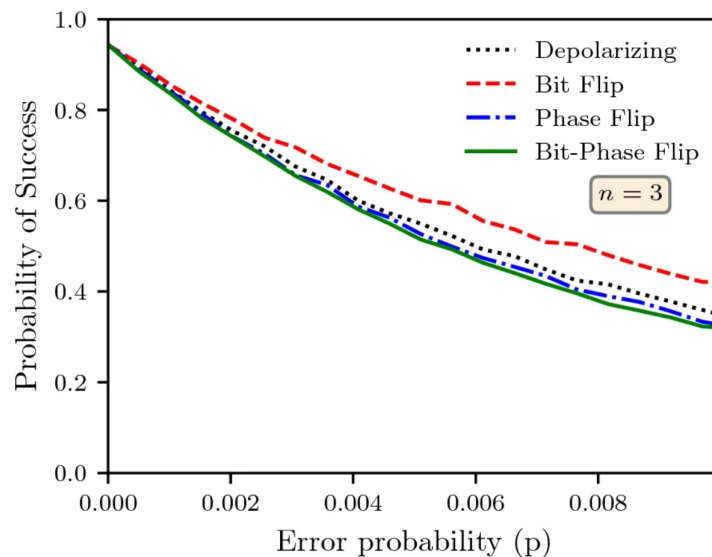
::... Grover's Algorithm - Applications

3. Machine Learning (Feature Selection) - In AI, we often deal with "black-box" functions. Grover's can be used to speed up:
 - Hyperparameter Tuning: Finding the best set of parameters for a neural network without checking every single combination.
 - Vector Quantization: Speeding up how algorithms cluster data points by finding the "closest" neighbor in a high-dimensional space significantly faster than classical k-means.
3. Solving NP-Complete Problems - While Grover's doesn't make NP-Complete problems "easy" (it doesn't turn exponential time into polynomial time), it provides a quadratic boost to Brute-Force Search.
 - Circuit Design: Checking if a complex logic circuit has any input that results in a specific output (the SAT problem).
 - Graph Theory: Finding specific paths or colorings in massive network architectures.

::... Grover's Algorithm - Applications

To be useful in a real application, Grover's algorithm requires:

- Logical Qubits: We need error-corrected qubits, as Grover's algorithm is sensitive to noise. In fact, it relies on precise amplitude amplification, which is easily disrupted by environmental interference, decoherence, and imperfect gate operations.

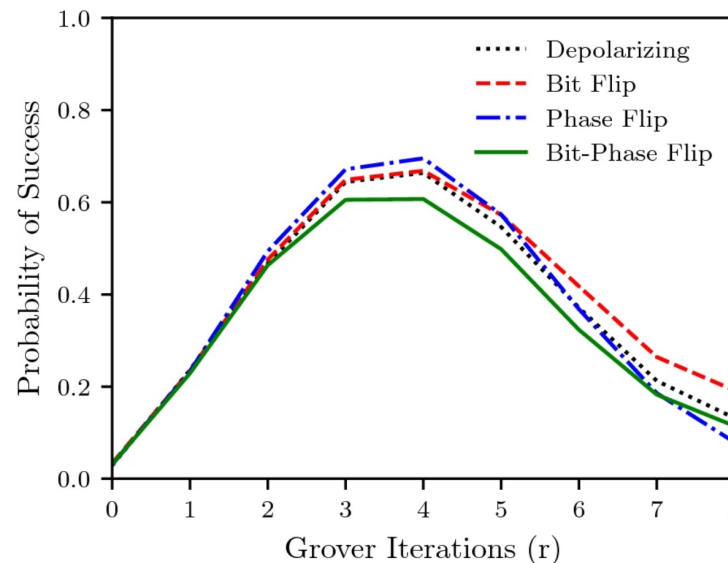


The more qubits involved in the algorithm, the higher the impact of the various noise factors on the success probability of Grover's.

::... Grover's Algorithm - Applications

To be useful in a real application, Grover's algorithm requires:

- Logical Qubits: We need error-corrected qubits, as Grover's algorithm is sensitive to noise. In fact, it relies on precise amplitude amplification, which is easily disrupted by environmental interference, decoherence, and imperfect gate operations.



Different noise channels with fixed probability of error $p = 0.0003$ and number of qubits $n = 5$

Since the Grover's algorithm merely rotates the marked state in a space spanned by the desired output state and the state orthogonal to it, the output is a periodic function of the number of iterations. The error due to the noise accumulates with the number of iterations.

::... Grover's Algorithm - Applications

To be useful in a real application, Grover's algorithm requires:

- Logical Qubits: We need error-corrected qubits, as Grover's algorithm is sensitive to noise. In fact, it relies on precise amplitude amplification, which is easily disrupted by environmental interference, decoherence, and imperfect gate operations.
- The Oracle: You have to build a quantum circuit (the "Oracle") that can recognize the solution. If the Oracle takes longer to run than a classical check, the quantum advantage evaporates.

Grover's algorithm is "optimal", as it has been mathematically proven that no quantum algorithm can search an unstructured list faster than $O(N)$.

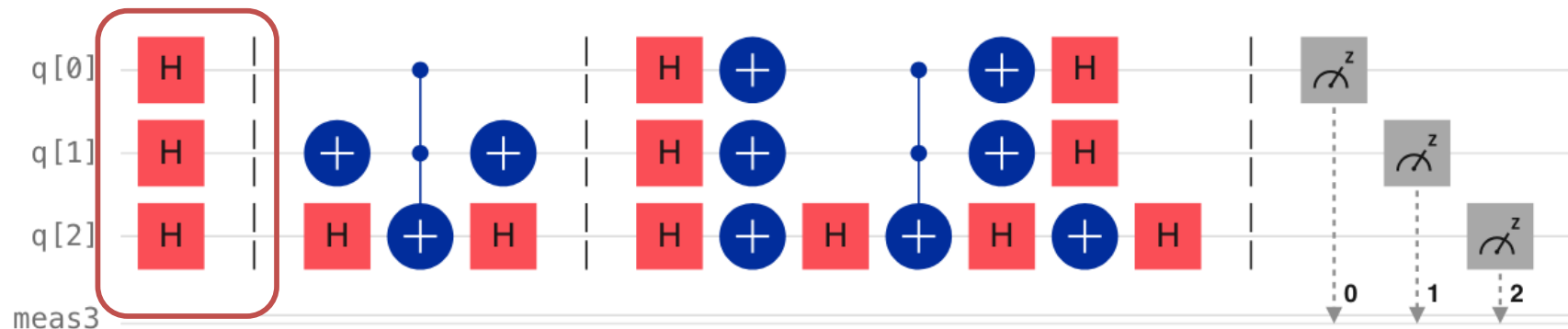
::... Grover's Algorithm - Key search

Let's use Grover's algorithm to deal with a 3-bit version of a key search. In this scenario, our "secret key" is the bitstring 101. In a real attack on AES, the Oracle would be thousands of gates long, but the logic remains the same: identify the target and flip its phase.

::... Grover's Algorithm - Key search

Let's use Grover's algorithm to deal with a 3-bit version of a key search. In this scenario, our "secret key" is the bitstring 101. In a real attack on AES, the Oracle would be thousands of gates long, but the logic remains the same: identify the target and flip its phase.

Initialize: Start in a superposition of all 8 possible keys ($2^3=8$).

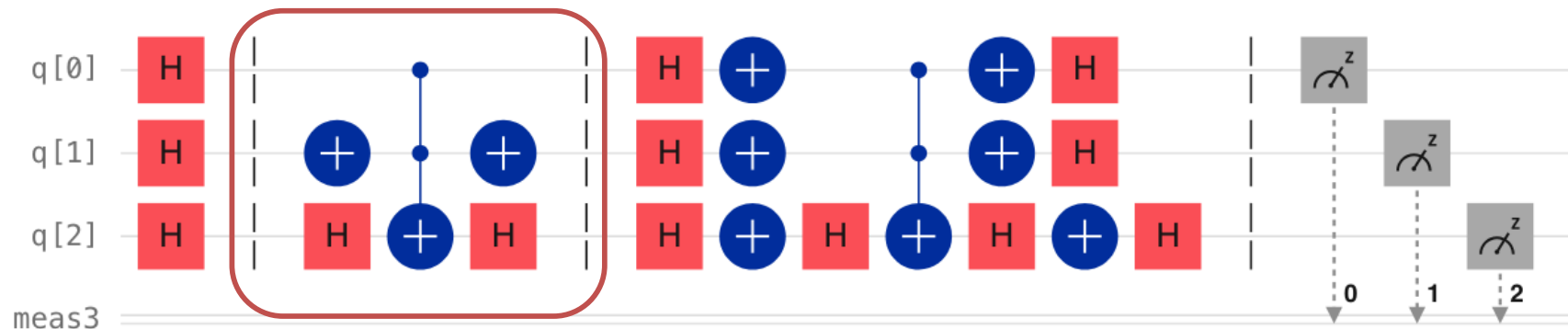


::... Grover's Algorithm - Key search

Let's use Grover's algorithm to deal with a 3-bit version of a key search. In this scenario, our "secret key" is the bitstring 101. In a real attack on AES, the Oracle would be thousands of gates long, but the logic remains the same: identify the target and flip its phase.

Initialize: Start in a superposition of all 8 possible keys ($2^3=8$).

Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.



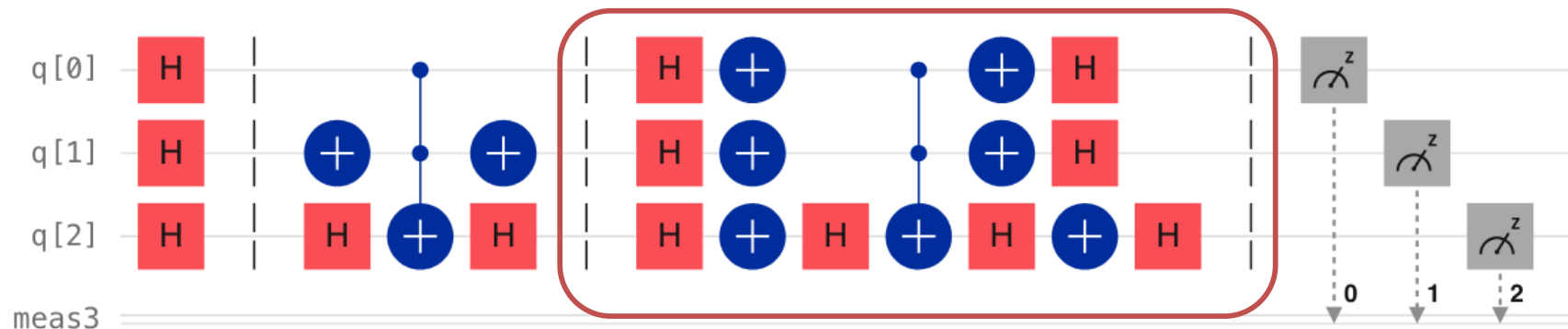
::... Grover's Algorithm - Key search

Let's use Grover's algorithm to deal with a 3-bit version of a key search. In this scenario, our "secret key" is the bitstring 101. In a real attack on AES, the Oracle would be thousands of gates long, but the logic remains the same: identify the target and flip its phase.

Initialize: Start in a superposition of all 8 possible keys ($2^3=8$).

Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.

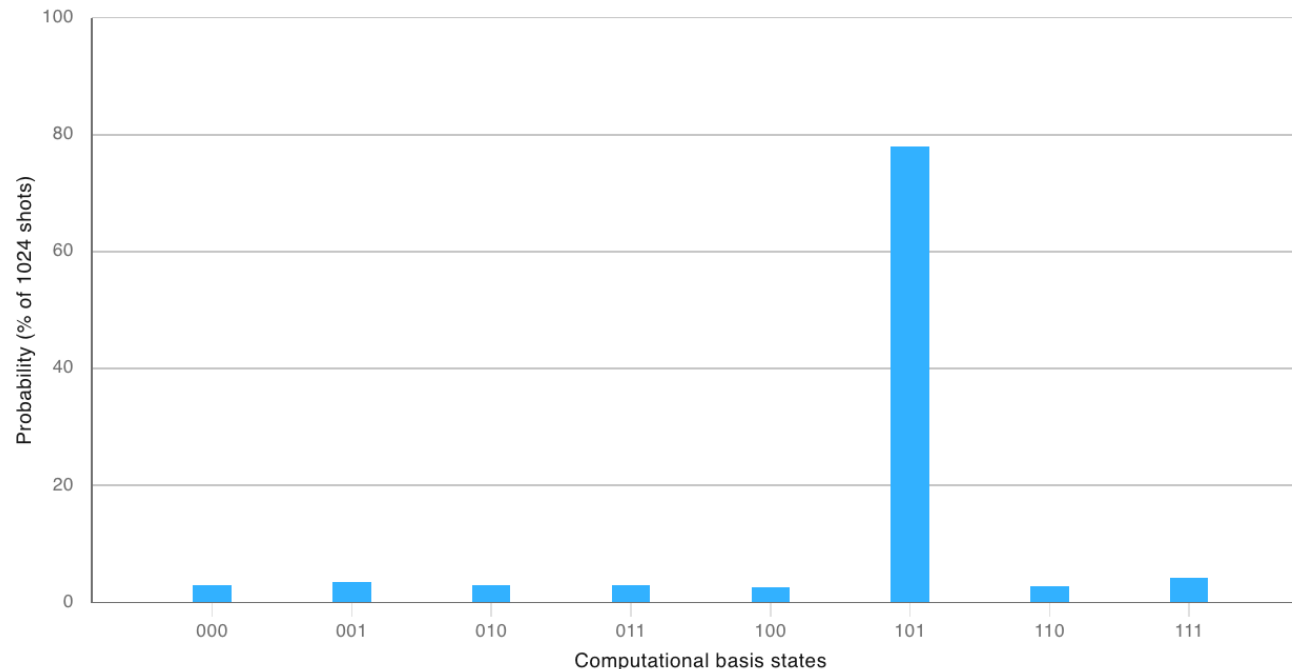
Diffusion: Amplify that flipped state.



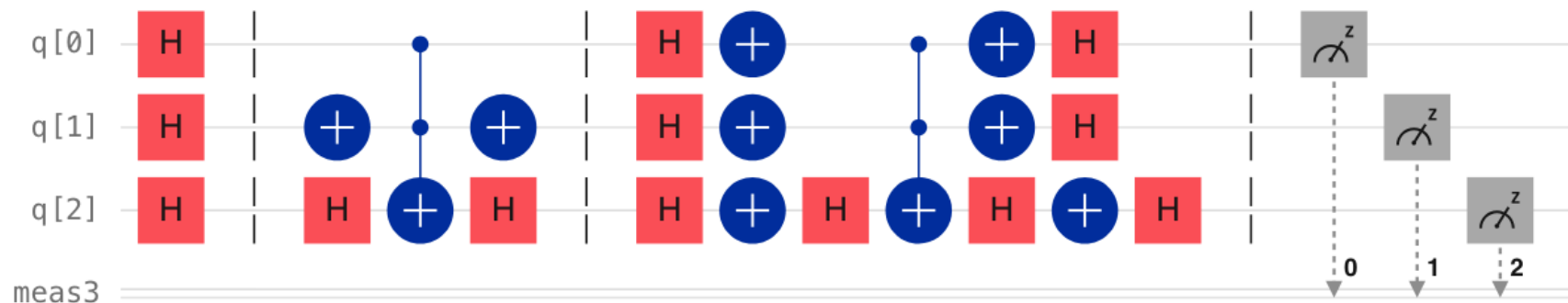
::... Grover's Algorithm - Key search

Let's use Grover's algorithm to deal with a 3-bit version of a key search. In this scenario, our "secret key" is the bitstring 101. In a real attack on AES, the Oracle remains the same: identical

Initialize
Oracle:
temporary
"see" at
Diffusion



).
X-gates to
Z) gate can



::... Grover's Algorithm - Key search

Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.

Let's move from a "hard-coded" oracle to a Functional Oracle, where encryption is executed and we have a Plaintext P and a Cypher Text C: this requires an additional register called the Work Register to store the result of the encryption before we compare it. This involves three distinct steps within the Oracle:

Encryption: Compute $E_k(P)$ and store it in a temporary register.

Comparison: Check if that result matches our known ciphertext C.

Phase Flip: If they match, flip the phase of the relative key.

Uncompute: (Crucial!) Reverse the encryption to "clean up" the work register so it doesn't interfere with the interference pattern.

::... Grover's Algorithm - Key search

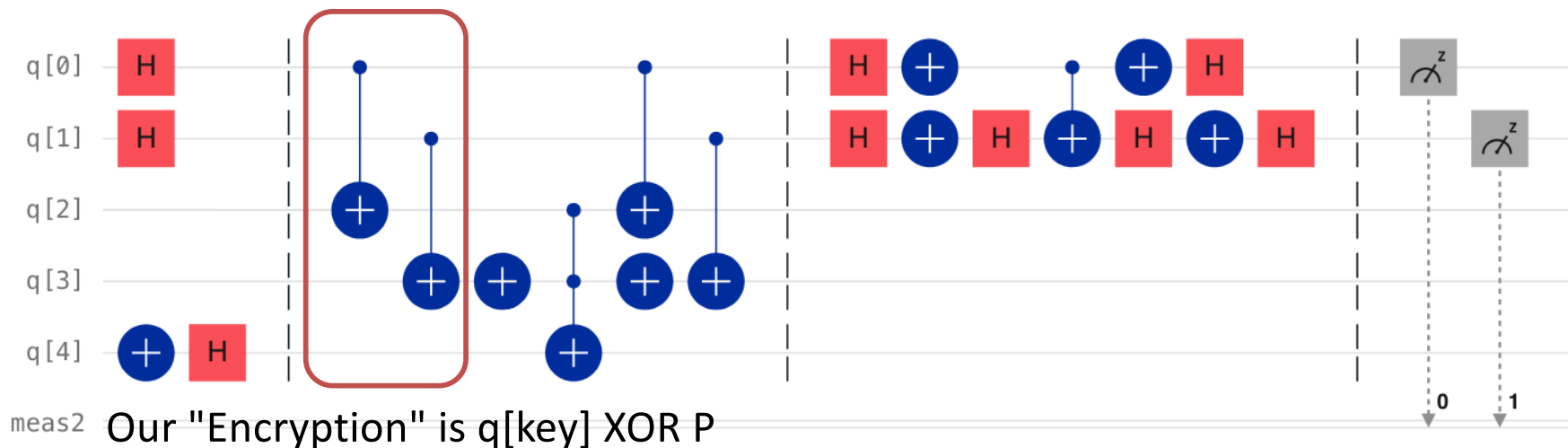
Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.

Encryption: Compute $E_k(P)$ and store it in a temporary register.

Comparison: Check if that result matches our known ciphertext C .

Phase Flip: If they match, flip the phase of the relative key.

Uncompute: (Crucial!) Reverse the encryption to "clean up" the work register so it doesn't interfere with the interference pattern.



::... Grover's Algorithm - Key search

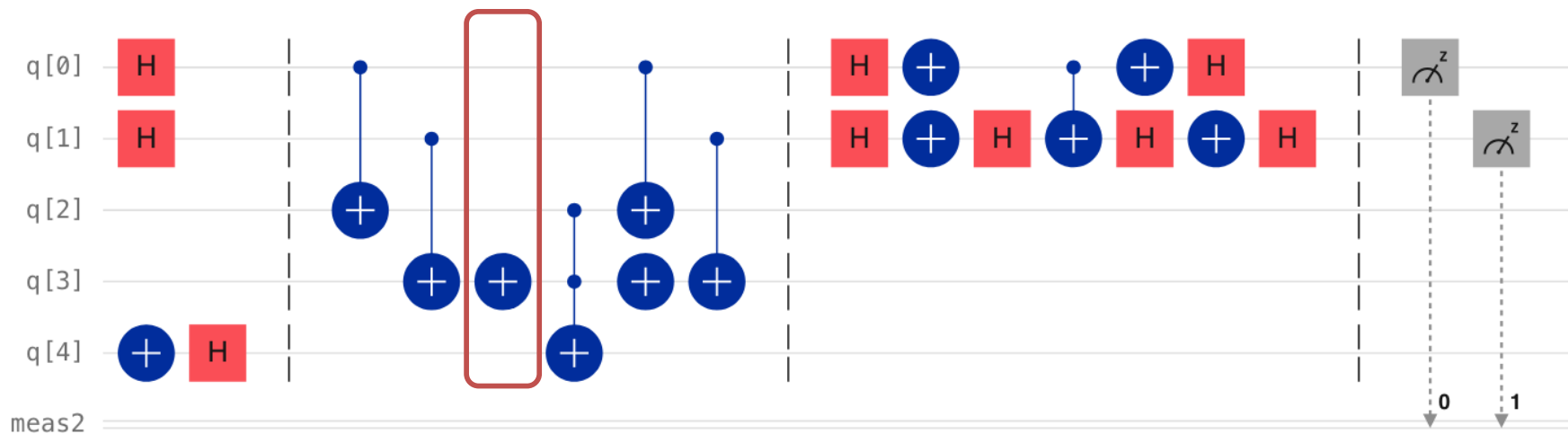
Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.

Encryption: Compute $E_k(P)$ and store it in a temporary register.

Comparison: Check if that result matches our known ciphertext C.

Phase Flip: If they match, flip the phase of the relative key.

Uncompute: (Crucial!) Reverse the encryption to "clean up" the work register so it doesn't interfere with the interference pattern.



::... Grover's Algorithm - Key search

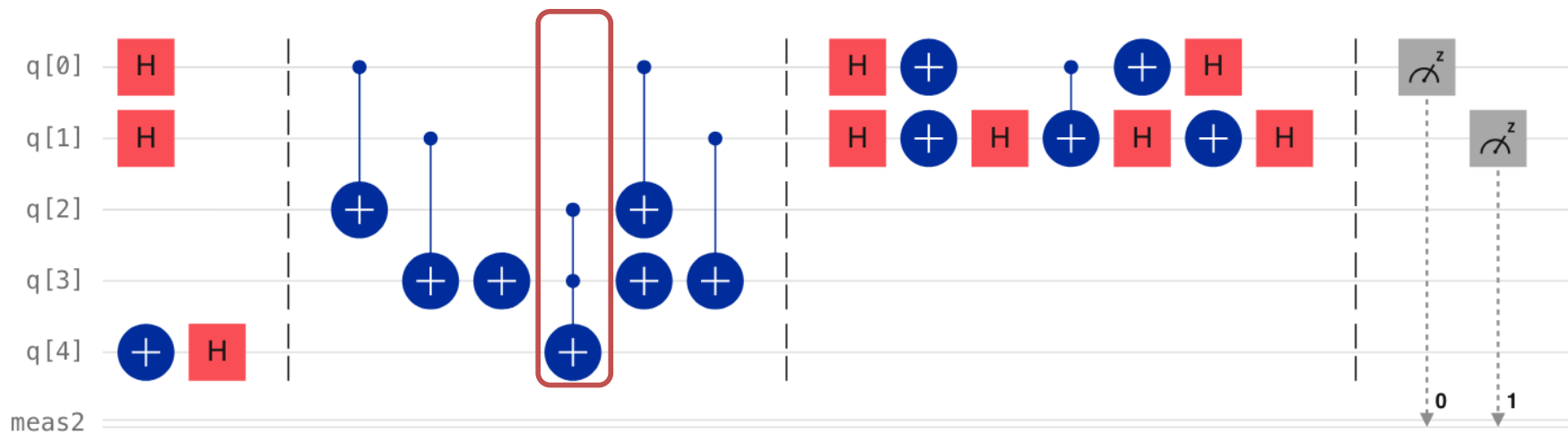
Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.

Encryption: Compute $E_k(P)$ and store it in a temporary register.

Comparison: Check if that result matches our known ciphertext C.

Phase Flip: If they match, flip the phase of the relative key.

Uncompute: (Crucial!) Reverse the encryption to "clean up" the work register so it doesn't interfere with the interference pattern.



::... Grover's Algorithm - Key search

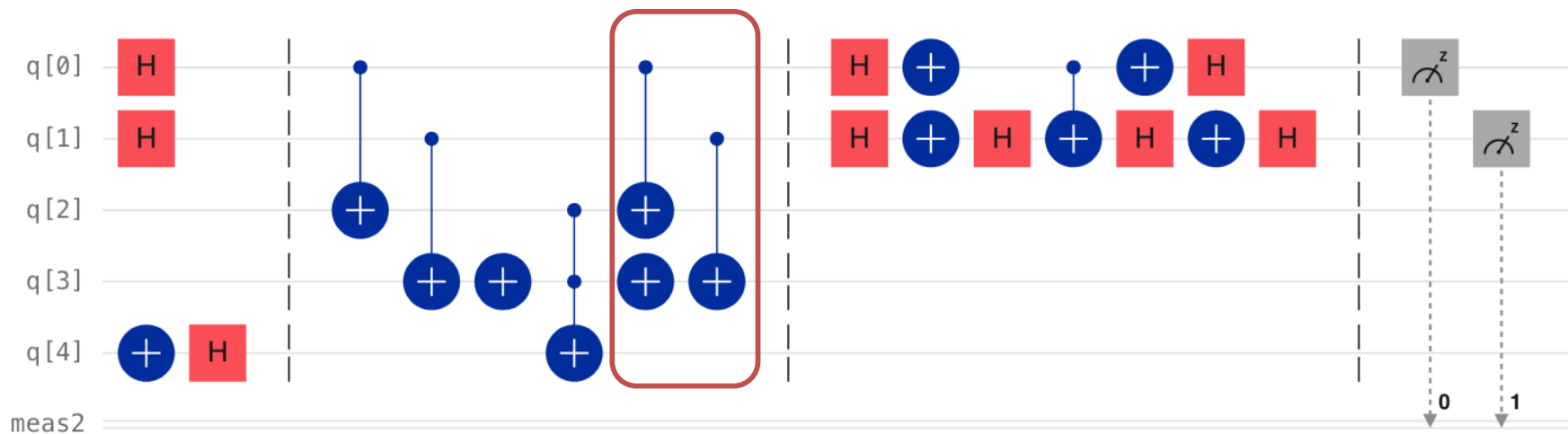
Oracle: A circuit that only triggers when it sees 101. It uses X-gates to temporarily flip 0 to 1 so that a Multi-Controlled Z (MCZ) gate can "see" an all-ones state and flip the phase.

Encryption: Compute $E_k(P)$ and store it in a temporary register.

Comparison: Check if that result matches our known ciphertext C.

Phase Flip: If they match, flip the phase of the relative key.

Uncompute: (Crucial!) Reverse the encryption to "clean up" the work register so it doesn't interfere with the interference pattern.



::... Grover's Algorithm - Key search

Uncomputation is the equivalent of "cleaning up your tracks" so that the Measurement Problem won't occur.

- The "Trash" Problem (Entanglement)

When you run an encryption function $E_k(P)$ inside the Oracle, you are using the Key Register to change the Work Register, and in quantum terms, these two registers become entangled.

Key Register: Holds the superposition of all possible keys.

Work Register: Holds the results of the encryption for every key.

If you leave the result in the Work Register and try to move to the Diffusion step, the information exists in the Work Register acts like a hidden measurement.

Grover's algorithm relies on Interference. For the Diffusion operator to work, all the "wrong" keys need to be able to cancel each other out or be manipulated as a single group.

::... Grover's Algorithm - Key search

If the Key Register is entangled with "trash" (the leftover ciphertext in the Work Register), the keys are no longer in a "pure" state. They are "tagged."

Without Uncomputing: The interference pattern breaks. The Diffusion operator fails to amplify the right answer. Your probability of measuring the correct key drops from $\sim 100\%$ back down to $1/N$ (random guessing).

With Uncomputing: You reverse the encryption gates. This disentangles the registers. The Work Register returns to $|00\dots 0\rangle$, but the Phase Flip remains on the Key Register.

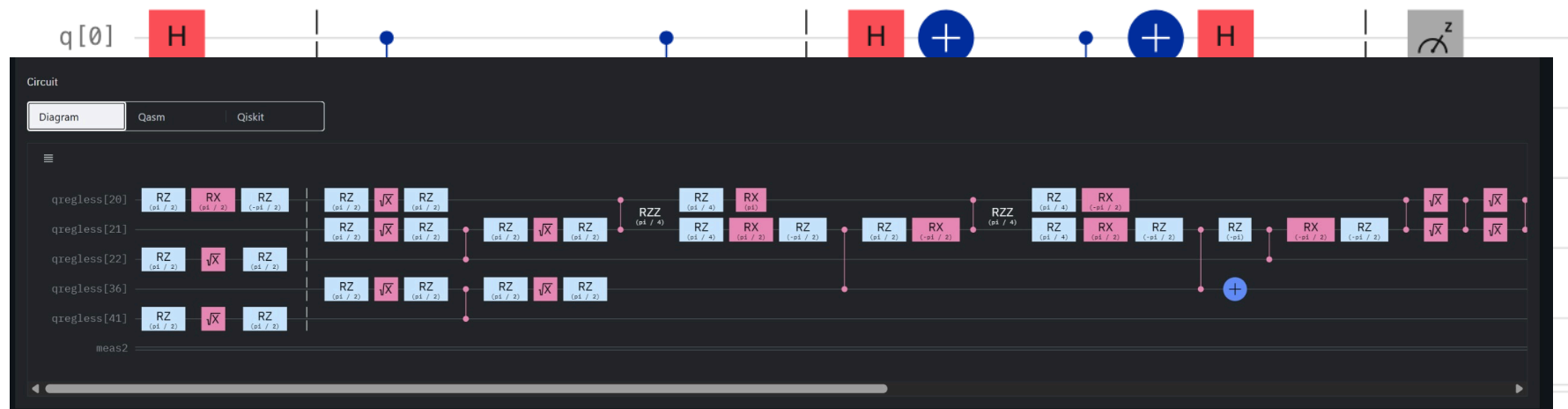
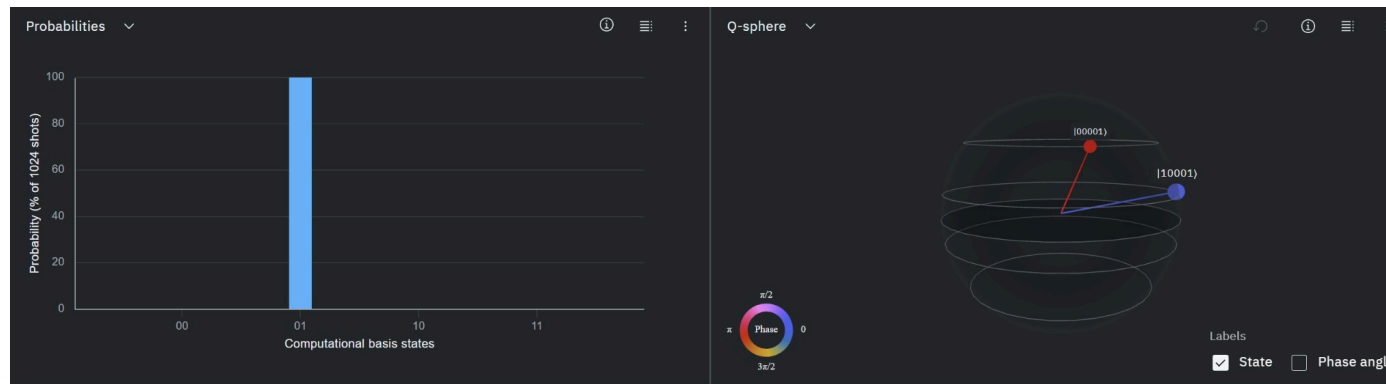
Encryption: $C = E(K, P)$
 Comparison: $C \neq E(K, P)$
 Phase Flip: If $C \neq E(K, P)$
Uncompute: $P = D(K, C)$
 register so it can be used

- 1. It uses X-gates to
- Z (MCZ) gate can
- ry register.
- a ciphertext C.
- ve key.
- lean up" the work
- attern.

[illegible]

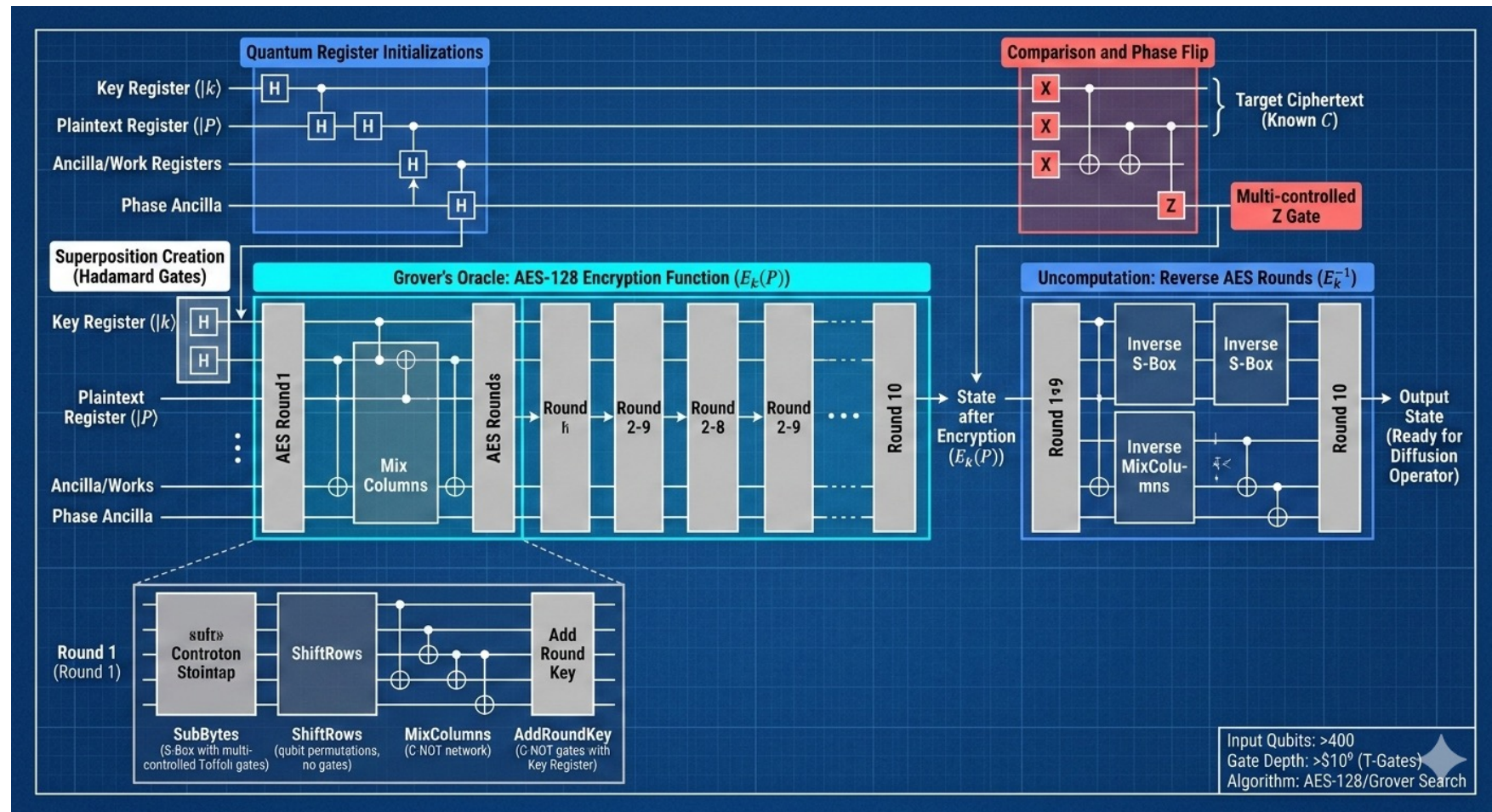
::... Grover's Algorithm - Key search

The plaintext is 11, and the cyphertext is 01 generated as key XOR plaintext, so the key is 01.



::... Grover's Algorithm - Key search

Imagine implementing AES in the Oracle to run key search:



::... QUBO Optimization

Quadratic Unconstrained Binary Optimization (QUBO) is a class of optimization problems in which a quadratic objective is optimized over binary variables, with no explicit constraints other than “each variable can assume 0 or 1 as a value”.

$$\min_{x \in \{0,1\}^n} x^T Q x,$$

where x is a binary vector and Q is a real-valued coefficient matrix. In the upper-triangular convention, the diagonal entries of Q are the linear coefficients and the off-diagonal entries are the pairwise interaction terms:

$$\min_{x_i \in \{0,1\}} \left(\sum_i Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j \right).$$

each variable contributes its own individual cost, and each pair of variables can also contribute an interaction cost. Because the variables are binary, $x_i^2 = x_i$, so diagonal quadratic terms collapse into linear terms, which is why the model is usually written with diagonal entries for linear effects and off-diagonal entries for pairwise couplings.

... QUBO Optimization

Quadratic Unconstrained Binary Optimization (QUBO) is a class of optimization problems in which a quadratic objective function is minimized over binary variables with no explicit constraints other than “each variable can be either 0 or 1”.

The objective has terms up to degree 2, like x_i and x_{ij} .

There are no explicit side constraints in the final model. Many real problems have constraints, but they are incorporated into the objective via penalty terms, so the final model remains an unconstrained binary minimization.

where x is a binary vector. In the standard upper-triangular convention, the diagonal entries of Q are the linear coefficients and the off-diagonal entries are the pairwise interaction terms:

$$\min_{x_i \in \{0,1\}} \left(\sum_i Q_{ii} x_i + \sum_{i < j} Q_{ij} x_i x_j \right).$$

each variable contributes its own individual cost, and each pair of variables can also contribute an interaction cost. Because the variables are binary, $x_i^2 = x_i$, so diagonal quadratic terms collapse into linear terms, which is why the model is usually written with diagonal entries for linear effects and off-diagonal entries for pairwise couplings.

... QUBO Optimization

Quadratic Unconstrained Binary Optimization (QUBO) is a class of optimization problems in which a quadratic objective is minimized over binary variables with no explicit constraints other than “each variable can

The objective has terms up to degree 2, like x_i and x_{ij} .

There are no explicit side constraints in the final model. Many real problems have constraints, but they are incorporated into the objective via **penalty terms**, so the final model remains an unconstrained binary minimization.

where x is a binary vector. In the standard upper-triangular convention, the diagonal entries of Q are the linear coefficients and the off-diagonal entries are the pairwise interaction terms.

Here is the basic idea. Suppose your original problem is

$$\min f(x) \text{ s.t. } g(x) = 0,$$

a standard QUBO reformulation is

$$\min f(x) + P[g(x)]^2,$$

with $P > 0$ large enough that violating the constraint becomes too expensive. The square is useful because it is zero when the constraint is satisfied and positive otherwise.

::... QUBO Optimization

If Q_{ij} is positive, then $x_i = x_j = 1$ increases the objective, so the model discourages that pair from being simultaneously active. If Q_{ij} is negative, the model rewards turning both on together. Diagonal terms Q_{ii} control whether activating variable x_i is individually attractive or costly. a QUBO is a very expressive “language” for representing preferences, conflicts, compatibilities, and penalties among binary decisions.

QUBO is especially convenient when the original problem already has a binary-combinatorial flavor. In those settings, the main modeling work is choosing the right penalties so that infeasible or undesirable configurations become energetically expensive.

- If the applied penalty is too small, an infeasible solution may incorrectly look better than the best feasible solution.
- If the applied penalty is too large, the energy landscape can become poorly scaled, which can make optimization harder in practice.

The penalty coefficients must preserve the intended semantics while remaining numerically manageable.

::... QUBO Optimization

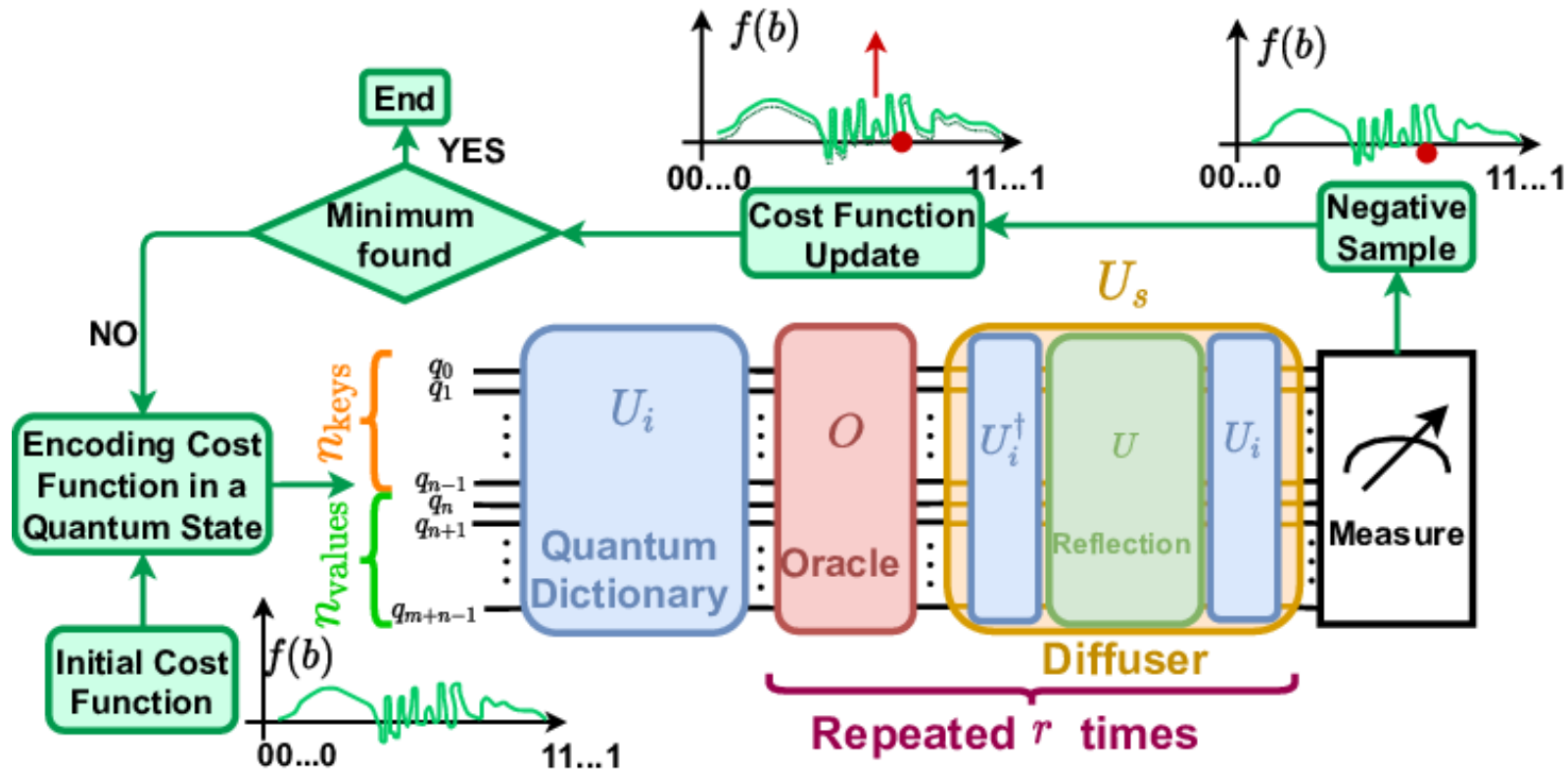
QUBO is an NP hard problem, in general, and its corresponding decision version “Does there exist $x_i \in \{0,1\}^n$ such that $x^\top Q x \leq K$?” is NP-complete. This is because many known NP-hard combinatorial problems can be reduced to QUBO, so a polynomial-time algorithm for arbitrary QUBO would give polynomial-time algorithms for all of them.

- In general, a solution to the problem x^* is not unique, but there may be a set of minimizing vectors with equal value w.r.t. the objective function. The complexity of QUBO arises from the number of candidate binary vectors to be evaluated, as the set of binary vectors of fixed length $n \in \mathbb{N}$ grows exponentially in n .

Some restricted subclasses are easier, if the structure of Q is very special, so the problem can become solvable in polynomial time (e.g., all coefficients are positive/negative, Q is diagonal or all off-diagonal elements of Q are non-positive).

QUBO can be solved using integer linear programming solvers.

:... Grover Adaptive Search (GAS)



Grover Adaptive Search (GAS) is a way to turn optimization into a sequence of Grover searches. Instead of asking “which bitstring is the solution?”, it asks “is there any bitstring whose objective value is better than my current threshold?” and then keeps tightening that threshold as better solutions are found.

::... Grover Adaptive Search (GAS)

GAS solves a minimization problem over a finite discrete set,

$$\min_{x \in \{0,1\}^n} x^\top Q x,$$

by replacing it with repeated yes/no questions of the form

$$\exists x \in C: f(x) < y?$$

where y is the current best-known objective value. If the answer is yes, GAS tries to find such an x , updates $y \leftarrow f(x)$, and repeats. This is the optimization analogue of the Dürr–Høyer minimum-finding idea, which showed that a minimum in an unsorted table can be found in $O(c\sqrt{N})$ time with high probability. At iteration t a threshold y_t and define a predicate

$$\chi_{y_t} = \begin{cases} 1 & f(x) < y_t \\ 0 & \text{otherwise} \end{cases},$$

Grover search is then used to amplify precisely the states with $\chi_{y_t}(x) = 1$. If one of those states is measured, it is defined, and strictly better solution, so the threshold is replaced with its value and GAS continue. The search space of improving states shrinks as the threshold improves, until ideally only globally optimal states remain below threshold.

::... Shor's Algorithm

Shor's algorithm is a quantum algorithm for integer factorization and its fame comes from the fact that it can factor large integers in polynomial time, while no polynomial-time classical factoring algorithm is known. Classically, factoring is believed to be hard for large N , and this is one of the reasons RSA cryptography works. Shor's algorithm showed that a quantum computer could factor N efficiently enough to break RSA in principle.

Suppose there is a composite integer N , and the problem to deal with is to find a nontrivial factor of N , meaning a divisor other than 1 and N .

For example, if

$$N = 15,$$

then the nontrivial factors are 3 and 5.

At a high level, Shor's algorithm turns factoring into a period-finding problem.

::... Shor's Algorithm

Pick an integer a such that $1 < a < N$ and a and N are co-prime, i.e., $\gcd(a, N) = 1$. Then consider the function

$$f(x) = a^x \bmod N.$$

This function is periodic: there exists a smallest positive integer r such that

$$a^r \equiv 1 \pmod{N}.$$

That integer r is called the order of a modulo N . Shor's algorithm uses a quantum computer to find this period r , and Once r is known, factors of N can be extracted. If r is even, then $a^r \equiv 1 \pmod{N}$ and this implies $a^r - 1 \equiv 0 \pmod{N}$. Factor the left-hand side:

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1).$$

So

$$(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}.$$

That means N divides the product. If neither factor is itself congruent to 0 $\pmod{N}$, then taking $\gcd(a^{r/2} - 1, N)$ and $\gcd(a^{r/2} + 1, N)$ often gives nontrivial factors of N .

::... Shor's Algorithm

This works provided:

- r is even, and
- $a^{r/2} \not\equiv -1 \pmod{N}$.

If either condition fails, another a can be chosen and the algorithm retries.

Example with $N = 15$ - Take $a = 2$ and compute powers modulo 15:

$$2^1 \equiv 2 \pmod{15}$$

$$2^2 \equiv 4 \pmod{15}$$

$$2^3 \equiv 8 \pmod{15}$$

$$2^4 \equiv 16 \equiv 1 \pmod{15}$$

So the period is $r = 4$. Since r is even, compute $2^{r/2} = 2^2 = 4$.

Then

$$\gcd(4 - 1, 15) = \gcd(3, 15) = 3$$

and

$$\gcd(4 + 1, 15) = \gcd(5, 15) = 5.$$

So we recover the factors 3 and 5.

::... Shor's Algorithm

This is the basic mechanism of Shor's algorithm, which is composed of two parts:

- Classical part handles:
 - choosing randomly an integer a , such that $1 < a < N$,
 - computing $d = \gcd(a, N)$,
 - If $d > 1$, a nontrivial factor was already found and the other one is N/d . If $d = 1$, a is coprime to N , and the quantum step is triggered.
- Quantum part performs efficient period finding for the function

$$f(x) = a^x \bmod N.$$

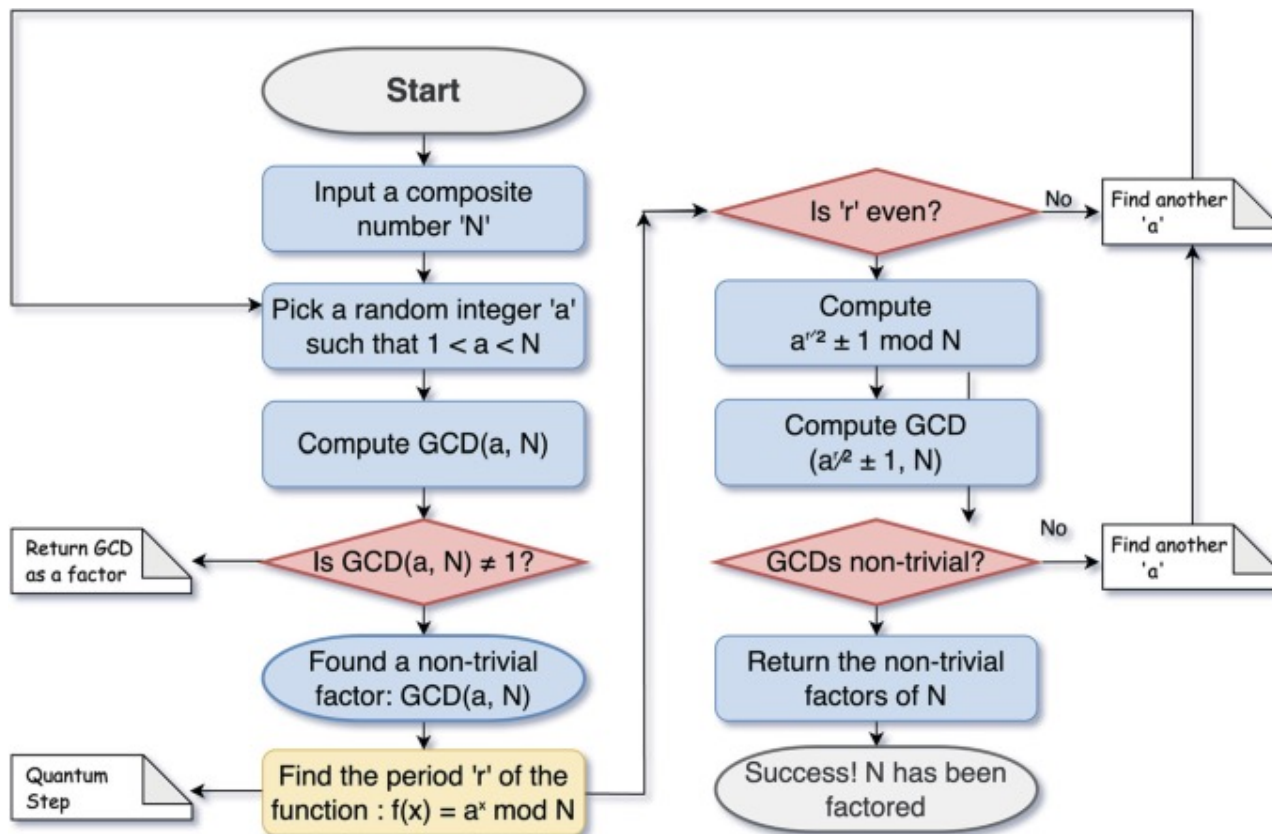
The quantum part is the part that gives the speedup and we want to find the order r such that

$$a^r \equiv 1 \pmod{N}.$$

The quantum computer does this by exploiting superposition of many x -values at once, compute $f(x)$ for all of them simultaneously, and then use the quantum Fourier transform to extract the period.

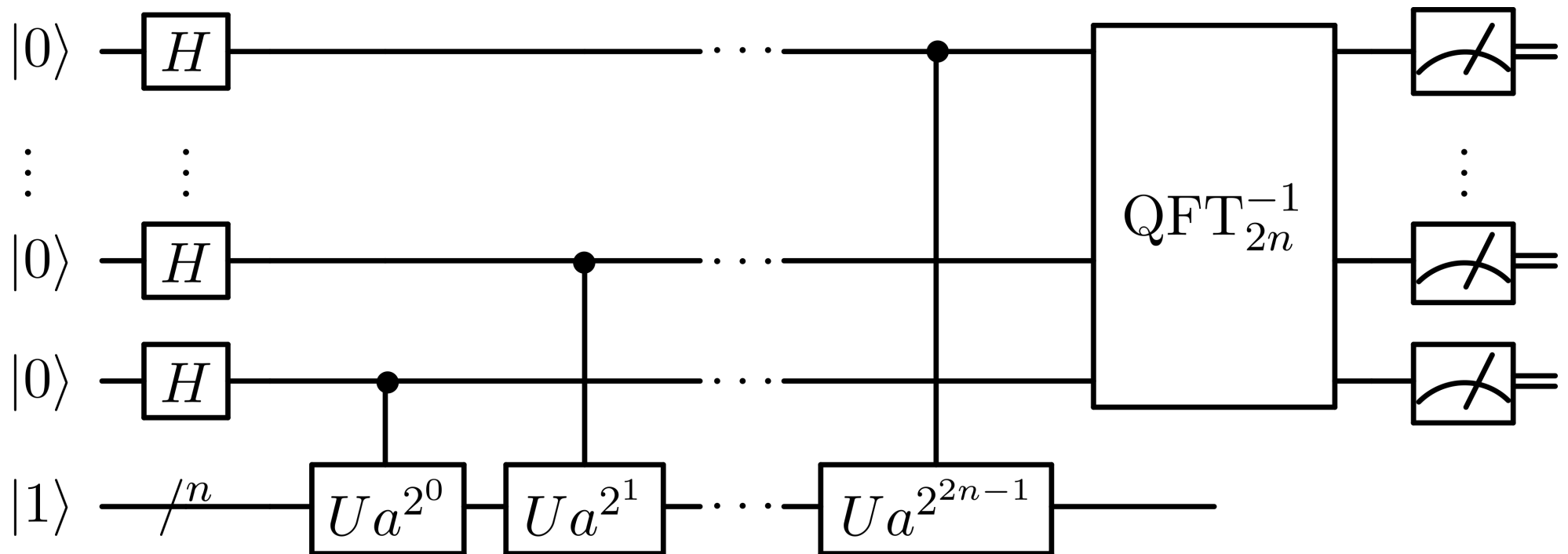
::... Shor's Algorithm

If r is odd, then go back to step 1 of the classical part of the algorithm. Compute $g = \gcd(N, a^{r/2} + 1)$ and if it is nontrivial, the other factor is N/d , and the algorithm terminates. Otherwise, go back to step 1 of the classical part.

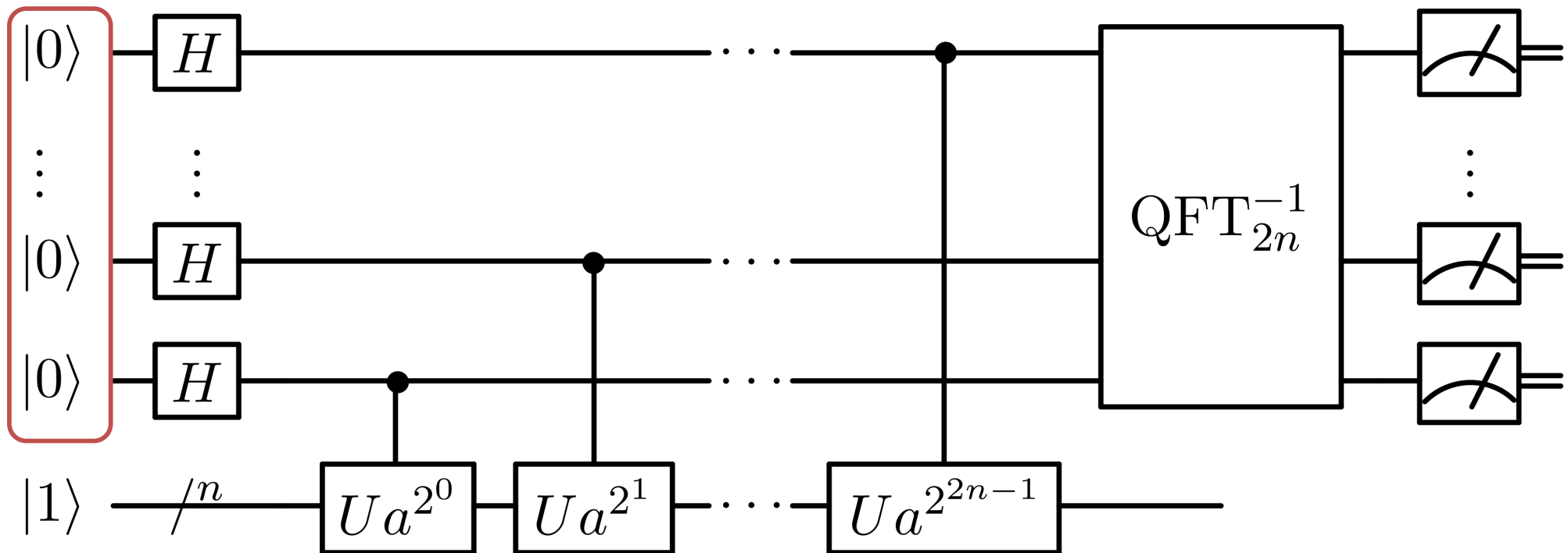


It has been shown that this will be likely to succeed after a few runs. In practice, a single call to the quantum order-finding subroutine is enough to completely factor N with very high probability of success if one uses a more advanced reduction.

... Shor's Algorithm



... Shor's Algorithm



The circuit estimates the period of the function

$$f(x) = a^x \bmod N,$$

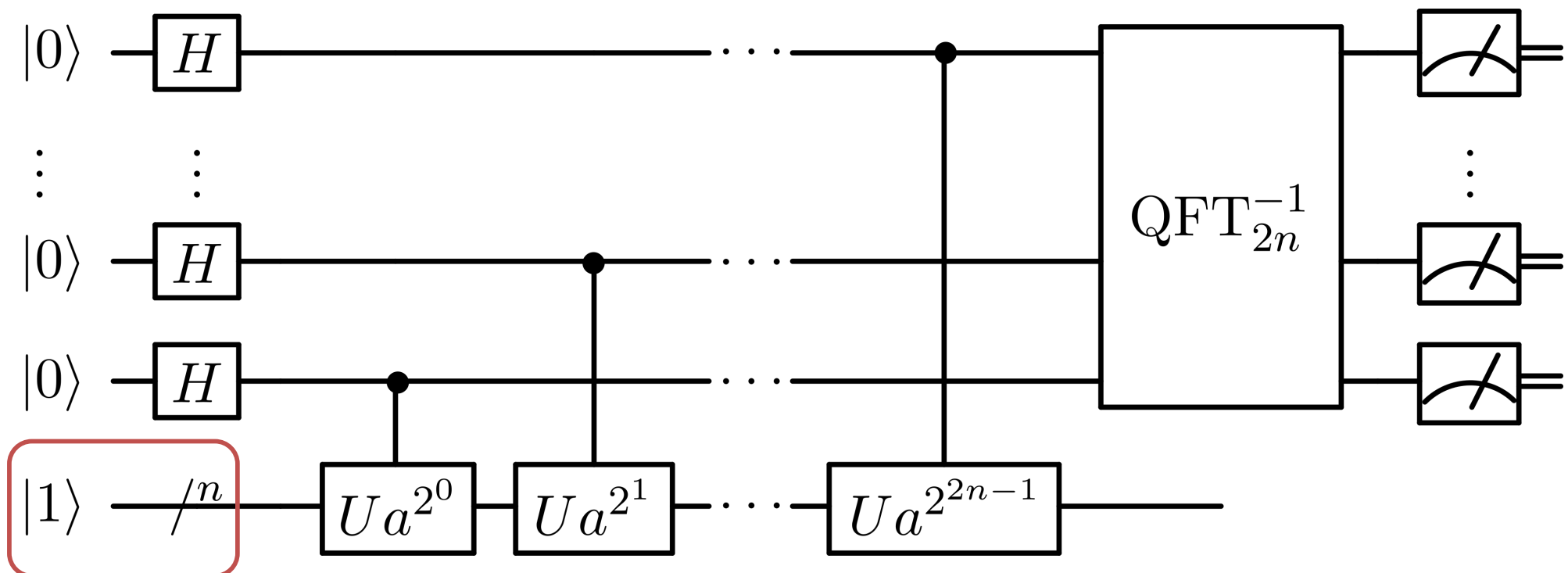
and takes two inputs: the first one holds the superposition over exponents x .

If N is the number to factor, the approach usually takes

$$Q = 2^t \text{ with } N^2 \leq Q < 2N^2,$$

so the first register has t qubits.

... Shor's Algorithm



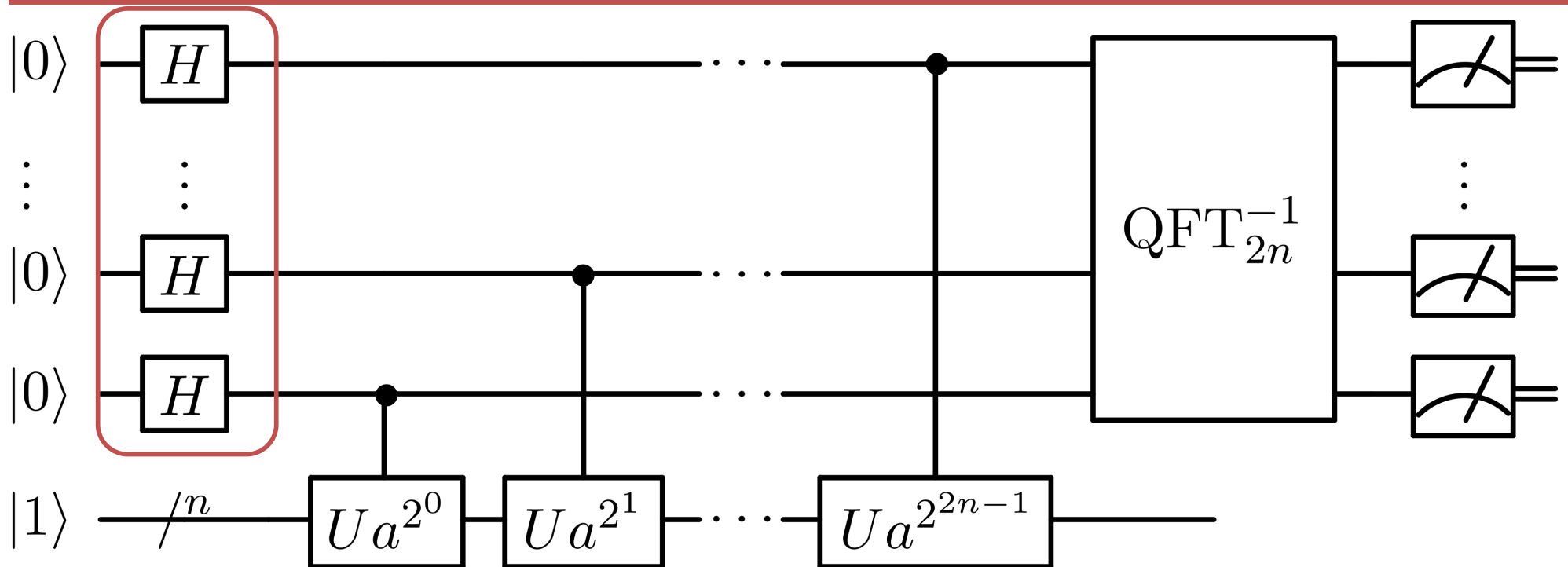
The second input stores modular values $a^x \bmod N$ and is made of n qubits where
 $n = \lceil \log_2 N \rceil$,

sometimes a few more in practical implementations. Initial state:

$$|0\rangle^{\otimes t} |1\rangle.$$

The work register, or the second input, is initialized to $|1\rangle$ because modular exponentiation will map it to $a^x \bmod N$.

... Shor's Algorithm



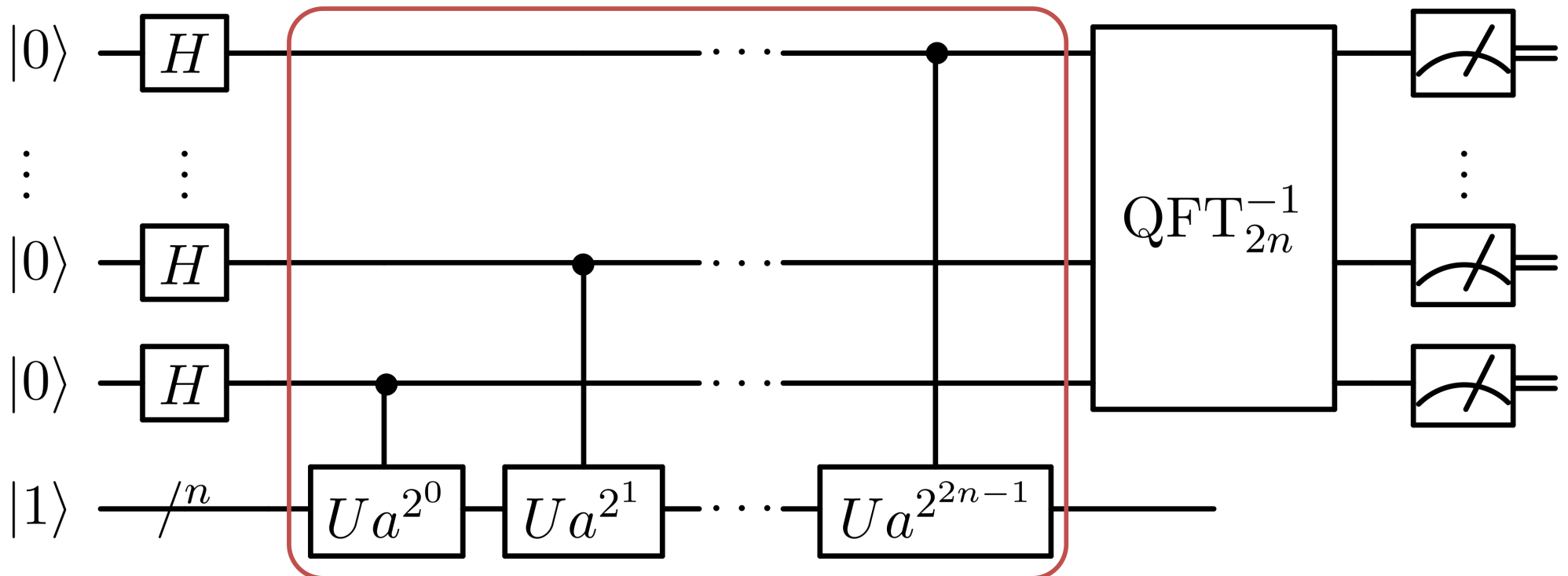
In the first step, a Hadamard gate is applied to each qubit in the first register:

$$H^{\otimes t} |0\rangle^{\otimes t} = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle.$$

So the full state becomes

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |1\rangle.$$

... Shor's Algorithm



The key unitary is $|x\rangle|y\rangle \mapsto |x\rangle|y \cdot a^x \bmod N\rangle$, and since $|y\rangle = |1\rangle$ we have that

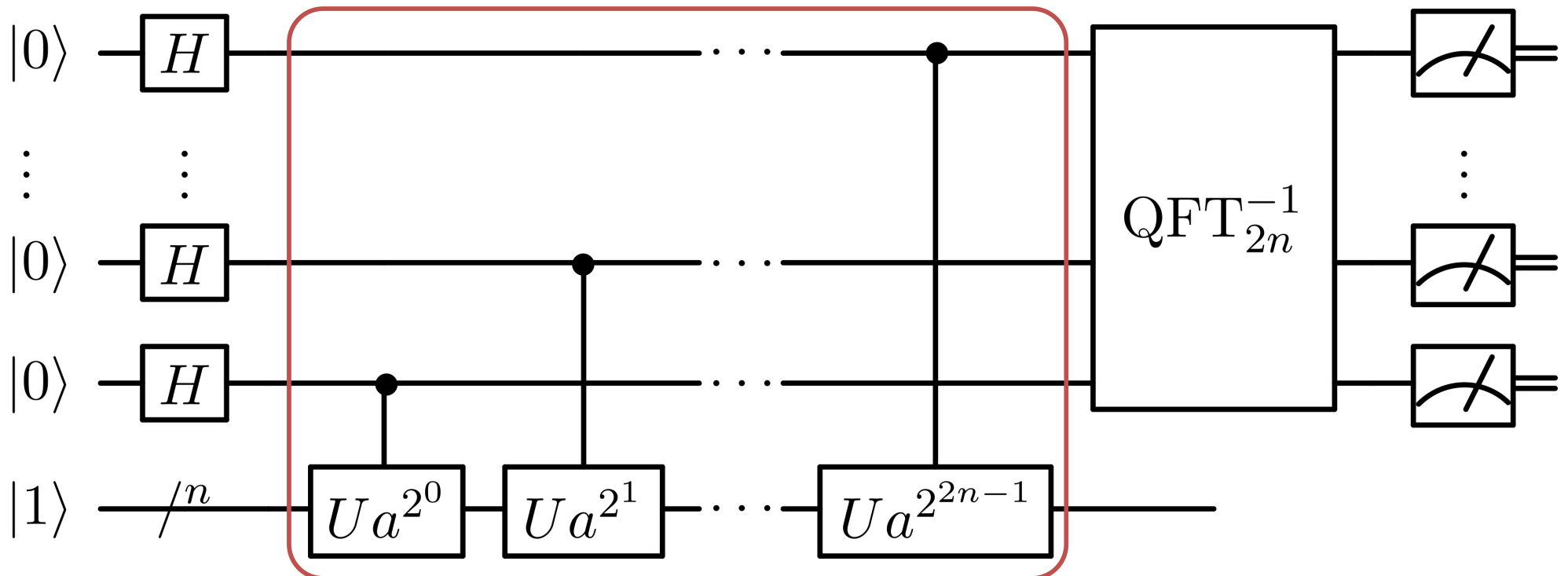
$$|x\rangle|1\rangle \mapsto |x\rangle|a^x \bmod N\rangle$$

This is implemented bitwise from the binary expansion of x . If

$$x = x_0 + 2x_1 + 2^2x_2 + \dots + 2^{t-1}x_{t-1}, \text{ then}$$

$$a^x \bmod N = \prod_{k=0}^{t-1} \left(a^{2^k} \bmod N \right)^{x_k} \bmod N.$$

... Shor's Algorithm



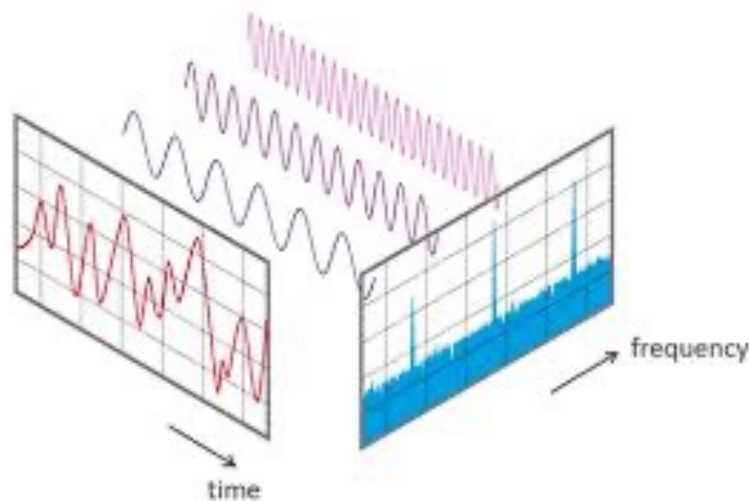
So the circuit applies, for each control qubit x_k , a controlled modular multiplication by the precomputed constant $a^{2^k} \bmod N$. That is why the circuit often appears as a ladder of controlled- $U_{a^{2^k}}$ gates. The state becomes

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |a^x \bmod N\rangle$$

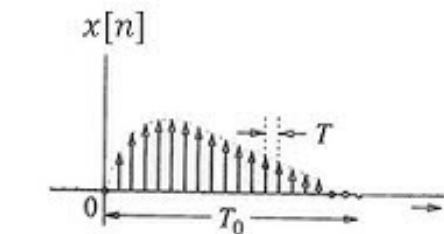
As $a^x \bmod N$ is periodic in r , the first register contains hidden phase data on r .

... Shor's Algorithm

One of the key building blocks used in quantum algorithms is the Quantum Fourier Transform (QFT).



The classical (discrete) Fourier Transform (FT) converts a finite sequence of numbers into another sequence of the same length, representing the strength and phase of different frequency components. It changes data from a description in terms of sampled values to a description in terms of oscillations.



f_s : Sampling rate, $T=1/f_s$

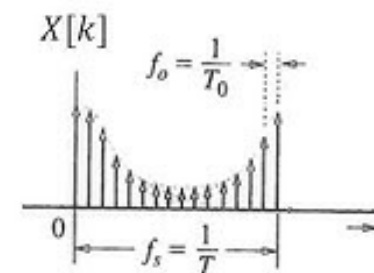
T_0 : Length of signal

f_o : Frequency resolution $f_o = 1/T_0$

F : Digital frequency $F = \frac{f_0}{f_s}$ cycles/samples

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-jk\Omega_0 n}$$

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{jk\Omega_0 n}$$



N : Number of samples $N = \frac{T_0}{T} = \frac{f_s}{f_0}$

$$\Omega_0 = \omega_0 T = 2\pi f_0 T = 2\pi \frac{f_0}{f_s} = 2\pi \frac{1}{N}$$

... Shor's Algorithm

For $N \in \mathbb{N}$, representing the length of the input of the transformation, let $\omega_N = e^{\frac{2\pi i}{N}}$. The Fourier transform $F_N : \mathbb{C}^N \mapsto \mathbb{C}^N$ is defined by

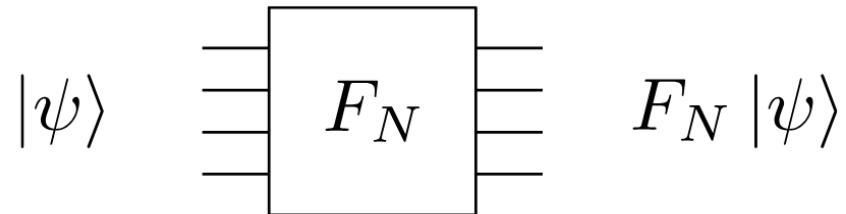
$$F_N = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & \omega_N & \cdots & \omega_N^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \cdots & \omega_N^{(N-1)(N-1)} \end{bmatrix}.$$

More concisely, $(F_N)_{j,k} = \omega_N^{jk} / \sqrt{N}$ for $j, k \in \{0, \dots, N-1\}$. The rows or columns of F_N are the Fourier modes, which are the fundamental, sinusoidal building blocks used to represent complex signals.

The QFT is the quantum counterpart of this transform and the basic idea is to perform the same operation but in a quantum state $|x\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle$. In this case, the output is another quantum state $|y\rangle = \sum_{j=0}^{N-1} \beta_j |j\rangle$ where:

$$\beta_j = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \alpha_k e^{\frac{2\pi i j k}{N}} = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \alpha_k \omega_N^{jk}$$

... Shor's Algorithm



Since these form an orthonormal basis, the Fourier transform F_N is a unitary operation, so that the Fourier transform can be considered as a quantum operation. Assuming that $N = 2^n$, the operation F_N acts on an n qubit state:

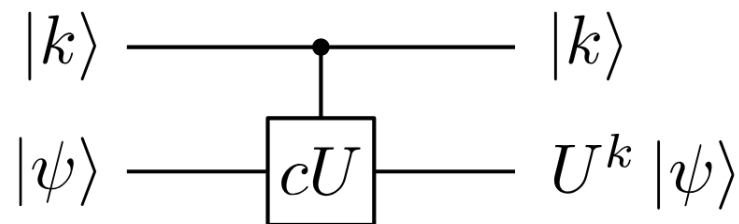
$$|\psi\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle \mapsto F_N |\psi\rangle = \sum_{j=0}^{N-1} \left(\frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega_N^{jk} \alpha_k \right) |j\rangle.$$

The Quantum Fourier Transform (QFT) achieves an exponential speedup over the classical Fast Fourier Transform (FFT) by operating on quantum superpositions. While FFT requires $O(N \log N)$ time for N data points, QFT implements the transformation using $O(n^2)$ or $O(n \log n)$ quantum gates (where $n = \log_2 N$ qubits), providing a polylogarithmic efficiency advantage in terms of input size N .

... Shor's Algorithm

A first important application of the quantum Fourier transform is quantum phase estimation. Assume access to a unitary U and eigenvector $|\psi\rangle$ such that $U |\psi\rangle = e^{2\pi i\theta} |\psi\rangle$ for some $\theta \in [0, 1)$. We can use the QFT to estimate the phase θ . The intuition behind this is that repeatedly applying U to $|\psi\rangle$ yields a “signal” $e^{2i\theta} |\psi\rangle$ that rotates with angular velocity θ .

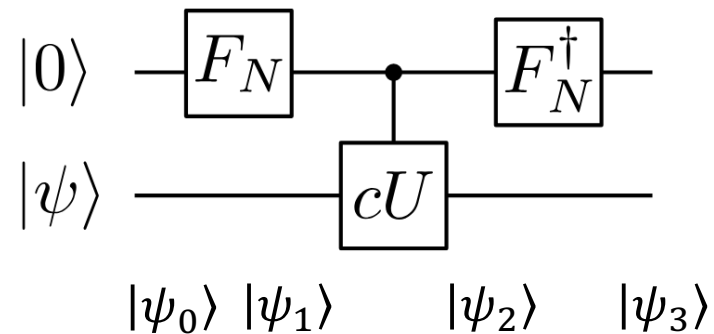
For some $N = 2^n$, we assume that θ is such that $N\theta$ is an integer. Consider the controlled version of U , represented by the following circuit:



where $k \in \{0, 1, \dots, N - 1\}$.

::... Shor's Algorithm

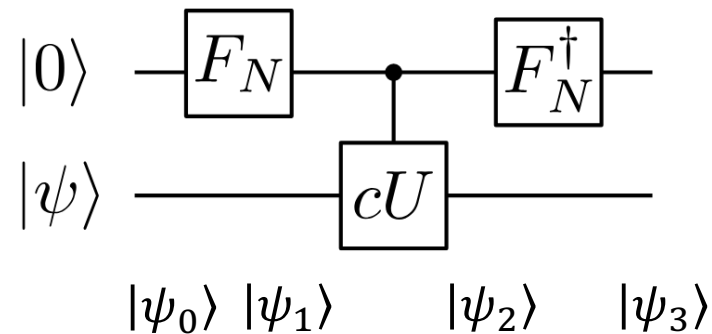
The circuit for quantum phase estimation is the following:



We can track the evolution:

::... Shor's Algorithm

The circuit for quantum phase estimation is the following:

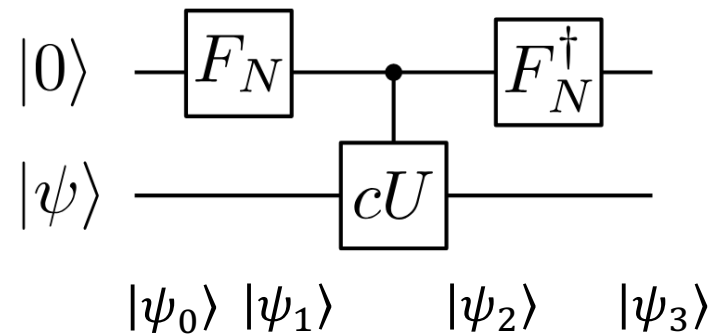


We can track the evolution:

1. $|\psi_0\rangle = |0^n\rangle |\psi\rangle$

::... Shor's Algorithm

The circuit for quantum phase estimation is the following:



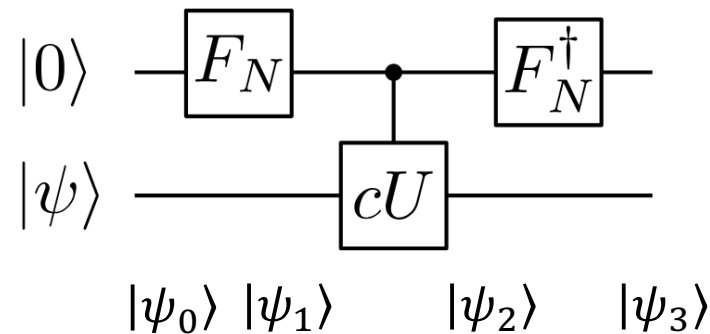
We can track the evolution:

1. $|\psi_0\rangle = |0^n\rangle |\psi\rangle$
2. The application of QFT produces an equal superposition of all computational basis states in the Fourier basis $|\psi_1\rangle$

$$= \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle |\psi\rangle$$

::... Shor's Algorithm

The circuit for quantum phase estimation is the following:



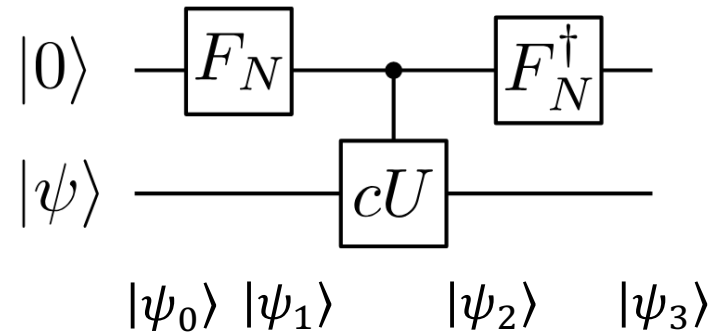
We can track the evolution:

1. $|\psi_0\rangle = |0^n\rangle |\psi\rangle$
2. The application of QFT produces an equal superposition of all computational basis states in the Fourier basis $|\psi_1\rangle$

$$= \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle |\psi\rangle$$
3. $|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle U^j |\psi\rangle = \left(\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{2\pi i \theta j} |j\rangle \right) |\psi\rangle$

::... Shor's Algorithm

The circuit for quantum phase estimation is the following:



We can track the evolution:

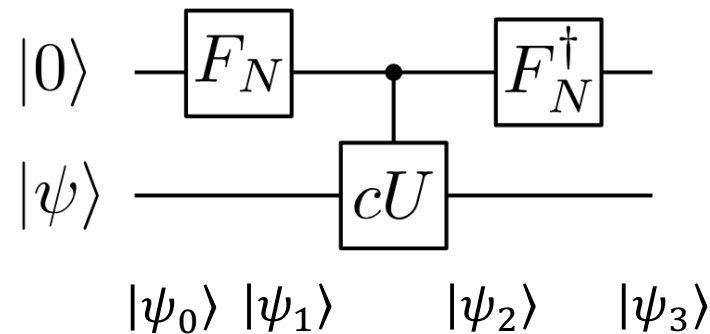
1. $|\psi_0\rangle = |0^n\rangle |\psi\rangle$
2. The application of QFT produces an equal superposition of all computational basis states in the Fourier basis $|\psi_1\rangle$

$$= \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle |\psi\rangle$$
3. $|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle U^j |\psi\rangle = \left(\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \boxed{e^{2\pi i \theta j}} |j\rangle \right) |\psi\rangle$

$$= \left(\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \omega_N^{(N\theta)j} |j\rangle \right) |\psi\rangle$$

... Shor's Algorithm

The circuit for quantum phase estimation is the following:



We can track the evolution:

1. $|\psi_0\rangle = |0^n\rangle |\psi\rangle$
2. The application of QFT produces an equal superposition of all computational basis states in the Fourier basis $|\psi_1\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle |\psi\rangle$
3. $|\psi_2\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} |j\rangle U^j |\psi\rangle = \left(\frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} \omega_N^{(N\theta)j} |j\rangle \right) |\psi\rangle$
4. The first register now corresponds to a simple Fourier mode $|\tilde{k}\rangle$ with $k = N\theta$. Applying the inverse Fourier transform yields the final state $|\psi_3\rangle = |N\theta\rangle |\psi\rangle$, so that by measuring we have the phase.

::... Shor's Algorithm

The complexity of phase estimation is typically dominated by the maximum number of times the unitary U has to be implemented, which is $N - 1$ times. If the phase $\theta \in [0, 1)$ does not have an exact n -bit expansion, then quantum phase estimation returns with high probability an n -bit approximation to θ .

::... Shor's Algorithm

The complexity of phase estimation is typically dominated by the maximum number of times the unitary U has to be implemented, which is $N - 1$ times. If the phase $\theta \in [0, 1)$ does not have an exact n -bit expansion, then quantum phase estimation returns with high probability an n -bit approximation to θ .

The quantum circuit for the Quantum Fourier Transform (QFT) is a structured sequence of Hadamard gates, controlled phase-rotation gates and a final SWAP gates to reverse qubit order. Usually written as CR_k or controlled- R_k , where

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}.$$

So:

- R_2 adds phase $e^{i\pi/2}$
- R_3 adds phase $e^{i\pi/4}$
- R_4 adds phase $e^{i\pi/8}$, etc.

... Shor's Algorithm

For qubits labeled from top to bottom as $q_0, q_1, \dots, q_{n-1}$, the QFT circuit proceeds like this:

- Apply H to q_0
- Apply controlled- R_2 , controlled- R_3 , ..., controlled- R_n from later qubits onto q_0
- Move to q_1
 - apply H
 - then controlled- R_2 , ..., controlled- R_{n-1}
- Continue this pattern for all the involved qubits.

Suppose the input basis state is

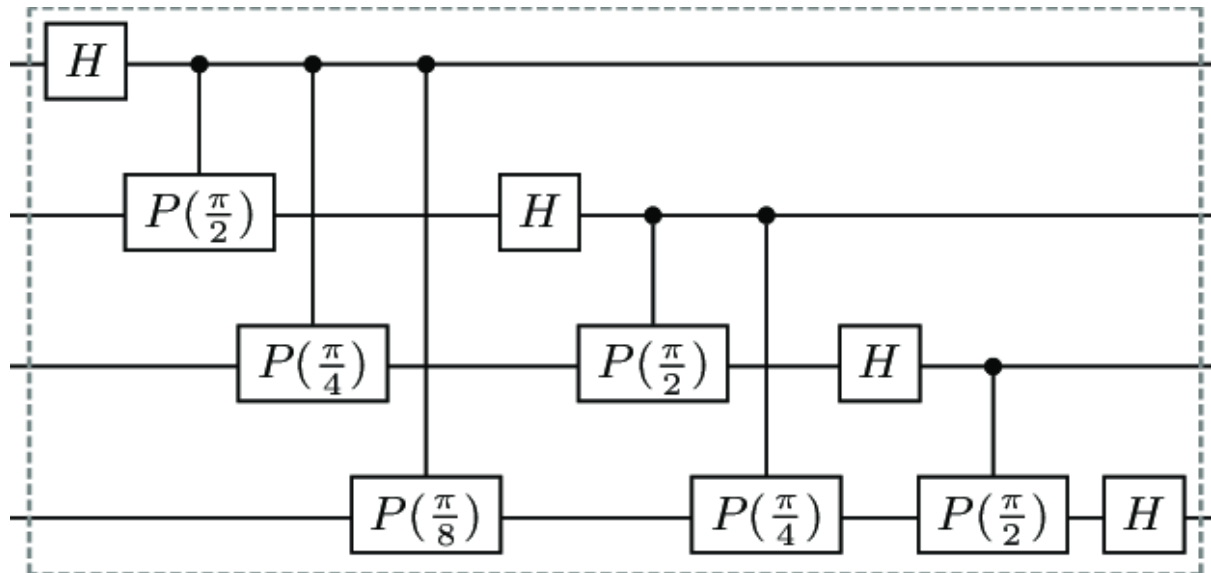
$$|x_1 x_2 \dots x_n\rangle.$$

The QFT can be factorized into a tensor product form:

$$\begin{aligned} QFT|x_1 x_2 \dots x_n\rangle = \frac{1}{2^{\frac{n}{2}}} & (|0\rangle + e^{2\pi i 0.x_n}|1\rangle)(|0\rangle + e^{2\pi i 0.x_{n-1}x_n}|1\rangle) \dots (|0\rangle \\ & + e^{2\pi i 0.x_1 x_2 \dots x_n}|1\rangle). \end{aligned}$$

::... Shor's Algorithm

Each qubit picks up a phase determined by the lower-significance bits of x . The Hadamards create superpositions, and the controlled phase gates add exactly those fractional binary phases.

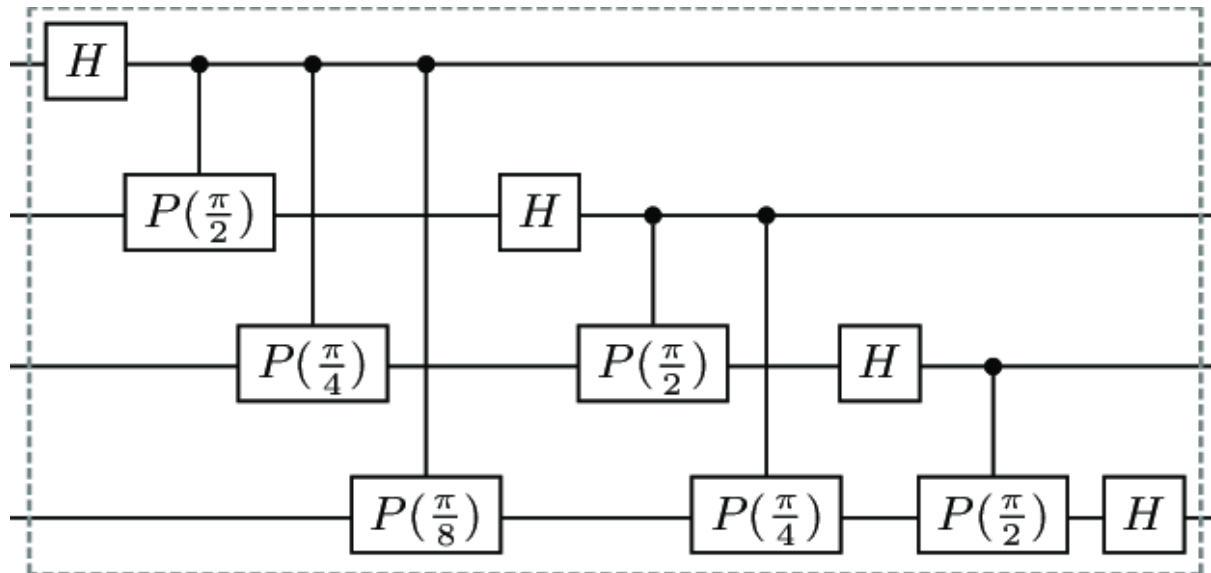


After all qubits are processed, reverse the qubit order with SWAPs

So the QFT circuit is essentially a triangular pattern of controlled phase gates followed by qubit reversal.

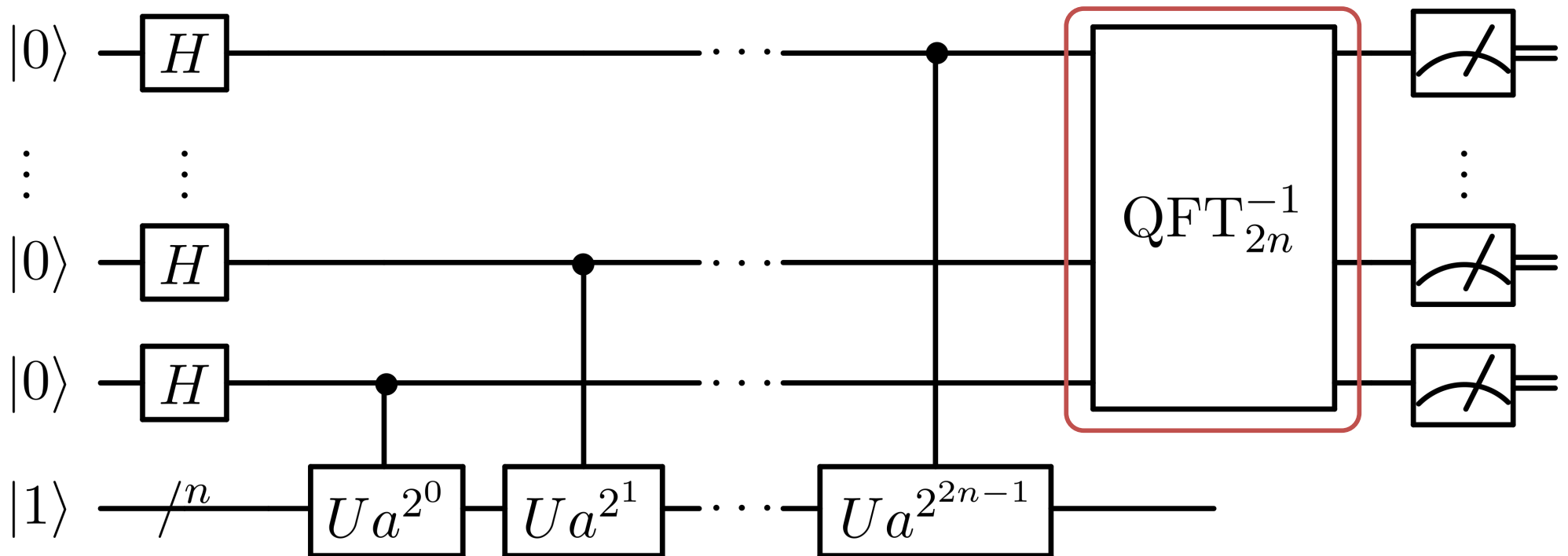
::... Shor's Algorithm

Each qubit picks up a phase determined by the lower-significance bits of x . The Hadamards create superpositions, and the controlled phase gates add exactly those fractional binary phases.



The inverse QFT is just the reverse circuit with conjugated phase gates: reverse the order of gates and replace each R_k with $R_k^\dagger$.

... Shor's Algorithm



Applying QFT^{-1} transforms periodic structure in the computational basis into peaks near multiples of

$$\frac{jQ}{r}, \quad j = 0, 1, \dots, r-1.$$

So after QFT^{-1} , measuring the first register yields some value m likely satisfying

$$\frac{m}{Q} \approx \frac{j}{r}$$

for a random integer j . This is exactly what lets the classical step recover r .